



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ, СОФИЯ

Факултет по компютърни системи и технологии

[TODO: department, exactly as the faculty writes it]

Алекс Иванов Цветанов

[TODO: dissertation title]

ДИСЕРТАЦИОНЕН ТРУД

за придобиване на образователна и научна степен „доктор“

[TODO: professional field and doctoral programme, from the enrolment order]

Научен ръководител:

[TODO: any second supervisor, or

[TODO: supervisor, with academic title]

delete this block]

София, **[TODO: year]**

СЪДЪРЖАНИЕ

СПИСЪК НА СЪКРАЩЕНИЯТА	6
УВОД	7
I. СЪСТОЯНИЕ НА ПРОБЛЕМА	10
1.1. Как един нов протокол на HTTP влиза в употреба	10
1.2. Обвързването на протокола със сокета	13
1.3. Къде се взема изборът при останалите сървъри	14
1.4. Какво струва един допълнителен слушател	18
1.5. Разпределяне на QUIC връзки между работни нишки	18
1.6. Библиотеки за уеб приложения на C++	22
1.7. Какво липсва в прегледаното	25
1.8. Цел, задачи и хипотези	26
1.8.1. Цел	26
1.8.2. Задачи	26
1.8.3. Хипотези	27
1.9. Изводи	28
II. ТЕОРЕТИЧНИ ОСНОВИ	30
2.1. Едновременност и паралелизъм	30
2.2. Корутини без собствен стек	30
2.2.1. Какво съдържа кадърът	31
2.2.2. Какво пресича точка на спиране	32
2.2.3. Кадър и стек: какво определя размера на всеки	32
2.2.4. Протоколът на изчакването	33
2.2.5. Симетрично прехвърляне	33
2.3. Границата на собственост върху кадъра	34
2.4. Съпоставяне срещу множество регулярни изрази	36
2.4.1. Последователното съпоставяне	36
2.4.2. Обединеният автомат	37
2.4.3. Какво е наследено и какво не се доказва наново	37

2.5.	Минимизация на слушащите дескриптори	38
2.5.1.	Предположенията поотделно	40
2.5.2.	Случаят UDP	41
2.5.3.	Ако непресичането отпадне	43
2.6.	Изводи	44
III.	АРХИТЕКТУРА	45
3.1.	Демултиплексиране на един слушащ дескриптор	45
3.1.1.	Задачата, поставена от изключващите се пътища	45
3.1.2.	Класификация по първия октет	46
3.1.3.	Прочитане с връщане вместо MSG_PEEK	47
3.1.4.	QUIC и границата на подхода	49
3.1.5.	Измерен резултат	49
3.1.6.	Ограничения на твърдението	51
3.2.	Разпределяне на QUIC връзки по идентификатор	51
3.2.1.	Защо хешът по четворка не работи за QUIC	52
3.2.2.	Отговорът е вътре във въпроса	53
3.2.3.	Кои пакети се препращат	54
3.2.4.	Сравнение с решението в ядрото	54
3.2.5.	Величината, която решава спора	54
3.3.	Разделяне на API и изгледи, и отложено получаване на данни	55
3.3.1.	Извикване без сокет	55
3.3.2.	Двата режима на извикване	56
3.3.3.	Защо това не е ранно изпращане	57
3.3.4.	Как се откриват неготовите полета	57
3.3.5.	Границата на сигурност	58
3.3.6.	Резервен път без JavaScript	58
3.3.7.	Известна граница на реализацията	58
3.3.8.	Състояние на измерването	58
3.4.	Животът на връзката и неговите срокове	59
3.4.1.	Прозорецът на класификацията е обхват, а не поредица от изходи	59
3.4.2.	Срокът следва това, върху което действа	60

3.4.3.	Обезвреждането трябва да значи „обезвредено и не се изпълнява в момента“	61
3.4.4.	Срокът при бездействие е декоратор, а не четири реализации	61
3.4.5.	Бездействие, а не срок на операция, и каква грешка вижда извикващият	62
3.4.6.	Какво е проверено и какво остава	62
3.5.	Опашката от таймери	63
3.6.	Какво архитектурата отказва да прави	64
3.7.	Изводи	65
IV.	РЕАЛИЗАЦИЯ	67
4.1.	Един интерфейс, четири механизма за вход и изход	67
4.1.1.	Една опашка от таймери вместо нишка на таймер	69
4.1.2.	Таймаут, който всеки backend пазеше и никой не спазваше	70
4.2.	Опасността при подновяване от вътрешността на <code>await_suspend</code>	71
4.2.1.	Формата на грешката	71
4.2.2.	Поправката е в наредбата, не в заключването	73
4.2.3.	Трите места	74
4.2.4.	Същата форма при отложените стойности	75
4.3.	Дефекти, открити при насочено фъзване	75
4.3.1.	Какво се фъзва и какво не	75
4.3.2.	<code>Range</code> : приемане на непълен вход и на обърнат интервал	77
4.3.3.	<code>НРАСК</code> : граница върху входа вместо върху изхода	77
4.3.4.	Размер на парче: препълване и липса на таван	78
4.3.5.	Рамкиране по <code>HTTP/1.1</code> : несъгласие, а не контрабанда	78
4.3.6.	Процентно декодиране: приет невъзможен <code>escape</code>	80
4.3.7.	Общата форма на десетте дефекта	81
4.3.8.	Какво фъзването не намери	82
4.3.9.	Как това остава поправено	82
4.4.	Реализации на протоколите	83
4.4.1.	<code>HTTP/1.1</code> : собствен анализатор, защото там живеят правилата . . .	83
4.4.2.	<code>HTTP/2</code> : собствено рамкиране, чужд <code>НРАСК</code>	84
4.4.3.	<code>HTTP/3</code> : две машини на състоянията, които не притежават нищо . .	85

4.5.	Типово безопасни параметри на маршрута	86
4.6.	Реализация на отложената стойност	87
4.6.1.	Събиране без деклариране	88
4.6.2.	Границата на екраниране	88
4.6.3.	Средата на страницата и отказът	89
4.7.	Изводи	90
V.	МЕТОДИКА НА ИЗСЛЕДВАНЕТО	92
5.1.	Какво трябва да произведе методиката	92
5.2.	Сравнени подходи	92
5.3.	Инструменти за натоварване	95
5.4.	Генератор за кампанията на три платформи	95
5.5.	Съгласувано пропускане	96
5.6.	Квантуване на таймера и корекция на темпа	98
5.7.	Едно рамо и друго рамо от един и същ двоичен файл	99
5.8.	Средата като контролирана променлива	100
5.9.	Конфигурацията под Windows и какво тя не позволява	101
5.10.	Правила за допустимост, обявени предварително	102
5.11.	Когато отказите не са разпределени еднакво между рамената	108
5.12.	Праг за отказ по натоварване, а не общо за проекта	109
5.13.	Ред на изпълнение	109
5.14.	Статистика	109
5.15.	Производна величина назовава изпълненията, от които идва	110
5.16.	Кога обща за двете рамена цена наистина се съкращава	111
5.17.	Разлагане на измерена величина на съставки	111
5.18.	Изводи	112
VI.	РЕЗУЛТАТИ И АНАЛИЗ	114
6.1.	Средата и какво тя позволява	114
6.2.	Една кампания, три проекта, и защо се обработват поотделно	115
6.3.	Допустимост: колко изпълнения бяха отхвърлени	116
6.4.	X1: цената на класификацията след приемането	118
6.5.	Разпределението на латентността по цялата си дължина	120

6.6. Процесорно време и памет	122
6.7. Правилото за докладваемост върху процесорното време	122
6.8. Разпределението на p99.9 е двумодално и затова не се използва	123
6.9. Мащабиране по работни нишки	124
6.10. Размер на отговора	125
6.11. Дължина на опашката за приемане	126
6.12. Преброяване на слушащите дескриптори	126
6.13. Какво тази кампания не измерва	130
6.14. Запланувани измервания под Linux и macOS	130
6.15. Изводи	132
VII. ПРИЛОЖЕНИЕ И ВАЛИДАЦИЯ	133
7.1. Примерното приложение	133
7.1.1. Маршрутите	134
7.1.2. Услуги и модели	135
7.1.3. Каналът за известия и връзката му с HTTP маршрутите	135
7.1.4. Middleware	136
7.2. Трите твърдения, проверени в код	136
7.3. Отделни примери за отделни свойства	139
7.4. Три изпълними проверки	140
7.5. Ограничения на тази проверка	142
7.6. Изводи	142
ЗАКЛЮЧЕНИЕ	144
СПРАВКА ЗА ПРИНОСИТЕ	149
ПУБЛИКАЦИИ ПО ДИСЕРТАЦИЯТА	153
ЛИТЕРАТУРА	155

СПИСЪК НА СЪКРАЩЕНИЯТА

Списъкът съдържа съкращенията, използвани в текста, и само тях. Имената на системни извиквания, на функции и на типове не са съкращения и не се превеждат; те се дават с шрифт за код на мястото на употребата им.

Съкращение	Значение
ALPN	Application-Layer Protocol Negotiation, договаряне на протокола на приложно ниво в рамките на TLS ръкостискането [1]
API	Application Programming Interface, програмен интерфейс
BCa	Bias-corrected and accelerated, вид бутстрап доверителен интервал [2]
eBPF	Extended Berkeley Packet Filter, механизъм за изпълнение на проверени програми в ядрото на Linux
HPACK	Компресия на заглавни полета в HTTP/2 [3]
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HTTP над TLS
IOCP	Input/Output Completion Ports, механизъм за вход и изход по завършване в Windows
IP	Internet Protocol
JSON	JavaScript Object Notation
MTU	Maximum Transmission Unit, най-голям размер на предаваната единица
QPACK	Компресия на заглавни полета в HTTP/3 [4]
QUIC	Транспортен протокол над UDP с вградена криптография [5]
RFC	Request for Comments, поредица от документи на IETF
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UDP	User Datagram Protocol
XSS	Cross-Site Scripting, вид уязвимост при вграждане на неекранирани данни

Термините, оставени на английски в текста по правилата за терминология, не са съкращения и затова не влизат тук. Такива са *handler*, *middleware*, *callback*, *stream*, *backend* и *endpoint*.

УВОД

Актуалност

Уеб сървърът вече не обслужва един протокол. Той обслужва три, и трите съществуват едновременно, а не последователно: HTTP/1.1 остава необходим за съвместимост, HTTP/2 носи мултиплексиране върху една TCP връзка, а HTTP/3 премества транспорта върху UDP, за да премахне блокирането на редицата в самия TCP [4]. Никой от трите не измества останалите, защото всеки от тях е задължителен за различна част от клиентите.

Това не е преходно състояние, а свойство на начина, по който протоколите се въвеждат в употреба. Нов протокол се разпространява чрез договаряне и чрез обявяване на алтернативна услуга [6], тоест през период, в който старият продължава да работи. Периодите се припокриват, и резултатът е, че съвместното обслужване на три протокола е нормалното състояние на един сървър, а не изключение.

Съществуващите реализации се справят с това чрез умножаване. Криптиран и некриптиран трафик изискват отделни слушащи сокети, защото изборът на протокол е обвързан с избора на сокет. Всеки допълнителен слушащ сокет носи не само дескриптор, а и конфигурационна повърхност: път до сертификат, правило в защитната стена, ред в наблюдението.

Отделно от това QUIC внася задача, каквато TCP не поставя. Връзката преживява смяна на адреса на клиента по замисъл, докато разпределянето на входящите дейтаграми между работни нишки се извършва по хеш върху адресите. Двете не си съответстват, и разминаването е структурно, а не рядък случай. Съществуващото решение работи в ядрото и не е налично на всички платформи.

Актуалността на настоящата работа следва от тези две наблюдения. Първото е въпрос за архитектура: може ли изборът на протокол да бъде отделен от избора на сокет, и на каква цена. Второто е въпрос за преносимост: може ли разпределянето на QUIC връзки да се извърши без зависимост от механизъм, който съществува само на една платформа.

Обект и предмет на изследването

Обект на изследването е библиотека за изграждане на уеб приложения на C++, обслужваща HTTP/1.1, HTTP/2, HTTP/3 и WebSocket.

Предмет на изследването е броят и организацията на слушащите дескриптори при едновременно обслужване на няколко протокола, както и цената на решенията, които този брой намаляват.

Извън предмета остават: производителността на самия синтактичен анализ, компресията на заглавните полета, и всичко, което не се променя при добавяне или премахване на протокол.

Къде са целта, задачите и хипотезите

Те не са тук, и това е умишлено.

Формулирането на цел преди прегледа на състоянието на проблема поставя заключението преди основанията му. Целта на настоящата работа произтича от това, което глава I установява за съществуващите реализации, тоест може да бъде формулирана честно едва след него. Затова целта, задачите и хипотезите са в края на глава I, след прегледа, а не преди него.

Структура на изложението

Изложението е подредено така, че всяка глава да зависи само от предходните.

Глава I установява състоянието на проблема. Тя описва как съществуващите сървъри организират слушащите си сокети, и го прави чрез позоваване на тяхната документация, а не чрез твърдение. Завършва с целта, задачите и хипотезите.

Глава II излага теоретичните основания: модела на изпълнение с корутини без стек, алгоритъма за съпоставяне срещу множество шаблони, и формалната постановка на минимизацията на слушащите дескриптори. Последната е предложение с доказателство, а не описание.

Глава III описва архитектурата: демултиплексирането над TCP, разпределянето на QUIC връзки по идентификатор, и разделянето на API от изгледи заедно с двата режима на получаване на данни.

Глава IV описва реализацията, включително един клас грешки, допуснат три пъти, чиято поправка е промяна на интерфейса, а не на кода.

Глава V определя методиката на измерване. Тя е подредена около начините, по които измерването на сървъри може да бъде тихо погрешно, тъй като неправилно проведеното измерване не отказва да работи, а произвежда правдоподобни числа.

Глава VI прилага методиката и представя резултатите.

Глава VII описва примерно приложение, изградено с библиотеката, което служи за проверка на използваемостта на интерфейсите ѝ.

Заклучението обобщава установеното, а справката за приносите разделя приносите по вид и посочва за всеки от тях с какво е обоснован.

Бележка за оформлението. Числата, цитирани в текста, не се преписват на ръка. Те се генерират от измерванията и се вмъкват по име; стойност, за която измерване липсва, се отпечатва като видим маркер, а не като правдоподобно число. Това е описано в глава V и е причината някои места в текста да съдържат такъв маркер, докато кампанията не е проведена.

I. СЪСТОЯНИЕ НА ПРОБЛЕМА

Главата установява какво съществуващите реализации правят и защо, за да може целта на работата да бъде формулирана като следствие, а не като предпоставка. Тя завършва с целта, задачите и хипотезите.

Редът е следният. Първо се обосновава защо трите протокола на HTTP се обслужват едновременно, защото от това следва, че цената на едновременното обслужване е постоянна, а не временна. После се установява къде съществуващите сървъри вземат решението за протокола и какво струва това място. След това се разглежда втората задача, която QUIC поставя, а TCP не поставя. Накрая се посочва какво липсва в прегледаното, и оттам следват целта и задачите.

1.1. Как един нов протокол на HTTP влиза в употреба

Уводът твърди, че трите протокола съществуват едновременно. Твърдението се обосновава тук, защото цялата глава зависи от него: ако едновременното обслужване беше преходно състояние, цената му би била временна и не би оправдала архитектурно решение.

Разликата между версиите на HTTP не е само в кодирането на съобщенията. Тя е и в това как клиентът разбира, че може да използва по-новата версия. Начинът на въвеждане е различен и за трите версии, и точно той определя дали по-старата може да бъде изключена.

HTTP/1.1 не се договаря. Той е подразбиращото се съдържание на TCP връзка към порт 80. Клиентът изпраща ред на заявка и очаква отговор; няма стъпка, на която да се уговори нещо друго [7]. Тъкмо това го прави невъзможен за изключване: липсва механизъм, чрез който сървърът да съобщи на клиент, говорещ само HTTP/1.1, че трябва да говори друго. Единственото, което сървърът може да направи, е да не отговори.

HTTP/2 се договаря по два начина, и единият беше изоставен. Над TLS изборът се прави от разширението ALPN: клиентът изброява в първото си съобщение протоколите, които може да говори, а сървърът избира един от тях и го връща в отговора си [1]. Токенът за HTTP/2 е h2 [8]. Договарянето не струва допълнителен обмен, защото се пренася вътре в ръкостискането, което така или иначе се извършва.

Над некриптирана връзка RFC 7540 определя втори път: заявка по HTTP/1.1 с поле

Upgrade и токен h2c, на която сървърът отговаря със смяна на протокола [8]. RFC 9113, който заменя RFC 7540, определя този път като изоставен [9]. Остава стартирането с предварително знание: клиентът започва направо с преамбюла на HTTP/2, без да пита [9].

Следствието има пряко отношение към настоящата работа. Некриптираната връзка вече няма механизъм за договаряне. Сървър, който иска да обслужва HTTP/1.1 и HTTP/2 без TLS на един и същ порт, е длъжен да ги различи по съдържанието на първите пристигнали байтове, защото няма какво друго да ги различи. Механизмът за това е предмет на раздел 3.1; тук е важно откъде идва необходимостта от него, а тя идва от самия стандарт.

HTTP/3 не може да бъде поискан пръв. HTTP/3 работи над QUIC, а QUIC работи над UDP [4]. Клиент, който вече е отворил TCP връзка, не може да премине по нея към HTTP/3: транспортът е друг, тоест смяна в рамките на връзката не съществува. Затова RFC 9114 определя откриването на крайна точка за HTTP/3 чрез обявяване на алтернативна услуга: сървърът съобщава в отговор по HTTP/1.1 или HTTP/2, че същият origin е достъпен и по h3, а клиентът може да опита [4].

Механизмът на това обявяване е Alt-Svc [6]. Три негови свойства решават въпроса за съвместното съществуване:

1. Обявата пътува по вече съществуваща връзка. Тоест за да съществува крайна точка за HTTP/3, трябва да съществува и крайна точка над TCP, която да я обяви.
2. Обявата е указание, а не пренасочване. Origin остава същият, а клиентът не е длъжен да използва алтернативата [6].
3. Обявата има срок на валидност, след който клиентът се връща към origin, и трябва да може да се върне и когато опитът към алтернативата е неуспешен [6].

Тоест крайната точка над TCP не може да бъде изключена след обявяването. Тя не е само съвместимост със стари клиенти; тя е част от механизма, чрез който новият протокол изобщо става достъпен, и остава резервният път, когато алтернативата е недостижима.

WebSocket показва другата половина на сметката. WebSocket се договаря вътре в съществуваща връзка по HTTP/1.1: клиентът изпраща заявка с поле Upgrade, а сървърът отговаря със смяна на протокола [10]. Тоест той е четвърти протокол, обслужван от същия слушател, и не добавя нито един слушащ сокет.

Сравнението с HTTP/3 е поучително. Двата се различават не по сложност, а по

това къде се извършва договарянето. Договаряне вътре във връзката не струва слушател. Договаряне извън връзката, каквото е обявяването чрез Alt-Svc, струва слушател, защото алтернативната услуга е достъпна по друг транспорт и следователно през друг сокет. Цената, която тази глава брои, е цената на второто, и WebSocket е контролният случай, който показва, че тя не се дължи на броя на протоколите сам по себе си.

Криптирането не е избор при два от трите. QUIC изисква TLS 1.3 без изключение [11], тоест некриптиран HTTP/3 не съществува. Същевременно порт 80 продължава да се използва, най-малкото за пренасочване към HTTPS: Caddy прави точно това по подразбиране [12].

Оттук следва минималният набор на един съвременен публичен сървър, без нито едно допълнително изискване: некриптиран TCP, криптиран TCP, криптиран UDP. Три различни комбинации от транспорт и криптиране, всяка от които съществува по документирана причина, а не по инерция. Начините на започване са събрани в таблица 2.

Таблица 2. Как клиентът научава, че може да използва даден протокол

Протокол	Транспорт	Начин на започване	Източник
HTTP/1.1	TCP	без договаряне; подразбиращото се съдържание на връзката	[7]
HTTP/2, h2	TCP с TLS	ALPN вътре в ръкостискането, без допълнителен обмен	[1, 8]
HTTP/2, h2c	TCP без TLS	Upgrade от HTTP/1.1, определен като изоставен; остава предварително знание	[8, 9]
HTTP/3, h3	QUIC над UDP	обявяване чрез Alt-Svc по друга връзка, после ALPN в ръкостискането	[6, 4]

Решението за протокола вече се взема след приемането на връзката. Едно наблюдение свързва този раздел със следващия. ALPN е разширение на TLS и пътува в първото съобщение на клиента, тоест сървърът научава кой протокол ще се говори едва след като е приел връзката и е прочел първите байтове от нея [1]. Всеки сървър, който обслужва h2 и http/1.1 на един и същ порт, вече взема решение за протокола след асерт.

Отлагането на решението следователно не е нов вид механизъм. То е разширяване на обхвата на механизъм, който вече е задължителен за криптирания случай. Настоящата

работа прилага същия ред на действията и към избора между криптирано и некриптирано, вместо да го спира на границата на TLS.

1.2. Обвързването на протокола със сокета

Централното наблюдение е просто и се проверява по документация, а не по измерване: при конвенционалните сървъри изборът на протокол е обвързан с избора на слушащ сокет.

Най-ясният документиран случай е nginx. Директивата `listen` приема параметър `ssl`, който указва, че всички връзки, приети на този порт, работят в режим SSL [13]. Документацията показва и прякото следствие: сървър, обслужващ едновременно HTTP и HTTPS, се конфигурира с две директиви `listen`, а не с една [14].

```
1 server {  
2     listen          80;  
3     listen          443 ssl;  
4     server_name     www.example.com;  
5     ssl_certificate  www.example.com.crt;  
6     ssl_certificate_key www.example.com.key;  
7 }
```

Листинг 1: Един сървър, два слушащи сокета, по документацията на nginx

Бележката в същата документация, че параметърът позволява „по-компактна конфигурация“ за сървър, обслужващ и двата вида заявки, се отнася за блока `server`, а не за броя на сокетите. Компактното е, че един блок обслужва двете, а не че ги обслужва от един сокет. Разграничението има значение, защото изречението често се цитира като твърдение за обратното.

Третият протокол добавя трета директива. Параметърът `quic`, наличен от версия 1.25.0, конфигурира порта да приема QUIC връзки [13], и този порт е над UDP.

Изборът се е местил веднъж, и посоката е показателна. Документацията на nginx отбелязва, че преди версия 0.7.14 SSL не е могло да се включва изборително за отделни слушащи сокети: включвал се е за целия сървър чрез отделна директива, което е правело невъзможно конфигурирането на един сървър едновременно за HTTP и за HTTPS [14].

Тоест единицата на решението вече е била премествана веднъж, от блока `server` към директивата `listen`. Преместването е направило конфигурацията по-изразителна, но

не е премахнало обвързването, а го е свалило с едно ниво. Настоящата работа предлага следващата стъпка в същата посока, от слушателя към отделната връзка, и разглежда какво тя струва.

Единицата в конфигурацията и единицата в ядрото не съвпадат. Директивата `listen` приема и параметър `reuseport`, който указва създаването на отделен слушащ сокет за всеки работен процес, така че ядрото да може да разпределя входящите връзки между процесите [13]. Един ред в конфигурацията отговаря на толкова дескриптора, колкото са работните процеси.

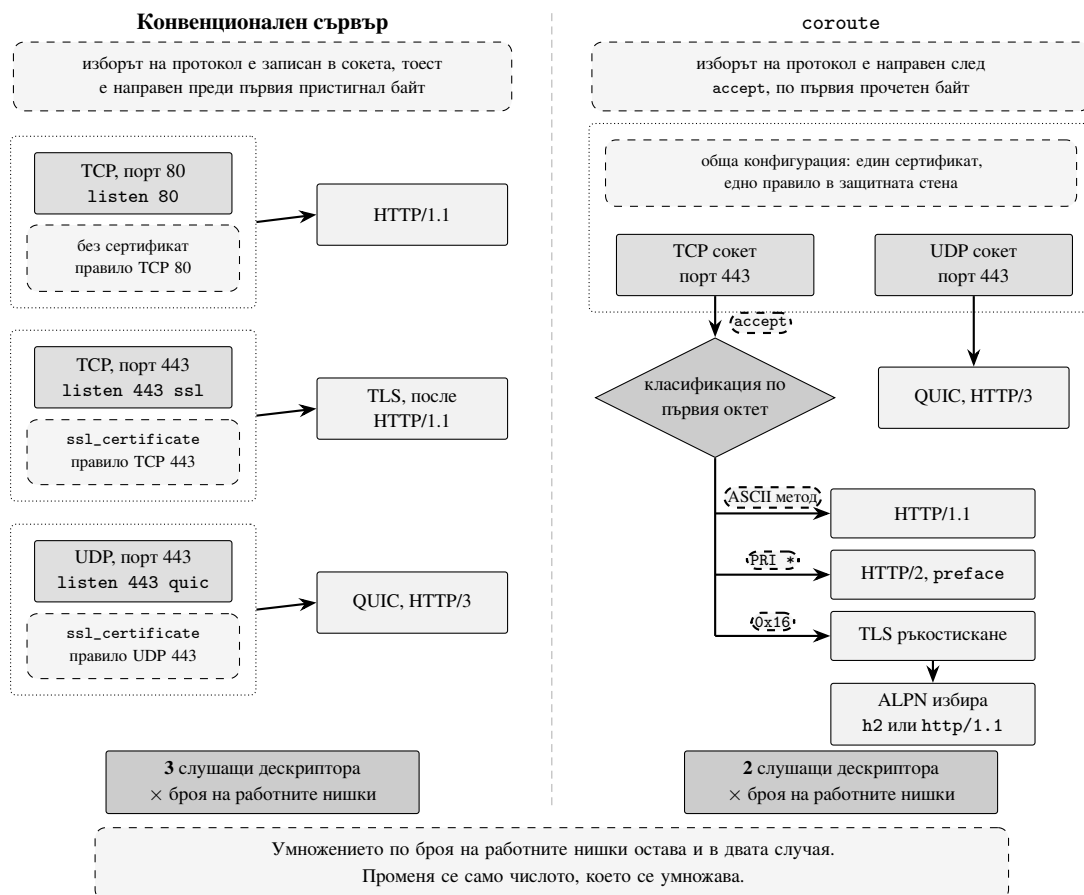
Това има значение за формалната постановка в раздел 2.5. Броят на слушащите дескриптори е произведение, а не сбор. Множителят, който брои протоколите, се умножава по броя на работните нишки, тоест намаляването му с единица премахва по един дескриптор на нишка, а не един дескриптор общо.

Виртуалният хост също се закача за слушателя. Параметърът `default_server` на директивата `listen` прави блока подразбиращ се за дадената двойка адрес и порт [13]. Тоест изборът измежду блоковете `server` се извършва в рамките на една двойка адрес и порт, а не глобално.

Следствието е практическо. Конфигурация, която на пръв поглед е свойство на името на хоста, в действителност виси на слушателя, и добавянето на слушател я задължава да бъде повторена или наследена. Същото се вижда и при IIS, където името на хоста е поле в стойността на `bindingInformation`, тоест част от описанието на самия `binding` [15].

1.3. Къде се взема изборът при останалите сървъри

Прегледът по-долу е по документацията на всеки от сървърите, а не по наблюдение върху работещ процес. Твърди се само това, което съответната документация казва. Там, където страницата не отговаря на въпроса, това се отбелязва, вместо да се допълни по аналогия с останалите редове. Установява се как производителят описва избора, а не как ядрото го изпълнява; второто изисква друг вид проверка и не е правено тук. Двата модела на слушане са съпоставени на (Фиг. 1).



Фигура 1. Двата модела на слушане. Вляво: конвенционален сървър, при който всеки протокол получава собствен слушащ сокет, а пунктираната рамка огражда сокета заедно с неразделната му конфигурация. Изборът е направен преди пристигането на първия байт. Вдясно: coroute, при който един TCP сокет и един UDP сокет са достатъчни, а протоколът се разпознава по първия октет след accept, като при TLS изборът между h2 и http/1.1 се доуточнява от ALPN.

Таблица 3. Единицата, в която се записва изборът на протокол

Сървър	Единица	Избор на TLS	Избор на QUIC
nginx	директива <code>listen</code>	параметър <code>ssl</code> на порта	параметър <code>quic</code> , отделна директива над UDP
Angie	директива <code>listen</code>	параметър <code>ssl</code> на порта	параметър <code>quic</code> , в синтаксиса алтернатива на <code>http2</code>
H2O	запис <code>listen</code>	атрибут <code>ssl</code> на записа	<code>type: quic</code> , отделен запис, свързващ UDP порт
Caddy	адрес на сайта	схемата в адреса	не е проверено
IIS	елемент <code>binding</code>	атрибут <code>protocol: http</code> или <code>https</code>	флаг в <code>sslFlags</code> на <code>HTTPS binding-a</code>

Таблицата дава единицата. Отделните редове заслужават повече от един ред, защото цената на избора не е еднаква при петте.

Angie. Синтаксисът изброява `ssl` и `quic` като параметри на самата директива `listen`, при което `http2` и `quic` са алтернативи една на друга [16]. Формата повтаря тази на `nginx` до параметъра.

Съвпадението обаче не е доказателство за нищо. `Angie` е разклонение на `nginx`, тоест синтаксисът е наследен, а не получен независимо. Изведено оттук твърдение за сходимост на две реализации би било грешка в разсъждението и рецензент би я видял веднага. `Angie` влиза в прегледа като поддържана реализация със собствена документация, а не като независимо потвърждение.

H2O. Документацията е още по-пряка: записът `listen` с атрибут `type`, зададен на `quic`, указва на сървъра да се свърже с UDP порт, а самият запис изисква атрибут `ssl` [17]. Двата протокола не споделят запис.

Тук се вижда и второто следствие в най-чист вид. Атрибутът `ssl` на записа носи пътищата до сертификата и до ключа [18], тоест конфигурацията на сертификата принадлежи на слушателя, а не на хоста. Два записа за един и същ номер на порт, един над TCP и един над UDP, означават две места, на които сертификатът е указан, освен ако конфигурационният език не се използва така, че да ги сподели.

Caddy. Единицата е адресът на сайта, а не слушателят. Схемата в адреса определя дали услугата е криптирана, а сертификатите се получават и подновяват автоматично за адресите, които отговарят на условията [12].

Това е единственият случай в прегледа, при който операторът не описва слушател. Слушателят се извежда от адреса. Обвързването обаче не изчезва, а само престава да се пише: изборът остава в адреса, тоест пак преди пристигането на първия байт.

Пренасочването от HTTP към HTTPS е другата половина на същото наблюдение. То използва порт 80 и работи успоредно с HTTPS на порт 443 [12]. Пренасочването съществува именно защото слушателят на 443 не може да обслужи некриптирана заявка. То не е услуга към клиента, а компенсация за разделянето, и документацията го описва като поведение, което може да бъде изключено отделно от издаването на сертификати [12], тоест като отделна единица, а не като свойство на слушателя на 443.

IIS. Единицата е елементът `binding`, а атрибутът `protocol` приема `http` или `https` [15]. Дотук моделът е същият.

Изключението е при QUIC. Там той не е отделен `binding`: стойност 16 на атрибута `sslFlags` изключва QUIC върху съществуващия HTTPS `binding` [15]. Тоест изборът се изразява чрез отсъствието на забраняващ флаг, а не чрез наличието на втора единица.

Изключението е полезно, защото показва, че обвързването не е неизбежно. То показва обаче и цената на тази алтернатива: изборът се превръща в бит в поле на друг обект, вместо в самостоятелна единица, и остава обвързан с наличието на HTTPS `binding`. Премахната е конфигурационната умножителност, но не и въпросът за броя на слушащите дескриптори, защото на него документацията на този слой не отговаря.

Ограничение на прегледа. При IIS връзките се приемат от драйвер в ядрото, а не от самия сървърен процес. Затова за него тук се твърди само за конфигурационната единица, но не и за броя на слушащите дескриптори: двете не са сравними с останалите редове на таблицата. Редът за Caddy е непълен по същата причина, по която е попълнен останалата част от таблицата: проверената документация описва адресите на сайтовете и автоматичния HTTPS, но не и организацията на UDP слушателя.

Прегледът е ограничен и по обем. Той обхваща пет сървъра, избрани заради това, че документацията им описва слушателите изрично. Отсъствието на друг сървър от таблицата не е твърдение за него.

1.4. Какво струва един допълнителен слушател

Общото свойство на таблицата е, че изборът на протокол се записва в конфигурацията на слушащата единица, тоест се взема преди пристигането на какъвто и да е байт. Затова добавянето на протокол добавя слушаща единица. Това не е ограничение на нито една от реализациите, а следствие от мястото, на което решението се взема.

Цената на тази допълнителна единица не е един дескриптор. Тя е списък, и всеки елемент от него следва от цитираното дотук.

Конфигурация на сертификата. Тя виси на слушателя, не на услугата. При N2O атрибутът `ssl` принадлежи на записа `listen` [18]. При Drogon всеки елемент на масива `listeners` носи собствени път до сертификат и ключ [19]. Тоест едно и също удостоверение се указва толкова пъти, колкото са слушателите, които го използват.

Правило във филтрирането. Разрешаването на TCP на порт 443 не разрешава UDP на порт 443. Това са различни транспортни протоколи и се филтрират поотделно. Добавянето на HTTP/3 към сървър, който вече обслужва HTTPS, е промяна в защитната стена, а не само в конфигурацията на сървъра.

Наблюдение. Всяка величина, която се брои на слушател, се брои толкова пъти, колкото са слушателите: приети връзки, дължина на опашката за приемане, отказани връзки. Обединяването им обратно в едно число е работа, която някой върши.

Какво тук не се твърди. Нито един елемент от този списък не е измерен в настоящата работа. Те са преброени по документация и следват от нея, но твърдение за големината им би било твърдение за конкретна експлоатационна среда, а такава не е изследвана. Измерва се само цената на алтернативата, тоест какво струва решението за протокола да се вземе след приемането на връзката вместо преди него. Това е предмет на хипотеза X1 и на глава VI.

1.5. Разпределяне на QUIC връзки между работни нишки

Втората задача е специфична за QUIC и няма аналог при TCP. Тя не се отнася до броя на слушателите, а до това кой слушател трябва да получи даден пакет, и възниква

едва когато сървърът има повече от една работна нишка.

Защо хешът е правилният отговор за TCP. При TCP връзката е четворката от адреси и портове. Ядрото демултиплексира входящите сегменти именно по нея, и всеки сегмент от една връзка носи една и съща четворка по определение: ако четворката се промени, това е друга връзка. Механизмът SO_REUSEPORT разпределя входящите връзки между сокетите чрез хеш върху същата четворка, тоест разпределението съвпада с принадлежността на връзката по построение, а не по щастливо съвпадение.

Защо същият хеш е погрешен за QUIC. QUIC определя връзката чрез идентификатор. Основната функция на този идентификатор според RFC 9000 е промените в адресирането на по-долните слоеве да не водят до доставяне на пакетите при погрешна крайна точка [5]. Тоест транспортът изрично обявява четворката за неподходяща като идентичност на връзката.

Оттук произтича разминаването. SO_REUSEPORT хешира точно това, което QUIC е обявил за несъществено. Разминаването е структурно: то не е рядък случай, който да се отчете като шум, а следствие от определението на двата механизма, взети заедно.

Какво стандартът изисква и какво оставя на сървъра. RFC 9000 определя, че всяка от двете страни избира идентификаторите, които насрещната страна ще използва, тоест идентификаторите, с които клиентът адресира сървъра, са избрани от сървъра [5]. Дължината им е ограничена отгоре, а съдържанието им не е определено: стандартът допуска и идентификатор с нулева дължина, когато не е нужно по него да се насочва [5].

Това е свободата, върху която стъпват всички известни решения. Щом съдържанието на идентификатора е избор на сървъра, сървърът може да запише в него информацията, необходима за насочването. Стандартът не изисква това, но и не му пречи, а допускането на нулева дължина показва, че възможността за насочване по идентификатор е предвидена, а не изкопчена.

Кога разминаването се проявява. Миграцията на връзката е допустима след потвърждаване на ръкостискането, а новият път подлежи на проверка, преди да бъде използван пълноценно [5]. Освен това мигриращата страна използва идентификатор, който

не е бил използван по стария път, за да не могат наблюдатели да свържат двата [5].

Двете изисквания заедно определят задачата на сървъра. Той не избира кога клиентът ще смени адреса си, нито кой от издадените идентификатори ще бъде използван след смяната. Следователно всички идентификатори, издадени за една връзка, трябва да водят до една и съща работна нишка. Насочване, което работи само за текущия идентификатор, не е насочване.

Разминаването се проявява и без намерение за миграция. Промяна на адреса може да настъпи и поради пренастройване на NAT, тоест без нито една от двете страни да е решила да мигрира [5]. Честотата на явлението следователно не е под контрола на никого от двамата.

Границата, която стандартът поставя на всяко насочване. Идентификаторът на получателя в първата дейтаграма на клиента се избира от клиента и е с дължина поне осем байта [5]. В него няма и не може да има информация, поставена от сървъра. Тоест първата дейтаграма на всяка връзка не подлежи на насочване по съдържанието на идентификатора, при никоя реализация. Това е долна граница, а не недостатък на конкретен подход, и определя кои пакети изобщо е смислено да се препращат.

Погрешното насочване не е само загубено време. Пакет, попаднал при страна, която не може да го свърже с никакво състояние, може да предизвика Stateless Reset, а получаването на Stateless Reset прекратява връзката [5]. Тоест работна нишка, получила дейтаграма за чужда връзка и третираща я като непозната, може да прекъсне напълно изправна сесия.

Следствието е за формулировката на въпроса. Насочването не е оптимизация, а изискване за правилност: то трябва да съществува във всяка реализация с повече от една работна нишка над един UDP порт. Спорен е само начинът, тоест дали то да бъде в ядрото или в потребителското пространство. Точно този спор се нуждае от честотата, а не от наличието, и затова хипотеза X2 е за дела на препратените пакети, а не за това дали препращане е нужно.

Известното решение и неговата цена. Известният отговор е програма на eBPF, закачена за групата сокети, която разчита идентификатора в ядрото и насочва дейтаграмата, преди потребителското пространство да я види. Тя е по-бърза от препращане в потребителско пространство, защото спестява едно преминаване през границата на ядрото.

Цената ѝ е в наличността, а не в бързината. eBPF не съществува под macOS и под Windows, тоест подходът покрива една от трите платформи, които настоящата работа поддържа. Освен това зареждането на програма в ядрото изисква права, каквито сървърният процес невинаги притежава, и това ограничение е независимо от платформата: то важи и под Linux, когато процесът работи с намалени права.

Тук се твърди само това. Не се твърди колко по-бърз е подходът в ядрото, защото такова твърдение изисква измерване при съпоставими условия, а прегледът на документация не е измерване.

Основата под двата подхода. И насочването в ядрото, и препращането в потребителското пространство стъпват върху `SO_REUSEPORT`, тоест върху възможността няколко сокета да бъдат свързани към един и същ порт. Опцията е въведена в Linux 3.9 и разпределя входящите дейтаграми равномерно между приемащите нишки, а при TCP разпределя приетите връзки [20]. Тя не е част от POSIX и произхожда от BSD, което е и първата причина да не се разчита на еднакво поведение между платформите: под macOS опцията съществува, но не разпределя така, както под Linux.

Точно равномерното разпределяне е и източникът на задачата. То се извършва по хеш върху адресите, а QUIC определя връзката по идентификатор, който преживява смяната им [5]. Механизмът, който прави многонишковото приемане възможно, е същият, който разваля привързването на връзката към нишка.

Колко често това има значение в действителност. Измерванията в дивата природа дават указание, но не и отговор. Проучване на разгръщането на QUIC установява, че част от най-посещаваните услуги изобщо не поддържат миграция на връзката [21], а по-ранно наблюдение върху трафика показва бързо, но неравномерно разпространение на самия протокол [22]. Нито едно от двете не съобщава дела на пакетите, изискващи препращане при сървър с няколко работни нишки, защото това е величина от страна на сървъра, а не от страна на мрежата. Тя е предмет на хипотеза X2.

Величината, която липсва. Липсва и нещо по-просто от преносим еквивалент. Липсва публикувано измерване *колко често* насочването изобщо е необходимо, тоест какъв дял от получените пакети попадат при нишка, която не притежава връзката. Без това число сравнението между решение в ядрото и решение в потребителско пространство се води

на предположения: оптимизация на нещо, което се случва рядко, и оптимизация на нещо, което се случва постоянно, изглеждат еднакво убедително, докато честотата не бъде измерена. Това е формулирано като хипотеза X2.

1.6. Библиотеки за уеб приложения на C++

Предходните два раздела разглеждат готови сървъри. Настоящата работа обаче предлага библиотека, тоест сравнението трябва да се направи и на това ниво. Въпросът е дали обвързването от раздел 1.2 изчезва, когато слушателят се създава от код на приложението, а не от конфигурационен файл.

Не изчезва. Появява се в същата форма, само че като поле на обект.

Таблица 4. Библиотеки за уеб приложения на C++, по тяхната документация

Библиотека	Обявени протоколи	Модел на изпълнение	Слушатели
Drogon	HTTP/1.0, HTTP/1.1, WebSocket [23]	неблокиращ вход и изход над epoll, съответно kqueue; корутини [23]	масив listeners; https е булев признак на всеки [19]
Crow	HTTP, WebSocket [24]	не се описва в прегледаните страници	TLS е свойство на обекта app [25]
Oat++	не се посочват версии [26]	проста и асинхронна програмна повърхност; асинхронната се препоръчва при 1000 и повече едновременни клиенти [27]	ConnectionProvider като отделен обект [26]

Най-прекият случай е Drogon. Конфигурационният му файл описва слушателите като масив, в който всеки елемент носи булев признак https заедно със собствени път до сертификат и ключ [19]:

```

1 "listeners": [
2   { "address": "0.0.0.0", "port": 80, "https": false },
3   { "address": "0.0.0.0", "port": 443, "https": true,
4     "cert": "", "key": "" }
5 ]

```

Листинг 2: Слушатели в конфигурацията на Drogon

Това е същата структура като при `nginx`, изразена в JSON вместо в директиви. Изборът е записан в слушателя, тоест се взема преди приемането на връзката.

При Crow разделението е още по-едро: TLS се задава чрез метод на самия обект `app`, например `app.ssl_file(...)` [25]. Тоест признакът принадлежи на цялото приложение, а не на отделен слушател.

Разликата между двете е по-съществена от синтаксиса. При Drogon масивът от слушатели стои вътре в конфигурацията на едно приложение [19]. Умножава се слушателят заедно с конфигурацията на сертификата, но не и средата за изпълнение: маршрутизаторът, веригата от `middleware` и наборът от работни нишки остават общи. При Crow, където TLS е свойство на обекта `app` [25], обслужването на криптиран и некриптиран трафик умножава самия обект, тоест и всичко, което виси на него.

Тази разлика определя каква е действителната цена на обвързването в една библиотека. Тя не е дескрипторът. Тя е това, което стои зад слушателя и се дублира заедно с него.

Моделът на изпълнение определя цената на връзката. Drogon описва неблокиращ вход и изход над `eroll`, съответно `kqueue`, заедно с корутини [23]. При такъв модел цената на една връзка е дескриптор и състояние на спряна корутина, а не стек на нишка, тоест броят на едновременните връзки не е ограничен от броя на нишките.

Oat++ предлага две програмни повърхности и препоръчва асинхронната при 1000 и повече едновременни клиенти [27]. Самият праг е информацията: препоръка с праг има смисъл само ако цената на връзката расте различно при двете повърхности. Какво точно струва връзката при простата повърхност, документацията не казва, затова тук не се твърди повече от това.

За Crow прегледаните страници не описват модела на изпълнение [24, 25], тоест за него не се твърди нищо в тази посока.

Началното състояние на настоящата работа беше същото. Това се казва тук, а не се премълчава: преди промените, описани в глава III, обслужването на криптиран и

некриптиран трафик от тази библиотека изискваше два екземпляра на приложението, тоест две групи работни нишки, два маршрутизатора и две вериги от middleware. Раздел 3.1 премахва точно това. Наблюдението от раздел 1.2 не е открито откън.

Обявените протоколи. Нито една от трите прегледани библиотеки не обявява HTTP/2 или HTTP/3. Drogon изброява изрично HTTP/1.0 и HTTP/1.1 [23]; Crow се описва като средство за услуги по HTTP или WebSocket [24]; страниците на Oat++, които бяха прегледани, не посочват версии [26].

Формулировката е „не обявява“, а не „не поддържа“. Отсъствието в документацията е доказателство за това, което авторите представят, а не за това, което кодът съдържа. Достатъчно е обаче за целта тук: многопротоколното обслужване не е характеристика, около която тези библиотеки са проектирани, тоест въпросът за броя на слушателите при няколко протокола не е поставян от тях.

Струва си да се отбележи и обратното следствие. Библиотека, която обявява един протокол над TCP, никога не среща въпроса от раздел 1.1: при нея няма какво да се договаря и няма втора крайна точка, която да бъде обявена. Обвързването при нея не е решение, а липса на повод за решение.

Обхват на прегледа. Прегледът обхваща три библиотеки, при които документацията описва как се създава слушателят. Boost.Beast [28] и Pistache [29] не са прегледани по този въпрос и затова за тях тук не се твърди нищо, нито в едната, нито в другата посока.

Какво могат и какво не могат публикуваните сравнения. Съществуват публични сравнения на производителността на рамки за уеб приложения [30]. Те съпоставят различни рамки при едно и също натоварване, тоест отговарят на въпроса коя рамка е по-бърза при дадена задача.

Въпросът на настоящата работа е друг по форма. Той е за две конфигурации на един и същ двоичен файл: с разпознаване на протокола след приемането на връзката и без него.

Второто рамо заслужава уточнение, за да не се чете като нещо, което не е. То не създава отделен слушащ сокет за всеки протокол. То изключва класификацията и обслужва само HTTP/1.1, тоест пътят на заявката е същият, какъвто би бил при сървър с отделни слушатели: без допълнително четене, без класифициране и без обвивка, която

връща прочетеното. Разликата между отделни слушатели и един слушател е в броя на дескрипторите, а той не участва в цената на отделната заявка. Затова рамото измерва точно цената на класификацията, което е величината, която хипотеза X1 назовава, и не измерва разликата в броя на сокетите, която е предмет на преброяването в глава VI. Сравнение между рамки не може да му отговори, колкото и рамки да обхваща, защото разликите между реализациите са по-големи от разликата, която се търси, и я поглъщат. Затова измерването в глава VI е върху един двоичен файл в два режима, а не върху няколко проекта, и затова хипотеза X1 е формулирана като разлика между режими, а не като класация.

Какво не се сравнява тук. Механизмът за маршрутизиране не се сравнява по документация. Твърдението за него е за поведение при нарастващ брой маршрути, а такова твърдение се проверява с измерване, не с четене. То е формулирано като хипотеза X4. Кампанията от глава VI не я проверява: обхождане по броя на регистрираните маршрути не е част от нея, и X4 остава непроверена.

1.7. Какво липсва в прегледаното

Прегледът дотук установява какво съществуващите реализации правят. Този раздел казва какво не правят, и го казва в такава форма, че целта в следващия раздел да бъде отговор, а не пожелание.

Първо. Решението за протокола не е отделено от решението за сокета. При петте прегледани сървъра и при трите библиотеки изборът се записва в описанието на слушащата единица, независимо дали тя е директива, запис, адрес, елемент или обект. Единственото изключение, IIS, не отделя решението, а го премества в бит на друг обект и го оставя зависимо от наличието на HTTPS binding [15].

Тоест въпросът „колко слушателя са необходими за n протокола“ не е поставян като въпрос за проектиране. Той има отговор във всяка от прегледаните системи, но отговорът е следствие от конфигурационния модел, а не резултат от избор.

Второ. Преносимото насочване на QUIC е без публикуван отговор. Известното решение работи в ядрото на една платформа и изисква права за зареждане на програма.

Прегледаните източници не описват какво прави сървър, който няма нито eBPF, нито достатъчно права.

Това твърдение е за прегледаното, а не за съществуващото. То означава, че липсва източник, който да бъде цитиран, а не че такова решение няма никъде.

Трето. Честотата, която решава спора, не е публикувана. Спорът дали насочването да бъде в ядрото се води без числото, което би го решило: делът на пакетите, които изобщо изискват насочване. Това е и единственото от трите, което е въпрос за измерване, а не за проектиране, и затова е формулирано като хипотеза, а не като задача.

Общото между трите. И трите липси са от един и същ вид. Решение, взето рано, не се проверява, защото не изглежда като решение. Обвързването на протокола със сокета е такова: то е било направено веднъж, когато е имало един протокол, и оттогава се наследява като форма на конфигурацията. Настоящата работа го връща обратно във вида на въпрос и после измерва отговора.

1.8. Цел, задачи и хипотези

Формулировките по-долу следват от установеното в раздели 1.2 и нататък, а не го предшестват.

1.8.1. Цел

Целта на дисертационния труд е да се разработи и изследва архитектура на библиотека за уеб приложения на C++, при която броят на слушащите дескриптори не зависи от броя на обслужваните протоколи на приложно ниво, и да се установи цената на това решение чрез измерване.

Целта съдържа две части умишлено. Разработването без измерване би било инженерна работа; измерването без разработване не би имало какво да измери.

1.8.2. Задачи

1. Да се формулира формално зависимостта между броя на слушащите дескриптори, броя на работните нишки и броя на обслужваните протоколи, и да се докаже предложението за нейната минимизация.

2. Да се разработи механизъм за разпознаване на протокола след приемането на връзката, който не изисква отделен слушащ сокет за всеки протокол.
3. Да се разработи преносим механизъм за разпределяне на QUIC връзки между работни нишки, независим от механизми, специфични за едно ядро.
4. Да се разработи механизъм за отложено получаване на данни при изобразяване от сървър, при който незавършеността на стойността е изразена в типовата система от двете страни на връзката.
5. Да се реализира втори механизъм за вход и изход под Linux, така че видът на интерфейса да стане независима променлива.
6. Да се разработи методика за измерване, която изключва известните начини за тихо погрешно измерване на многопротоколни сървъри.
7. Да се проведе измерване и да се провери всяка от хипотезите по-долу.

1.8.3. Хипотези

Хипотезите са формулирани така, че да могат да бъдат отхвърлени. Всяка от тях посочва и измерването, което би я отхвърлило.

X1. Разпознаването на протокола по първия октет след приемането на връзката не води до измеримо влошаване на пропускателната способност или на латентността спрямо същия сървър без класификация, който е пътят на заявката при отделни слушащи сокети за всеки протокол.

Отхвърля се, ако при еднакво предложено натоварване разликата между двата режима на един и същи двоичен файл има доверителен интервал, изключващ нулата, и относителна големина над пет процента.

X2. Делът на QUIC пакетите, изискващи препращане между работни нишки при реалистична миграция, е достатъчно малък, за да не оправдава насочване в ядрото.

Отхвърля се, ако измереното отношение на препратените към получените пакети е съществено, тоест ако разликата във времето за обработка, дължима на препращането, е измерима при условията от глава V.

X3. Отложеното получаване на данни намалява времето до първия байт спрямо изчаканото, при еднакво предложено натоварване и без влошаване на времето до пълното

изобразяване извън границите на измерителната грешка.

Отхвърля се, ако времето до първия байт не се различава извън доверителния интервал, или ако времето до пълното изобразяване се влошава съществено.

X4. Времето за маршрутизиране остава практически независимо от броя на регистрираните маршрути в контекста на пълна заявка, а не само в изолиран микротест.

Отхвърля се, ако делът на времето за съпоставяне в общото време на заявката нараства съществено с броя на маршрутите.

1.9. Изводи

Едновременното обслужване на три протокола е следствие от начина, по който протоколите се въвеждат в употреба, а не преходно състояние. HTTP/1.1 няма механизъм за договаряне и затова не може да бъде изключен. HTTP/2 без TLS загуби пътя през Upgrade и се стартира с предварително знание [9]. HTTP/3 не може да бъде поискан пръв и се обявява по вече съществуваща връзка над TCP [4, 6]. Оттам следва минималният набор от три комбинации на транспорт и криптиране.

Обвързването на протокола със слушащата единица е документирано, а не предположено. То се среща еднакво при пет сървъра и при три библиотеки на C++, и се изразява по един и същ начин на двете нива: като параметър на директива, като атрибут на запис, като схема в адрес, като булев признак в JSON или като метод на обекта на приложението. Формата се променя, мястото на решението не се променя.

Цената на допълнителната слушаща единица също не е един дескриптор. Тя е дескриптор на всяка работна нишка [13], конфигурация на сертификата на всеки слушател [18, 19], отделно правило във филтрирането, и отделен ред във всяка величина, която се брои на слушател.

Изключението при IIS е важно, защото показва, че обвързването не е неизбежно. То показва обаче и че алтернативата, избрана там, премества избора в бит на друг обект, вместо да го премахне: QUIC остава привързан към наличието на HTTPS binding.

QUIC добавя втора задача, която няма аналог при TCP. Разпределянето на дейтаграми по хеш върху адресите противоречи на определението на връзката чрез идентификатор, който сървърът избира и който преживява смяна на адреса. Известното решение работи в ядрото и е ограничено до една платформа, а честотата, при която то

изобщо би имало значение, не е публикувана.

Оттук произтичат целта и задачите от раздел 1.8. Те не са предпоставка на прегледа, а негово следствие: щом решението се взема при конфигурирането на слушателя, то може да бъде преместено след приемането на връзката, и тогава остава да се установи какво струва това преместване.

II. ТЕОРЕТИЧНИ ОСНОВИ

Тази глава излага основанията, върху които стъпва работата: моделът на изпълнение, алгоритъмът за съпоставяне и двете твърдения, които работата формулира сама. Моделът и алгоритъмът са известни и се излагат, за да бъдат използвани, а не преразказани. Двете твърдения, границата на собственост върху кадъра на корутината и минимизацията на слушащите дескриптори, са нови и се излагат във вид, който позволява да бъдат оборени.

2.1. Едновременност и паралелизъм

Двете понятия се смесват достатъчно често, за да си струва разграничението да бъде направено изрично, още повече че българската литература не предлага установена двойка.

Едновременност (англ. *concurrency*) е свойство на *структурата* на програмата: наличие на няколко логически хода на управление, чиито стъпки могат да се преплитат. *Паралелизъм* (англ. *parallelism*) е свойство на *изпълнението*: няколко стъпки, извършвани физически едновременно.

Разликата има практическо значение тук. Сървър с една работна нишка, обслужващ десет хиляди връзки, е изцяло едновременен и напълно непаралелен. Обратното също съществува. Настоящата работа използва едновременност за структурата на връзките и паралелизъм за разпределянето им между работни нишки, и двете са независими оси.

2.2. Корутини без собствен стек

Корутината е подпрограма, чието изпълнение може да бъде спряно и подновено, като състоянието между двете се запазва [31]. Разликата, която има значение за сървър, е къде се пази това състояние.

Корутината със стек притежава собствен стек и може да спре от произволна дълбочина на извикванията. Цената е този стек: разпределен предварително, той се плаща за всяка връзка, дори когато е почти празен.

Корутината без стек, каквито са корутините в C++20 [32], пази само това, което пресича точка на спиране. Кадърът е с размер, определим при компилиране, и обикновено е порядъци по-малък от стек на нишка. Ограничението е, че спирането е възможно само в

тялото на самата корутина, а не от произволна дълбочина.

За сървър, при който броят на едновременните връзки е основната ос на мащабиране, размерът на състоянието за връзка е и основната цена, и това е причината изборът да е корутини без стек.

2.2.1. Какво съдържа кадърът

Стандартът изброява съставките на състоянието на корутината [32]: обектът `promise`; копия на формалните параметри; представяне на текущата точка на спиране, достатъчно за да се знае откъде продължава подновяването и какво трябва да се разруши при унищожаване; и памет за обектите, чийто живот пресича текущата точка на спиране.

Първите три са с фиксиран размер: `promise` е обект от известен тип, параметрите са с известни типове, а точката на спиране е индекс в крайно множество. Четвъртата зависи от тялото и точно тя е определима при компилиране. Точките на спиране са синтактични: всяко `co_await`, `co_yield` и `co_return`, плюс началното и крайното спиране. Множеството им се вижда от изходния текст, а за всяка от тях живите обекти следват от статичния анализ на живота. Размерът на кадъра е максимумът по точките на спиране, а не сборът по тях, и не зависи от нито един вход.

Оттук следва твърдението, което има значение за сървър: цената за връзка в памет е свойство на текста на програмата, а не на изпълнението ѝ. Тя се променя при промяна на тялото на корутината и не се променя при промяна на броя връзки или обема трафик.

Три уговорки, за да не бъде твърдението по-силно, отколкото е.

Определим при компилиране не значи четим от изходния текст. Стандартът не дава начин размерът да бъде получен като константа: `sizeof` не се прилага върху състояние на корутина. Числото е известно на компилатора, не на автора.

В кадъра стои обектът, а не това, което той владее. Локален `std::vector`, пресичащ точка на спиране, добавя само фиксирания си заглавен блок; буферът му остава в динамичната памет и зависи от входа.

Кадърът се заделя чрез `operator new`, освен ако реализацията не установи, че животът му се съдържа в този на викация [32]. Размяната следователно не е „голям резерв срещу нула“, а „голям резерв срещу много малки заделения“, и кое от двете е по-евтино зависи от разпределителя на паметта, а не от езика.

2.2.2. Какво пресича точка на спиране

Обект пресича точка на спиране, ако е жив преди нея и се използва след нея. Локална променлива, чиято последна употреба предхожда единственото `so_await` в обхвата ѝ, не влиза в кадъра. Оттук и практическото правило за цената: кадърът се свива, когато изчисленията са групирани между точките на спиране, и расте, когато през тях се пренасят междинни резултати.

Временните обекти в операнда на `so_await` също пресичат спирането, тъй като животът им продължава до края на пълния израз, а спирането е вътре в него. `Awaiter`-ът е такъв временен обект. Затова състоянието, което той носи, живее в кадъра на спрялата корутина, а не в активационния запис на подновяващата нишка. Страната, която унищожи кадъра, унищожава и `awaiter`-а: това е предпоставката за раздел 2.3.

Обратното е източникът на грешки. Указател към обект от активационния запис на подновяването не преживява спирането, защото този запис изчезва. Езикът не отбелязва разликата в типовата система, тоест тя се проверява от читателя на кода.

2.2.3. Кадър и стек: какво определя размера на всеки

Сравнението между корутина без стек и нишка не е сравнение между две числа, а между два начина размерът да бъде определен. Размерът на кадъра е максимум по крайно, статично известно множество, взет от компилатора върху текста на една функция: по-голям не е нужен, по-малък не е достатъчен.

Размерът на стека на нишка се задава като параметър при създаването ѝ или се наследява от подразбиране на системата. Този параметър не е свойство на функцията, която нишката ще изпълни. Той трябва да покрие най-дълбоката верига от извиквания, която нишката изобщо може да поеме, включително записите на библиотеки и на входовете към ядрото, върху които програмата няма контрол. Такава верига не е статично известна за произволна програма, тоест параметърът се избира като горна граница със запас, а не се извежда.

Следствието за сървър. При C едновременни връзки цената при корутини без стек е сборът от размерите на кадрите, всеки ограничен от константа, известна при компилиране. При нишка на връзка цената е C пъти резерв, избран за най-лошия случай, а не за обичайния. Резервът е адресно пространство и обичайните реализации го заделят лениво, което смекчава цената, но не премахва зависимостта от параметър, избран без да се знае

какво ще се изпълни в него. Числа не се твърдят тук: твърдението е за произхода на двете величини, а стойностите за настоящата реализация не са измервани.

Цената на избора е ограничението: спиране от произволна дълбочина изисква всяка функция по веригата да бъде корутина, тоест асинхронността се разпространява нагоре през интерфейсите и не може да бъде скрита. Глава IV показва къде това се усеща.

Спорът, чия е тази цена, е по-стар от корутините в C++. Аргументът, че моделът със събития е неизбежен при висока едновременност, е противопоставен на аргумента, че нишките дават същата едновременност при по-прост изходен текст, стига цената на контекста да бъде свалена [33], а междинните архитектури разделят обработката на етапи със собствени опашки [34]. Корутината без стек е трети отговор на същия въпрос: тя запазва изходния текст на нишковия модел и свежда цената на контекста до онова, което пресича точка на спиране. Настоящата работа не преповтаря този спор и не твърди изход от него; тя приема третия отговор и описва от какво зависи цената му.

2.2.4. Протоколът на изчакването

Изчакването се изразява чрез три операции. `await_ready` позволява спирането да се пропусне, ако стойността вече е налична, тоест плаща се само когато наистина се чака. `await_suspend` получава дескриптор на вече спряната корутина и решава какво следва. `await_resume` дава резултата при подновяване и нейната върната стойност е стойността на израза `co_await`.

Средната от трите има три възможни връщани типа и разликата между тях се използва по-нататък [32]. При `void` корутината остава спряна и управлението се връща на този, който я е подновил последно. При `bool` стойността `false` отменя спирането, тоест корутината продължава веднага, а `true` го потвърждава. При `std::coroutine_handle<>` върнатият дескриптор бива подновен, което е симетричното прехвърляне.

Средната от трите е и мястото, където се появява надпреварата от раздел 4.2, и причината е структурна: тя е единствената, която се изпълнява, след като кадърът вече е достъпен за друга страна. Точната формулировка е в раздел 2.3.

2.2.5. Симетрично прехвърляне

Когато корутина *A* завърши и трябва да поднови корутина *B*, наивната реализация извиква подновяването на *B* от тялото на `await_suspend` на *A*, тоест от активационния запис на *A*. Симетричното прехвърляне вместо това връща дескриптора на *B* и оставя

подновяването на механизма, който е извикал `await_suspend`. Формата с връщан дескриптор съществува точно за да може преходът да бъде опашково извикване, а не вложено [32].

Твърдение. Нека $A_1, A_2, \dots, A_k$ е верига, в която всяка корутина при завършването си подновява следващата, и нека $d(i)$ е дълбочината на стека в момента на подновяване на A_i . При наивната реализация $d(k) = \Theta(k)$; при симетрично прехвърляне $d(k) = \Theta(1)$.

Обосновка. Наивно: подновяването на A_{i+1} се извършва отвътре в активационния запис на A_i , тоест $d(i+1) = d(i) + c$ за някакво $c > 0$, определено от записите на подновяването и на `await_suspend`. По индукция $d(k) = d(1) + (k-1)c$. Нито един запис не се освобождава, преди A_k да завърши, тъй като всеки чака завръщането на следващия.

Симетрично: `await_suspend` на A_i връща дескриптора на A_{i+1} и се завръща. Записът на A_i се освобождава, преди A_{i+1} да бъде подновена, тоест $d(i+1) = d(i)$ и $d(k) = d(1)$, независимо от k . $\square$

Значението е в това какво представлява k . Дължината на веригата не е свойство на програмата: тя е броят на завършванията, готови за обработка в един обход на цикъла на събития, тоест величина, зависеща от трафика. Наивната реализация свързва дълбочината на стека с натоварването, и то в посоката, в която натоварването расте. Преливането тогава е въпрос за кога, а не за дали.

Веригата трябва и да завършва. Когато продължение няма, реализацията връща `std::noop_coroutine()`, чието подновяване не прави нищо и връща управлението на цикъла [32]. Така връщаният тип остава еднакъв за всички звена и опашковата форма важи и за последното.

Едно ограничение се заявява веднага. Постоянната дълбочина следва от това преходът да бъде реализиран като опашково извикване. Езикът задава семантиката, тоест че A_i е спряна, преди A_{i+1} да бъде подновена; че записът е освободен, е свойство на реализацията, каквото формата е предназначена да позволи. Дълбочината на стека не е измервана тук и граница за нея в числа не се твърди.

2.3. Границата на собственост върху кадъра

Разделът формулира първото от двете твърдения, които работата предлага; случаите, в които то е приложено, са в раздел 4.2. Разделянето е умишлено: твърдението

не е за конкретен механизъм за вход и изход, а за всяко подаване на операция, чието завършване се обработва от друга страна.

Означения. Нека C е корутина, спряна в точка p , и нека F е нейният кадър. Нека o е операция, подадена от `await_suspend` в точка p , а R е страната, която обработва завършването на o и подновява C . R може да бъде друга работна нишка, обработващият завършванията в цикъла на събития или същата нишка на по-късен обход. Твърдението не различава тези случаи и затова важи и за еднонишкова конфигурация.

Определение (момент на годност). Момент на годност $t(o)$ е първият момент, в който R може да наблюдава o като завършена. Той не съвпада с момента на завършване и по правило го предхожда.

Моментът е различен за различните механизми и точно затова се дефинира, а не се назовава. При интерфейс по завършване това е публикуването на заявката заедно с известяването на ядрото; при интерфейс по готовност, регистрацията на дескриптора в наблюдаваното множество; при операция, предадена през опашка, вписването в опашката. Общото между трите е, че моментът е *наблюдаем от подаващата страна*: тя знае кое извикване го причинява, дори когато не знае кога ще дойде завършването.

Предложение 1. След $t(o)$ кадърът F принадлежи на R . Всяко четене или писане в F от подаващата страна в момент, следващ $t(o)$, е надпревара с подновяването, която подаващата страна няма как да спечели надеждно.

Обосновка. По определението на $t(o)$ след този момент съществува страна R , която може да поднови C . Подновяването предава управлението в F и може да го изпълни до край, а при достигане на крайното спиране кадърът се унищожава [32]. Достъпите на подаващата страна след $t(o)$ не са наредени спрямо това изпълнение: единствената зависимост, установена от `awaiter`-а, е записването на продължението, а то предхожда $t(o)$. Следователно достъпите са едновременни с подновяването. Ако то е достигнало края, те са обръщение към унищожен обект; ако не е, те са състезание за данни. $\square$

Следствия за проектирането. Три на брой, и всяко е правило, а не съвет за внимание.

Всяка стойност, която трябва да бъде видима в кадъра, се записва *преди* $t(o)$. Оптимистичното записване, последвано от поправка по пътя на неуспеха, е правилната

форма, а не обратната.

Никоя стойност не се *чете* от кадъра след $t(o)$. Връщането на литерал вместо на член на `awaiter`-а не е стил: членът живее в кадъра, а кадърът вече не е на подаващата страна.

Пътят на неуспеха е безопасен точно когато лежи изцяло преди $t(o)$, тоест когато операцията не е станала годна. Тогава R не съществува и записът в кадъра е самотен.

Какво предложението не твърди. Не твърди, че надпреварата ще бъде наблюдавана. Прозорецът е тесен и вероятността зависи от механизма; глава IV съобщава един случай, в който тя се проявяваше редовно, и два, в които не беше възпроизведена.

Не твърди, че поправката е заключване. Ключалка около достъп до кадър, който вече може да е унищожен, не помага, защото самата ключалка е в него.

Не твърди и че границата е проверяема от компилатора. Подписът на `await_suspend` не различава двете страни на $t(o)$, тоест в C++20 правилото се спазва от автора, а не от типовата система. Оттам и това, че една и съща грешка беше допусната на три независими места.

2.4. Съпоставяне срещу множество регулярни изрази

Маршрутизирането е съпоставяне на един вход, пътя на заявката, срещу множество шаблони. Величините са две и се държат различно: дължината на входа N и броят на шаблоните M . За сървър N е малко и ограничено отгоре, тъй като редът на заявката има граница, докато M расте заедно с приложението. Затова интересната зависимост е тази от M .

2.4.1. Последователното съпоставяне

Обичайната реализация опитва шаблоните един след друг и спира при първото съвпадение. Ако всеки шаблон е превърнат в детерминиран автомат, всеки опит струва $O(N)$, тоест общо $O(N \cdot M)$ в най-лошия случай, когато съвпада последният шаблон или никой. Средният случай не е по-добър по ред на растеж: той зависи от подредбата на таблицата с маршрути, тоест от нещо, което приложението рядко подрежда съзнателно.

Границата $O(N)$ за един шаблон при това не е даденост. Механизъм с връщане назад има изрази, за които времето расте експоненциално с дължината на входа.

Симулацията на недетерминиран автомат по Томпсън избягва това и дава $O(N \cdot |Q|)$ за автомат с $|Q|$ състояния [35]; детерминизацията с подмножества премахва и множителя $|Q|$ от времето, като го премества в броя на състоянията [36]. Алтернативен път до същото минава през производните на Бжозовски, при които състоянията се строят по нужда, тоест построяването се плаща само за частта от автомата, до която входовете реално стигат [37].

Тези граници са за един шаблон. Множителят M остава, защото решенията са M на брой и независими помежду си.

2.4.2. Обединеният автомат

Идеята, която премахва M , е шаблоните да не бъдат отделни решения. M -те израза се обединяват в един, състоянията на приемане се етикетират с това кой шаблон е приел, и полученият автомат се детерминизира веднъж. Съпоставянето тогава е един обход на входа, тоест $O(N)$ *независимо от M* . За фиксирани низове това е конструкцията на Ахо и Корасик, при която построяването е линейно по общата дължина на шаблоните, а съпоставянето е линейно по входа [38]; за регулярни изрази това е същата идея върху по-широк клас езици.

Цената не изчезва, а се премества на три места.

Първо, времето за построяване. Детерминизацията с подмножества е в най-лошия случай експоненциална по броя на състоянията на недетерминирания автомат, тоест по общата дължина на шаблоните. За сървър това е поносимо, защото се плаща веднъж при стартиране, но не е безплатно и не е ограничено полиномиално.

Второ, паметта. Обединеният автомат заема място, зависещо от броя на състоянията и от размера на азбуката, а не от броя на заявките. Размяната е памет при стартиране срещу време при всяка заявка.

Трето, семантиката при съвпадение на няколко шаблона. Последователната реализация решава двусмислието безплатно: първото съвпадение печели, а редът е редът на обявяване. Обединеният автомат стига до състояние, което може да е етикетирано с няколко шаблона, тоест правилото за избор трябва да бъде въведено изрично. Това е задължение, което конструкцията създава, а не наследява.

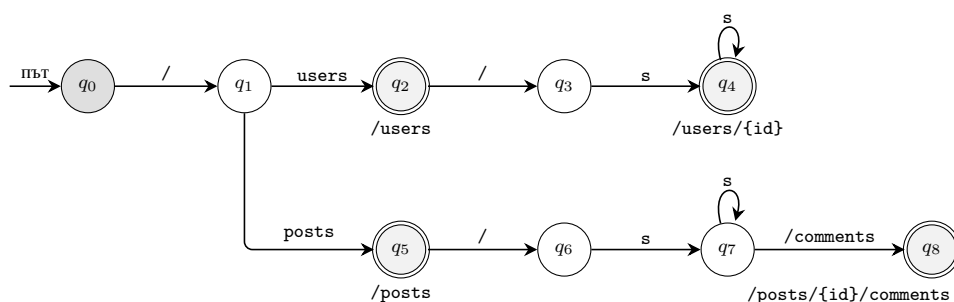
2.4.3. Какво е наследено и какво не се доказва наново

Алгоритъмът, използван тук, е публикуван по-рано [39]. От него се наследяват конструкцията, нейната коректност и границата $O(N)$ по време на съпоставяне.

Настоящата работа **не доказва наново** нито едно от трите и не претендира приоритет върху тях.

Това, което тя си поставя за задача, е проверка в друг контекст. Публикуваната граница е за изолирано съпоставяне. Въпросът за сървър е друг: остава ли времето за обслужване на *пълна* заявка практически постоянно при нарастване на броя на маршрутите, когато към съпоставянето се прибавят синтактичният анализ, изграждането на отговора и входът и изходът. Отговорът не следва от границата, защото не се знае какъв дял от общото време заема маршрутизирането.

Оттам и рискът, заявен предварително: ако маршрутизирането е достатъчно малък дял, разликата няма да бъде откриваема, и измерването няма да потвърди нищо за алгоритъма, а само че тежестта му е малка. Това е допустим резултат и е предвиден като такъв в глава V. Устройството на автомата е показано на (Фиг. 2).



Двойният кръг е приемащо състояние. До него стои маршрутът, който то разпознава. *s* е символ от сегмент на пътя, тоест A-Za-z0-9_.%-. Ребра с няколко символа са свити за четимост. В автомата всеки символ е отделен преход. Началните символи на свитите ребра се различават, така че изборът остава детерминиран.

Фигура 2. Детерминиран краен автомат за съпоставяне на маршрути. Показани са четири маршрута. Автоматът се строи веднъж, при регистрацията. При заявка пътят се чете отляво надясно с по един преход на символ, например /users/42 дава девет символа и девет прехода. Затова времето за съпоставяне зависи от дължината на пътя, а не от броя на регистрираните маршрути. Още маршрути добавят състояния, но не добавят стъпки за един път.

2.5. Минимизация на слушащите дескриптори

Централното твърдение на работата се формулира тук като предложение, за да може да бъде проверено, а не преразказано.

Означения. Нека T е множеството от използвани транспорти, тоест $T \subseteq \{\text{TCP}, \text{UDP}\}$. Нека P е множеството от обслужвани протоколи на приложно ниво. Нека $S \subseteq P$ е подмножеството, изискващо TLS, а $P \setminus S$ е това, което не изисква. Нека W е броят на работните нишки, а L е броят на слушащите дескриптори.

Обичайна конфигурация. При сървър, който не може да обслужва криптиран и некриптиран трафик от един слушащ сокет, броят на дескрипторите за TCP е

$$L_{\text{conv}} = W \cdot ([S \neq \emptyset] + [P \setminus S \neq \emptyset]),$$

където $[\cdot]$ е индикатор. Тоест обслужването и на двата вида изисква два дескриптора на работна нишка. Това не е свойство на конкретна реализация, а следствие от обвързването на избора на протокол с избора на сокет: директивата за слушане на `nginx` приема или наличие, или липса на `ssl`, но не и двете [13].

Предложение 2. При класификация след приемането на връзката броят на слушащите дескриптори е

$$L_{\text{demux}} = A \cdot |T|,$$

където A е броят приемащи дескриптори, които моделът на приемане на платформата изисква за един транспорт. Същественото в предложението не е стойността на A , а това, че изразът е *независим от* $|P|$.

Стойността на A е свойство на платформата, не на реализацията. При модел със `SO_REUSEPORT`, при който всяка работна нишка притежава собствен слушащ сокет, $A = W$. При модел с един слушател и множество едновременни приемания, какъвто използват `IOCP` под `Windows` и `kqueue` под `macOS`, $A = 1$ независимо от броя на работните нишки.

Разграничението е направено тук, защото по-ранна формулировка на това предложение пишеше $W \cdot |T|$ безусловно. Преброяването в раздел 6.12 го опроверга: под `Windows` процесът държи един слушащ дескриптор при една, две, четири и осем работни нишки. Твърдението, което работата защитава, преживява поправката непокътнато, защото то е за отсъствието на $|P|$ от израза, а не за множителя пред него. Множителят обаче беше сгрешен и това се казва тук, а не се премълчава.

Обосновка. Достатъчно е първият октет да разделя S от $P \setminus S$. Записът на TLS започва с ContentType със стойност 0x16 за ръкостискане [40]; всеки метод на HTTP/1.1 и преамбюлът на HTTP/2 започват с главна латинска буква [9]. Двете множества от начални октети не се пресичат, тъй като 0x16 е управляващ символ. Следователно едно разклонение по един октет разделя S от $P \setminus S$. Вътре в S изборът се извършва от ALPN по време на самото ръкостискане [1], тоест без допълнителен дескриптор; вътре в $P \setminus S$ изборът се извършва по преамбюла. Никое от трите разклонения не изисква отделен слушащ сокет. $\square$

2.5.1. Предположенията поотделно

Обосновката стъпва на пет предположения. Те се изброяват поотделно, защото имат различен статут, а смесеното изброяване скрива кое от тях е най-слабото.

П1: несечащи се начални октети. Множествата от възможни първи октети на TLS запис и на текстов метод нямат общ елемент. *Статут: изведено от спецификациите и проверено механично.* Изводът е по стойности: 0x16 е управляващ символ, а методите и преамбюлът започват с главна латинска буква [40, 9]. Проверката е изпълним тест, който обхожда всичките 256 стойности на първия октет и изисква за всяка точно една класификация. Обхождането е изчерпателно, а не извадка, тъй като 256 стойности са цялата област на решението. Резултатът е един октет за TLS, двадесет и шест за некриптиран трафик и останалите за отхвърляне, тоест без общ елемент.

П2: класификаторът вижда първия октет. *Статут: следва от транспорта; за реализацията се проверява отделно.* TCP предава подредена последователност от байтове, тоест първият прочетен след приемането е първият изпратен. Реализацията добавя две изисквания, които не следват от транспорта: да се разглежда само първият октет и прочетеното за класификация да достигне непокътнато до слоя отдолу. Те са свойства на кода, не на протокола, и имат собствени тестове.

П3: изборът вътре в S не изисква дескриптор. *Статут: изведено от спецификацията.* ALPN пътува вътре в ClientHello, тоест вътре в ръкостискането, което вече се извършва по приетата връзка [1]. Не се създава сокет и не се извършва допълнителен обмен.

П4: изборът вътре в $P \setminus S$ не изисква дескриптор. *Статут: изведено от спецификацията.* Преамбюлт на HTTP/2 е първите байтове на същия поток [9]. Тук решението не е по един октет: методът PROPFIND споделя с преамбюла началото PR, тоест в най-лошия случай са нужни три октета, за да бъде отхвърлен. Предложението не се променя, защото три октета също не са дескриптор, но случаят показва, че „един октет“ е удобното частно, а не същността на аргумента.

П5: по един слушащ дескриптор на транспорт на работна нишка. *Статут: предположение за модела на разгръщане.* Множителят W идва от това, че всяка работна нишка има собствен слушащ сокет, за да може ядрото да разпределя връзките чрез SO_REUSEPORT. При един споделен слушащ сокет и разпределяне в потребителско пространство би се получило $L = |T|$. И в двата случая L не зависи от $|P|$, тоест точно частта, която се твърди, не зависи от този избор.

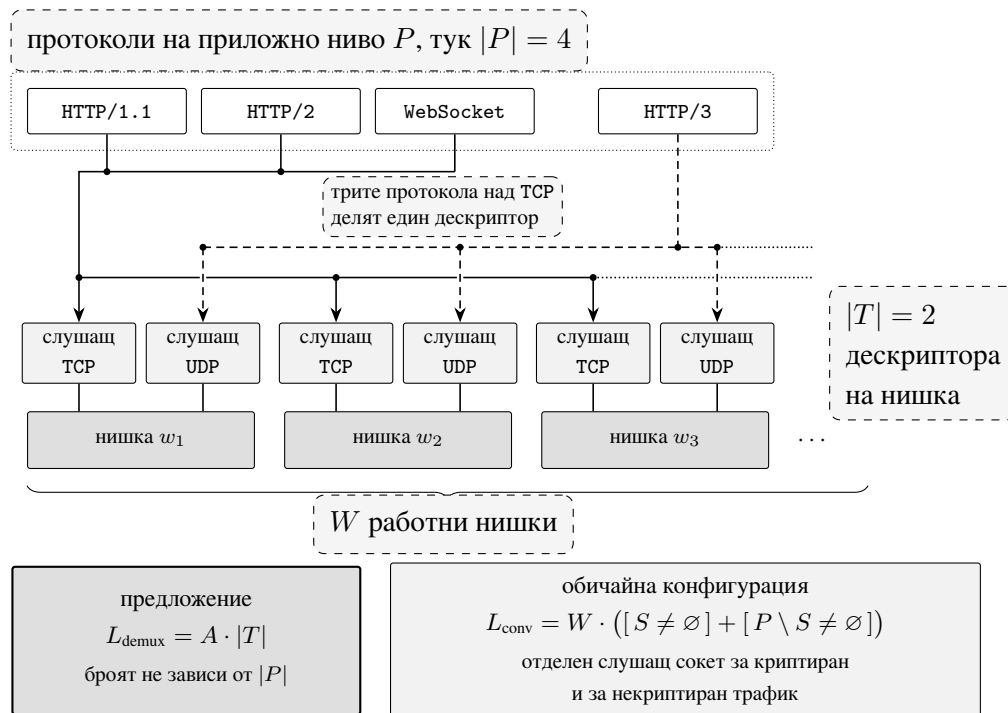
Какво проверката не проверява. Механичната проверка е върху реализацията. Ако тя и спецификацията се разминават, тестът минава, предложението остава вярно за спецификацията, а сървърът греша. Обосновката казва, че разделянето съществува; тестът казва, че този код прилага точно него. Предложението е показано и графично (Фиг. 3).

Какво предложението не твърди. То не твърди, че L е константа: множителят A остава, защото моделът на приемане на платформата е независима ос, и при SO_REUSEPORT той е равен на броя на работните нишки. Не твърди и че цената на класификацията е нула; твърди, че тя не изисква дескриптор. Дали е откриваема в измерване е отделен въпрос, поставен в глава V и решен в глава VI. Не твърди накрая, че класификацията проверява: един октет решава къде да отиде връзката, а не дали входът е смислен.

2.5.2. Случаят UDP

Над UDP предложението е тривиално и си струва да се каже защо, а не само че.

Множеството от протоколи, обслужвани над UDP, е едноелементно. HTTP/3 се пренася над QUIC, а QUIC няма некриптиран вариант: защитата на пакетите е част от протокола и дори първоначалните пакети са защитени с ключове, изведени от идентификатора на връзката [11]. Тоест $|P \cap \text{UDP}| = 1$, няма какво да се различава, класификация не се извършва изобщо и $L_{\text{UDP}} = W$. Предложението е вярно, но празно:



Фигура 3. Брой на слушащите дескриптори при класификация след приемане на връзката. Показан е моделът със SO_REUSEPORT, при който всяка работна нишка притежава по един слушащ дескриптор за TCP и по един за UDP; при модела с един споделен слушател дескрипторът е един на обслужван транспорт за целия процес. Протоколите от P се разделят след accept, по първия октет и по ALPN, а не чрез отделни слушащи сокети, затова линиите им се сливат преди дескрипторите и $|P|$ не участва в броенето. Остава $L_{\text{demux}} = A \cdot |T|$, в което A е броят приемащи дескриптори, изискван от платформата: W при модел със SO_REUSEPORT, единица при модел с един слушател. Вдясно долу е обичайната конфигурация за сравнение, при която разделянето на P на криптирана и некриптирана част се плаща с втори дескриптор на работна нишка.

то не спестява дескриптор, който иначе би съществувал.

Задачата над UDP е друга и това предложение не я решава. При W сокета на един порт въпросът не е колко са, а кой от тях получава даден пакет. `SO_REUSEPORT` разпределя по хеш върху адресите и портовете, докато QUIC връзката е определена от идентификатор, избран от сървъра, и преживява смяна на адреса на клиента по замисъл [5]. Това е обратната задача, разгледана в раздел 3.2. Отбелязва се тук, за да е ясно, че тривиалността над UDP не значи, че над UDP няма проблем. Ако някога над UDP се обслужва втори протокол на приложно ниво, въпросът за разделянето възниква наново и ще изисква собствен аргумент за непресичане, какъвто настоящата формулировка не съдържа.

2.5.3. Ако непресичането отпадне

Предложението е условно и условието е П1. Затова си струва да се каже какво остава от него, ако то бъде нарушено. Нека към P бъде добавен протокол, чието начало съвпада по първия октет с вече обслужван. Тогава един октет не решава и случаите са два.

Първо, ако съществува ограничена дължина k , при която началните езици на всички обслужвани протоколи са попарно различни, предложението **оцелява без промяна**. Класификацията чете k октета вместо един, а k октета не са дескриптор. Случаят вече присъства: разделянето на HTTP/1.1 от HTTP/2 не е по един октет, а по до три, и не струва сокет.

Второ, ако такава граница не съществува, тоест ако два протокола имат общи начала с произволна дължина, или ако при някой от тях сървърът говори пръв и клиентът не изпраща нищо, което да го идентифицира, класификацията след приемане не може да реши. Тогава разделянето изисква външен признак, тоест отделен номер на порт и с това отделен дескриптор, или договаряне в слой отдолу, каквото е ALPN. L получава допълнително събираемо и независимостта от $|P|$ се губи за този протокол.

Оттук и по-точната форма. Твърдението не е за първия октет, а за наличието на ограничен самоидентифициращ се префикс: равенството $L = A \cdot |T|$ важи за онова подмножество от P , чиито начални езици са попарно различни по префикс с ограничена дължина и при които клиентът говори пръв. Разделянето на TLS от текстов HTTP е частният случай, при който тази граница е единица. Формулировката с октета е по-конкретна и затова е използвана в основния текст, а тази е формата, която не се чупи при добавяне на протокол.

2.6. Изводи

Четирите основания се използват по различен начин в останалата част от работата.

Корутините без стек определят цената за връзка при компилиране, а не при изпълнение: кадърът е максимум по крайно множество от точки на спиране, докато стекът на нишка е предварително обявена горна граница. Симетричното прехвърляне прави дълбочината на стека независима от дължината на веригата от подновявания, тоест от натоварването.

Границата на собственост от раздел 2.3 е първото ново твърдение. Тя определя момента, след който кадърът престава да бъде собственост на подаващата страна, и превръща едно повтарящо се недоглеждане в правило за проектиране на интерфейс.

Детерминираният автомат прави времето за маршрутизиране независимо от броя на маршрутите. Това е твърдение, наследено от [39], което настоящата работа не доказва наново, а си поставя за задача да провери в контекста на пълна заявка. Проверката е формулирана като хипотеза X4, предварително е заявено какво би значел отрицателен резултат, и тя остава непроведена в кампанията от глава VI.

Предложението от раздел 2.5 е второто ново твърдение. То превръща централната теза в проверимо равенство, изброява петте си предположения заедно със статута на всяко от тях и посочва какво остава от равенството, ако най-слабото отпадне.

III. АРХИТЕКТУРА

Тази глава описва трите архитектурни решения, които носят приноса на работата, и ги подрежда по посоката на данните: как байтовете стигат до сървъра, как се разпределят между работните нишки и какво прави приложението с тях.

Раздел 3.1 разглежда демултиплексирането над TCP, където един слушащ дескриптор обслужва всички протоколи и разпознаването се извършва след приемането на връзката. Раздел 3.2 разглежда обратната задача над UDP, където един порт се обслужва от няколко сокета и въпросът е кой от тях трябва да получи даден пакет. Раздел 3.3 разглежда разделянето на API от изгледи и двата режима, по които изгледът получава данните си.

Трите решения са независими: всяко от тях остава смислено, ако останалите две отпаднат. Това е и причината да бъдат представени поотделно, а не като една архитектура, чиито части се обосновават взаимно.

След тях идват три по-къси раздела. Раздел 3.4 описва колко дълго връзката има право да мълчи и защо двата срока, които го налагат, са построени по различен начин. Раздел 3.5 описва общата опашка от таймери, върху която стоят и двата. Раздел 3.6 изброява решения, взети против, защото липсваща възможност изглежда еднакво, независимо дали е била отхвърлена или пропусната.

3.1. Демултиплексиране на един слушащ дескриптор

Централното архитектурно решение на настоящата работа е, че един слушащ сокет обслужва всички протоколи, пренасяни над TCP, а разпознаването кой е протоколът се извършва след приемането на връзката, а не преди него. Терминът *демултиплексиране* (англ. *demultiplexing*) се използва тук в точния му смисъл: разделяне на един входящ поток към няколко получателя по признак, съдържащ се в самия поток.

3.1.1. Задачата, поставена от изключващите се пътища

Исходното състояние на библиотеката прави решението необходимо. Методът, стартиращ сървъра, избира точно един от три взаимно изключващи се пътя според това дали е конфигуриран TLS, и блокира до края на изпълнението. Следствието не е неудобство, а структурно ограничение: за да обслужва едновременно HTTP и HTTPS, приложението трябва да създаде **два отделни екземпляра** на сървъра.

Цената на това не е един допълнителен дескриптор. Всеки екземпляр създава собствен контекст за вход и изход, следователно два пула от работни нишки, два цикъла на събития, два маршрутизатора с два пъти регистрирани маршрути, две вериги от middleware и два кеша за шаблони. Сесиите, метриките и пуловете от връзки не се споделят. Пренасочване от некриптиран към криптиран порт струва втори пълен сървър.

Броят на дескрипторите е видимият симптом. Структурната цена е дублирана среда за изпълнение, и именно тя е задачата, която архитектурата решава.

3.1.2. Класификация по първия октет

Разпознаването се основава на едно наблюдение за формата на входящите данни. Записът на TLS започва с поле `ContentType`, чиято стойност за ръкостискане е `0x16` [40]. Всеки метод на HTTP/1.1 започва с главна латинска буква, както и преамбюлът на HTTP/2, който започва с низа `PRI * HTTP/2.0`, следван от празен ред, `SM` и още един празен ред [9]. Двете множества не се пресичат: `0x16` е управляващ символ и не може да бъде начало на текстов метод.

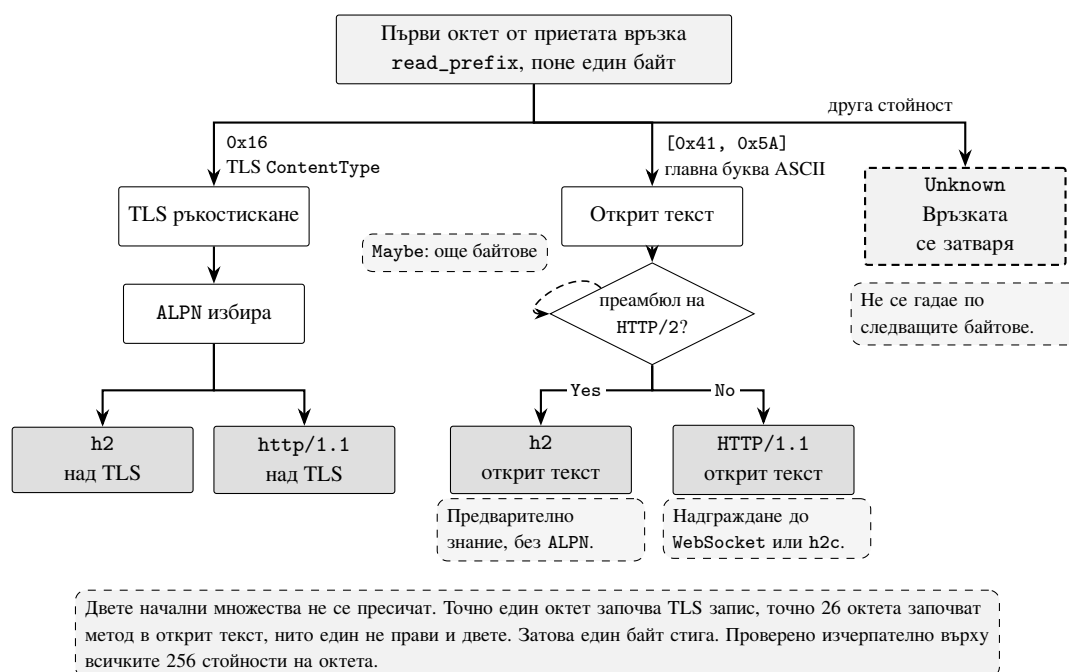
Един октет е достатъчен, за да се вземе първото решение. Това е и причината методът да е евтин: класификацията не изисква нито синтактичен анализ, нито изчакване на пълно съобщение.

```
1 enum class WireProtocol { Tls, Cleartext, Unknown };
2
3 WireProtocol classify(std::span<const std::uint8_t> prefix) noexcept
4 {
5     if (prefix.empty())
6         return WireProtocol::Unknown;
7
8     // ContentType на TLS запис за ръкостискане (RFC 8446).
9     if (prefix[0] == 0x16)
10         return WireProtocol::Tls;
11
12     // Всеки метод на HTTP/1.1 и преамбюлът на HTTP/2 започват с главна буква
13     ↩ .
14     if (prefix[0] >= 'A' && prefix[0] <= 'Z')
15         return WireProtocol::Cleartext;
16
17     return WireProtocol::Unknown;
18 }
```

Листинг 3: Класификация на протокола по първия октет

След първото разклонение решението продължава по вече съществуващи

механизми. При TLS изборът на протокол се прави от ALPN [1] по време на самото ръкостискане, тоест без допълнителен обмен. При некриптирана връзка се проверява дали началото съвпада с преамбюла на HTTP/2, което позволява h2c с предварително знание. Цялото разклонение е събрано на (Фиг. 4).



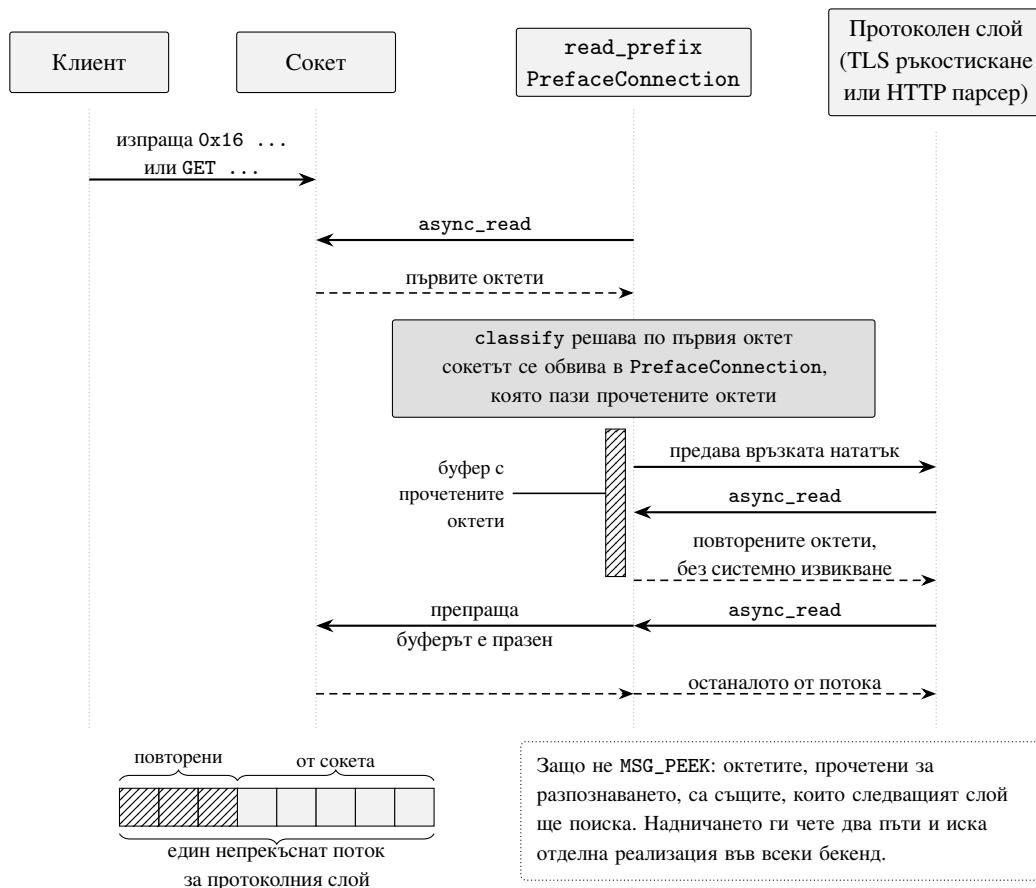
Фигура 4. Класификация по първия октет на приетата връзка. Един прочетен байт разделя TLS от открит текст, защото двете начални множества не се пресичат. При открит текст инкрементално сравнение с преамбюла на HTTP/2 разделя h2 с предварително знание от HTTP/1.1. Всяка друга стойност затваря връзката, вместо да бъде отгатната.

3.1.3. Прочитане с връщане вместо MSG_PEEK

Очевидната реализация е MSG_PEEK: прочитане на байта без изваждането му от буфера. Тя е отхвърлена, и мотивът е по-скоро архитектурен, отколкото свързан с производителността.

Байтовете, прочетени за класификация, са точно тези, от които следващият слой има нужда така или иначе. Редът на заявката отива в синтактичния анализатор на HTTP/1.1, записът ClientHello отива в ръкостискането на TLS, преамбюлът отива в инициализацията на HTTP/2. MSG_PEEK ги прочита два пъти и изисква отделна реализация във всеки от трите механизма за вход и изход. Пътят на байтовете е показан на (Фиг. 5).

Вместо това се използва декоратор върху връзката, който запазва прочетените байтове и ги връща на следващия читател. Броят на промените в механизмите за вход и изход е нула, а моделът вече съществува в кодовата база: обвиващата връзка на TLS работи



Фигура 5. Четене с връщане на прочетеното на пътя на приемане. `read_prefix` взема първите октети от сокета и `classify` решава по първия от тях. Сокетът се обвива в `PrefaceConnection`, която пази прочетеното. Протоколният слой получава първо повторените октети, после останалото от сокета, и вижда потока така, все едно никой не го е поглеждал.

по същия начин. Декораторът се композира и с нея, тоест TLS може да обвие връзката с връщане и да прочете повторения ClientHello без специален случай.

Отхвърлена оптимизация. Под Windows механизмът AcceptEx позволява приемането на връзка да върне и първите байтове от нея, с което класификацията би струвала нула допълнителни системни извиквания. Тя не се използва, защото задържа приемането до пристигането на данни, а това е отказ от обслужване при бавно свързващ се клиент. Семантиката на приемането се запазва еднаква на трите платформи, а оптимизацията се отбелязва като бъдеща работа.

3.1.4. QUIC и границата на подхода

Над UDP същият подход не е приложим и не е нужен. QUIC изисква TLS 1.3 без изключение [11], тоест няма некриптиран вариант, който да се разпознава. Затова демултиплексирането обхваща TCP, а UDP се обслужва от отделен сокет на същия номер на порт.

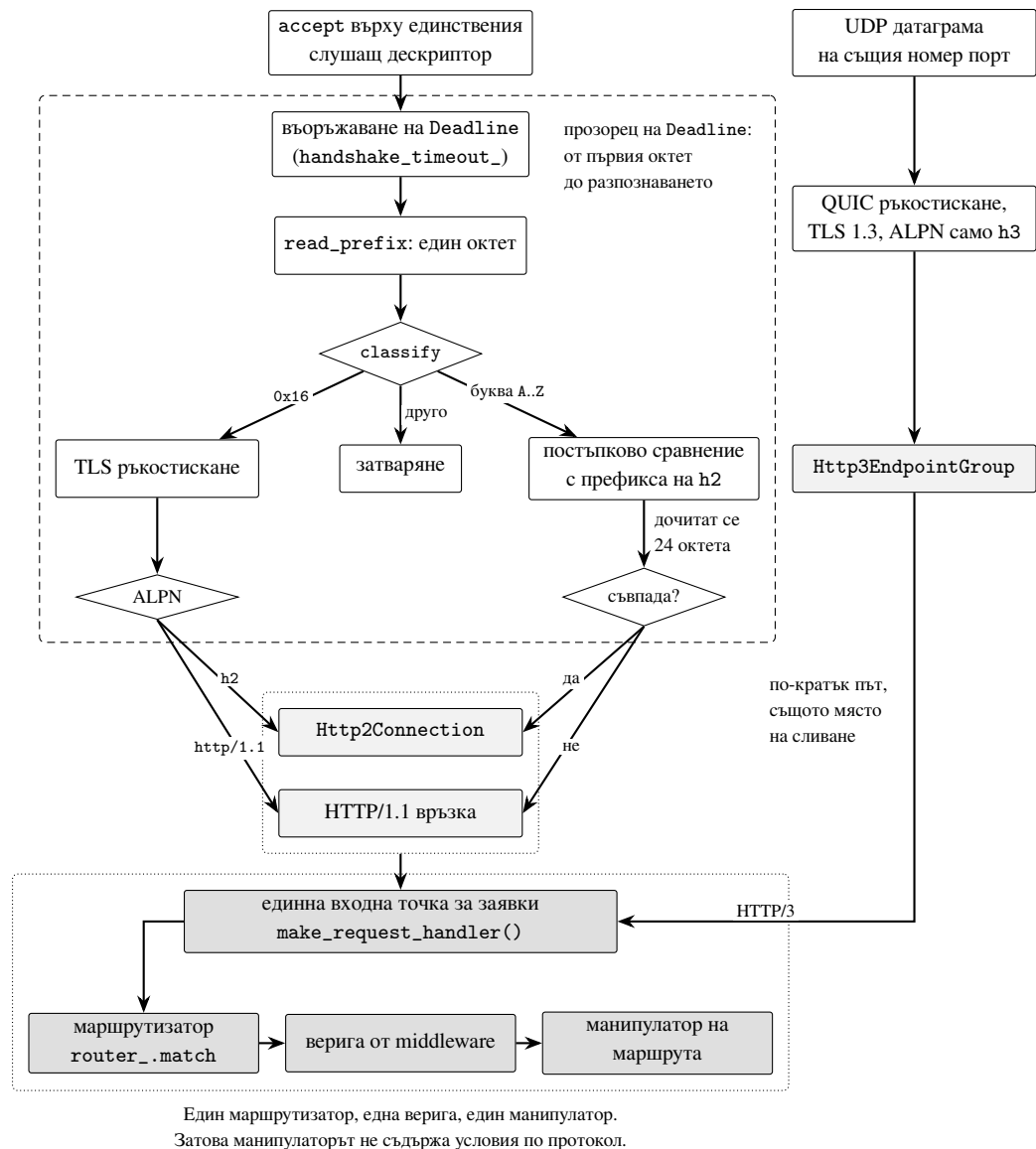
Това налага и разделяне на списъците за ALPN, което е лесно да се пропусне. Контекстът за TCP обявява h2 и http/1.1, но **не** обявява h3: клиент, който избере h3 над TCP, би договорил протокол, който никоя от двете страни не може да говори по тази връзка. Откриването на HTTP/3 става чрез Alt-Svc [6], изпращан от единствен middleware, който покрива и трите протокола.

3.1.5. Измерен резултат

Твърдението подлежи на проверка и е проверено. Един екземпляр на сървър, конфигуриран с един номер на порт, един сертификат и един набор от маршрути, обслужва HTTP/1.1, HTTP/2 и HTTP/3, като заявките минават през един и същи маршрутизатор, една и съща верига от middleware и едни и същи handler-и. Пътят, по който трите протокола стигат до един и същ маршрутизатор, е показан на (Фиг. 6).

Таблица 5. Слушащи дескриптори при четири работни нишки под Linux с backend io_uring. Числата следват от конфигурацията на този backend, а не от преброяване: преброяването от ядрото е направено под Windows и macOS и е дадено в раздел 6.12.

Транспорт	Дескриптори	Обслужвани протоколи
TCP	4	HTTP/1.1, HTTP/2
UDP	4	HTTP/3



Фигура 6. Път на приета връзка, от accept до манипулатора на маршрута. Един слушащ дескриптор носи и TLS, и открит текст: първият октет разделя двата случая, а Deadline покрива целия прозорец на разпознаването, включително ръкостискането. Четирите TCP изхода (TLS с h2, TLS с http/1.1, префикс на h2 по открит текст, HTTP/1.1) стигат до едно и също място. HTTP/3 идва по UDP по отделен, по-кратък път и се влива в същата точка.

Твърдението трябва да се формулира точно, защото по-силната му форма не издържа. То не е „един дескриптор общо“ навсякъде: под Linux сокетите за TCP са по един на работна нишка, за да може ядрото да разпределя връзките между нишките чрез `SO_REUSEPORT`, а по един сокет за UDP на работна нишка има само при `backend io_uring`; под macOS и Windows един споделен слушащ сокет носи няколко едновременни приемания, защото там `SO_REUSEPORT` или липсва, или не балансира. И в двата случая това е независима ос от броя на протоколите. Твърдението е, че **броят слушащи дескриптори се определя от модела на приемане на платформата и от броя транспорти, но не и от броя на обслужваните протоколи.**

Величината за сравнение с `nginx` следователно е броят слушащи дескриптори на транспорт. При `nginx` конфигурация, обслужваща едновременно некриптиран и криптиран трафик, изисква два, защото директивата `listen` не приема едновременно `ssl` и липсата му за един и същи сокет [13].

3.1.6. Ограничения на твърдението

Две ограничения се заявяват тук, а не в заключението.

Първо, браузърите не говорят некриптиран HTTP на порт 443. Практическата полза следователно е един изложен порт и по-малка таблица от сокети, а не наполовина по-малко портове. Броят на дескрипторите е косвена мярка.

Второ, действително измеримото е цената на класификацията, а не броенето на дескриптори. Тя се измерва в глава VI чрез сравнение на два режима на един и същи двоичен файл, избирани по флаг при изпълнение. Компилирането на два отделни файла е недопустимо тук: разликите между сборки, дължащи се на подредбата на кода и на реда на свързване, са от порядъка на пет до десет процента, докато разликите между изпълнения на една и съща машина са един до два процента.

3.2. Разпределяне на QUIC връзки по идентификатор

Разделът по-горе разглежда TCP, където един слушащ дескриптор обслужва няколко протокола. Над UDP задачата е обратната: при `backend io_uring` един порт се обслужва от няколко сокета, по един на работна нишка, и въпросът е кой от тях трябва да получи даден пакет; при `epoll` сокетът е един, а под `kqueue` и `IOCP` сокет за дейтаграми изобщо не се създава, тоест въпросът не се поставя.

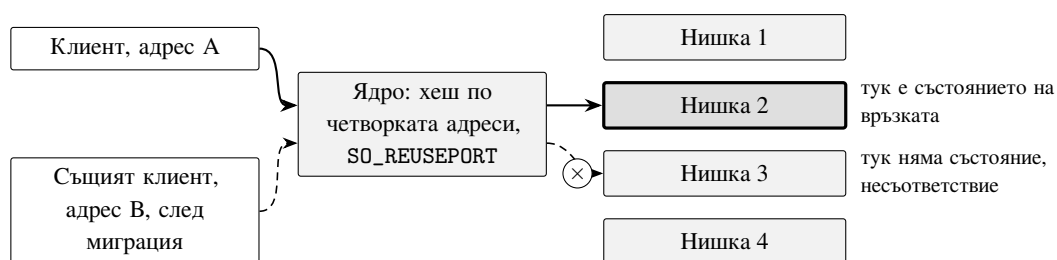
3.2.1. Защо хешът по четворка не работи за QUIC

Механизмът `SO_REUSEPORT` разпределя входящите дейтаграми между сокетите, като хешира четворката източник и получател. За транспорт, при който връзката е четворката, това е достатъчно, и TCP е такъв транспорт.

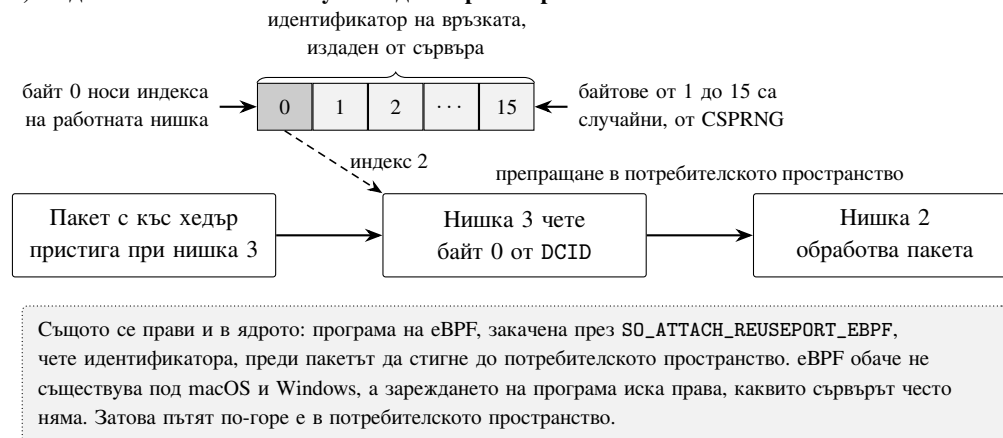
QUIC не е. Връзката преживява смяна на адреса на клиента по замисъл, за да може телефон, преминаващ от безжична мрежа към клетъчна, да запази сесията си [5]. В мига, в който това се случи, четворката се променя, хешът се променя, и дейтаграмите пристигат при работна нишка, която не знае нищо за връзката.

Същото се случва и без миграция. Клиентът има право да смени използвания идентификатор на връзката, а сървърът е длъжен да предостави няколко, така че разпределението по четворка и действителната принадлежност на връзката се разминават по повече от една причина. Разминаването и отговорът на него са съпоставени на (Фиг. 7).

а) Хешът по четворката греша след миграция



б) Индексът на нишката пътува в идентификатора



Фигура 7. Разпределяне на QUIC връзки по работни нишки. Горе: `SO_REUSEPORT` хешира четворката адреси, затова след миграция на клиента дейтаграмите отиват при нишка, която няма състояние за връзката. Долу: байт нула на идентификатора носи индекса на притежаващата нишка, останалите байтове са криптографски случайни, и погрешно доставеният пакет се препраща в потребителското пространство до притежателя.

3.2.2. Отговорът е вътре във въпроса

Това, което действително идентифицира QUIC връзка, е идентификаторът на връзката (англ. *connection ID*), и той се избира от сървъра. Следователно сървърът може да постави отговора вътре във въпроса: байт нула на всеки идентификатор, който издава, носи индекса на работната нишка, която го притежава.

Възстановяването на притежателя от погрешно доставен пакет е един достъп до масив.

```
1 // RFC 9000, раздел 5.1: идентификаторът е най-много 20 байта.
2 inline constexpr std::size_t max_cid_length = 20;
3 inline constexpr std::size_t server_cid_length = 16;
4
5 bool cid_fill(std::span<std::uint8_t> out, std::size_t worker_index) noexcept
6 {
7     if (out.empty() || out.size() > max_cid_length)
8         return false;
9
10    // Останалите байтове са криптографски случайни. Това не е украса:
11    // предвидим идентификатор позволява на наблюдател извън пътя да вмъква
12    // пакети в чужда връзка, тоест е пробив в сигурността, а не подробност
13    // за производителността.
14    if (RAND_bytes(out.data(), static_cast<int>(out.size())) != 1)
15        return false;
16
17    out[0] = static_cast<std::uint8_t>(worker_index);
18    return true;
19 }
20
21 std::size_t cid_worker(std::span<const std::uint8_t> dcid,
22                        std::size_t worker_count) noexcept
23 {
24     if (dcid.empty() || worker_count == 0)
25         return 0;
26     return static_cast<std::size_t>(dcid[0]) % worker_count;
27 }
```

Листинг 4: Записване и разчитане на индекса на работната нишка

Изискването за случайност идва от стандарта, а не от предпазливост: RFC 9000 изисква идентификатори, които наблюдател извън пътя не може да отгатне [5]. Когато източникът на случайност е недостъпен, връзката се отказва, вместо да получи слаб идентификатор.

3.2.3. Кои пакети се препращат

Препращат се само пакети с къс хедър, и това ограничение е съществено.

Първата дейтаграма, която клиентът изпраща, носи идентификатор на получателя, *избран от клиента*. Разчитането на индекс от него би било разчитане на случайни байтове, и по-голямата част от новите връзки биха били препратени към произволна нишка без никаква причина. Затова пакет с дълъг хедър и непознат идентификатор се приема на място, а не се препраща.

След ръкостискането клиентът използва идентификатора, издаден от сървъра, а пакетите са с къс хедър. Точно тогава миграцията става възможна, тоест точно тогава препращането има смисъл. Редът на проверките следва това:

1. версия, която сървърът не поддържа, получава Version Negotiation;
2. познат идентификатор се обслужва на място;
3. непознат идентификатор с къс хедър се препраща към нишката, записана в него;
4. непознат идентификатор с дълъг хедър създава нова връзка;
5. всичко останало получава Stateless Reset.

Търсенето предхожда приемането, за да не създаде повторно изпратена първа дейтаграма втора връзка за клиент, който вече има една.

3.2.4. Сравнение с решението в ядрото

nginx и Angie решават същата задача с програма на eBPF, закачена през SO_ATTACH_REUSEPORT_EBPF, която разчита идентификатора още в ядрото и насочва дейтаграмата, преди потребителското пространство да я види. Където е налична, тя е по-бърза от препращането в потребителско пространство.

Тя обаче не е налична навсякъде: eBPF не съществува под macOS и Windows, а зареждането на програма изисква права, каквито сървърът често няма. Реализацията тук е преносимата половина на същата идея.

3.2.5. Величината, която решава спора

Дали решението в ядрото си струва, е въпрос за измерване, а не за предположение. Ако делът на пакетите, изискващи препращане, е малък, то оптимизира нещо, което почти не се случва, и това е резултат, а не пропуск.

Наличните измервания сочат натам, без да отговарят. Миграцията е свойство, което

QUIC въвежда съзнателно и което е част от мотивацията за протокола [41], но проучване на разгърнатите сървъри установява, че значителна част от най-посещаваните услуги изобщо не я поддържат [21]. Освен това реализациите се различават съществено в избора си на дължина и на съдържание на идентификатора [42], тоест поведението не следва еднозначно от спецификацията. Нито едно от тези наблюдения не дава дела на препратените пакети при сървър с няколко работни нишки: това е величина от страна на сървъра и се измерва тук.

Затова всяка крайна точка брои: получени дейтаграми, препратени навън, приети отвън. Частното `forwarded_in` към `received` е числото, което настоящата архитектура трябва да произведе.

Състояние на измерването. Механизмът е реализиран, но пътят за препращане **още не е бил задействан**: нищо в текущите проверки не мигрира, тоест наблюдаваната стойност е нула препращания при сто получени дейтаграми. Това не е резултат, а липса на резултат. Принудителната миграция изисква два адреса на клиента, тоест двойката мрежови пространства, свързани с `veth`, която глава V приема за метод. Тя не е проведена: глава VI я отчита сред измерванията, които кампанията не е направила, раздел 6.13.

Windows и macOS. Там сокет за дейтаграми изобщо не се създава: създаването му е реализирано само за `epoll` и `io_uring`, а базовата реализация връща нищо, тоест крайна точка за HTTP/3 не може да бъде отворена. Пътят за препращане следователно не носи никакъв дял от пакетите под тези две платформи, защото пакети по него не минават, и числа за HTTP/3 оттам не се докладват.

3.3. Разделяне на API и изгледи, и отложено получаване на данни

Двата предходни раздела се занимават с това как байтовете стигат до сървъра. Този се занимава с това какво прави приложението с тях, и по-точно с една асиметрия: изгледът има нужда от данни, които API-то вече предоставя, и въпросът е дали да го изчака.

3.3.1. Извикване без сокет

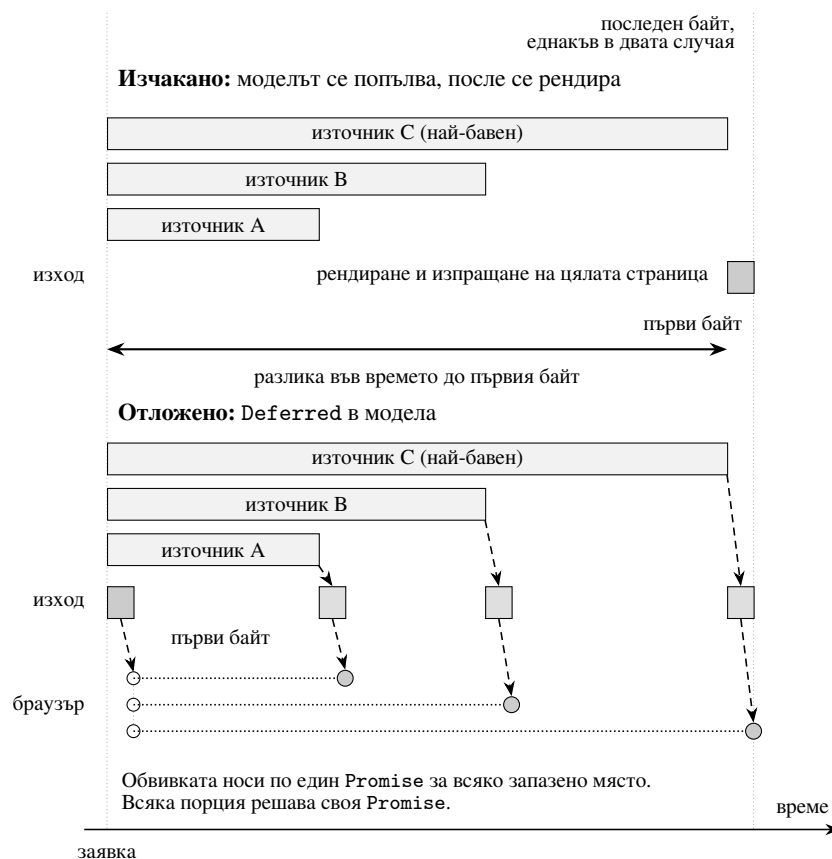
Маршрутите за изгледи се държат отделно от маршрутите за API, всеки със собствен маршрутизатор. Разделението не е само организационно: то позволява изгледът да поиска данните си от API-то, без заявката да напуска процеса.

Извикването се извършва вътре в процеса, тоест без сокет, без преминаване през мрежовия стек и без изграждане и разбор на HTTP съобщение. Тялото обаче остава сериализирано: Response носи тяло от тип низ, тоест API-то сериализира стойността, а изгледът я разчита обратно. Алтернативата, при която изгледът изпраща HTTP заявка към собствения си сървър, е разпространена и струва пълно кръгово пътуване плюс сериализация в двете посоки, за да се получи стойност, която се намира на няколко кадъра по-надолу в стека.

3.3.2. Двата режима на извикване

Оттук следват два режима, и разликата между тях е кой чака.

При *изчакано* извикване изгледът изчаква стойността. Тя присъства в модела, преди да започне изобразяването, и страницата пристига пълна. Това е правилното поведение, когато данните са бързи. Двата режима са съпоставени върху обща времева ос на (Фиг. 8).



Фигура 8. Изчакано и отложено рендиране върху обща времева ос. Горе моделът се попълва изцяло, преди да тръгне какъвто и да е байт, затова първият байт съвпада с края на най-бавната заявка. Долу обвивката на страницата тръгва веднага, със запазени места, а всяка стойност пътува като отделна порция, щом пристигне. Последният байт е на едно и също място в двата случая, защото и в двата се определя от най-бавната заявка. Спестява се време до първия байт, а не общо време.

При *отложено* извикване изгледът не чака. Страницата се изпраща веднага, с дупка на мястото на стойността, а стойността се предава по-късно, когато пристигне. Това е правилното поведение, когато една бавна заявка иначе би задържала иначе бърза страница.

Отложеният режим не е безплатен: отговорът остава отворен, а клиентът се нуждае от начин да разбере стойност, която още я няма.

3.3.3. Защо това не е ранно изпращане

Ранното изпращане на част от страницата не е ново. BigPipe го прави от 2010 г. Приносът тук е в естеството на запълнителя.

Запълнителят не е маркер за подмяна на елемент в дървото на документа. Той е сървърната страна на Promise: типът `Deferred<T>` в C++ има съответствие от другата страна на връзката, което кодът на страницата може да изчака, да композира с `Promise.all` и към което може да закачи обработка на грешка. Конвенция, при която сървърът подменя съдържанието на елемент, не изразява нито едно от трите.

Тоест приносът е **типизиран договор, обхващащ компилиран и скриптов език**, а ранното изпращане е механизмът, който го пренася.

```
1 struct Dashboard
2 {
3     std::string title;
4     int visitors_now;
5     Deferred<std::string> slow_report;    // започва да се изпълнява веднага
6 };
```

Листинг 5: Модел на изглед със стойност, която още не е пристигнала

Работата започва при конструирането, а не при първото четене. Мързеливото изчисление би подредило няколко отложени полета едно след друго, тоест би сериализирало точно това, заради което си струва да се отлага.

3.3.4. Как се откриват неготовите полета

Полетата, които още нямат стойност, се събират по време на сериализацията на модела, а не се декларират от него. Декларацията би повторила това, което преобразуването към JSON така или иначе обхожда, и би остаряла при първото добавено поле, което някой е забравил да впише. Събирането при записването на запълнителя

означава, че двете не могат да се разминат.

Разрешена стойност се сериализира като себе си и не струва на страницата нищо. Неразрешена стойност без активен събирач се сериализира като `null`, а не като запълнител: щом никой не предава, запълнителят би бил обещание, което страницата никога не изпълнява, а дупка, която остава празна завинаги, е по-лоша от честна липса.

3.3.5. Границата на сигурност

Стойността се вгражда в елемент `<script>`, и това е граница на сигурност, а не въпрос на форматиране. Синтактичният анализатор на HTML търси затварящия етикет вътре в елемента, преди JavaScript изобщо да види съдържанието, тоест стойност, съдържаща `</script>`, прекратява блока предсрочно и всичко след нея се разчита като разметка. Достатъчен е низ, предоставен от потребител, което прави това съхранен XSS на всяка страница с отложено поле.

Затова знаците `<`, `>` и `&` се заместват с `\u` форма, невидима за `JSON.parse` и безвредна за анализатора на HTML. Знаците `U+2028` и `U+2029` също се заместват: те прекратяват ред в JavaScript, но не и в JSON.

3.3.6. Резервен път без JavaScript

Запълнителят носи и атрибут, който средата за изпълнение попълва пряко. Страница, която не пише собствен скрипт, се запълва; страница, която пише, получава типизирания вариант. Двата пътя съществуват едновременно, а не един вместо друг.

3.3.7. Известна граница на реализацията

Отложените полета се изпращат в реда на своите позиции, а не в реда на пристигане. Тъй като всички започват при конструирането, те се изпълняват едновременно и общото изчакване е това на най-бавното при всеки от двата подхода. Цената на подредбата е, че стойност, пристигнала рано, чака зад по-бавна, преди да достигне страницата. Първото изпращане, което носи основната полза, не е засегнато.

3.3.8. Състояние на измерването

Свойството за подредба е проверено функционално: обвивката на страницата пристига преди стойността, а не заедно с нея. Числената разлика обаче **не се съобщава тук**, и причината е методологична, а не техническа.

Функционалната проверка е извършена на виртуализирана машина. Правилата за допустимост в глава V отхвърлят такава машина като източник на записи за производителност, и това правило се прилага и към собствените резултати на настоящата работа, а не само към сравняваните системи. Времената за изчакано срещу отложено изобразяване не идват и от кампанията в глава VI: тя ги отчита сред неизвършените измервания, защото loadgen не изобразява страница и не отделя времето до първия байт от времето до последния, раздел 6.13. Местата за тях са **[?coroute.deferred.ttfb]** срещу **[?coroute.awaited.ttfb]** милисекунди.

Докато измерванията не съществуват, тези две места се отпечатват като видим маркер за липсваща стойност, а не като правдоподобно число.

3.4. Животът на връзката и неговите срокове

Трите раздела дотук описват къде отиват байтовете. Този описва колко дълго връзката има право да не изпраща никакви.

Исходното положение е дефект, а не липса. Интерфейсът на връзката има метод `set_timeout`, и четирите механизма за вход и изход го приемат и запазват стойността. Никой от четирите не действа на нея. Тоест извикващият е писан срещу интерфейс, който не работи: цикълът за keep-alive задава срок и обработва `Error::is_timeout()` в пътя си за грешки, а този клон е недостижим. Клиент, който завърши заявка и след това замълчи, държи корутина, докато не се разкачи сам. Демултиплексирането добавя и втори такъв прозорец, между приемането на връзката и момента, в който протоколът е известен, и той е отворен от настоящата архитектура, тоест затварянето му е нейно задължение.

Двата прозореца се затварят от два различни механизма, защото имат различна форма. Първият е ограничен и има точно един край. Вторият няма край: връзката с keep-alive живее, докато я използват, и въпросът не е дали е изтекло време, а дали нещо се е случило през него.

3.4.1. Прозорецът на класификацията е обхват, а не поредица от изходи

Срокът върху класификацията е обект със състояние, чийто деструктор го обезврежда. Очевидната алтернатива е извикване при всеки изход от прозореца. Изходите са единадесет: мълчалив партньор, липсващ сертификат, неуспешно създаване на TLS

връзка, неуспешно ръкостискане, договорен h2, договорен http/1.1, сборка без вградена поддръжка на TLS, недочетен преамбюл, съвпаднал преамбюл, несъвпаднал преамбюл и неразпознат първи октет.

Единадесет места, на които трябва да се помни едно и също извикване, е същата форма като състезанието при подновяване от `await_suspend` в раздел 4.2: когато обикновената употреба на един интерфейс съдържа лесна грешка, тази грешка се прави. Затова прозорецът е обхват, а излизането от обхвата е обезвреждането. За да е възможно това, разпознаването на протокола е отделна корутина, а не клон вътре в обслужването на връзката.

```
1 Task<std::optional<Detected>> App::detect_protocol(std::unique_ptr<Connection
  ↳ > conn)
2 {
3     // Обезвреждането е деструкторът и нищо друго.
4     net::Deadline deadline(*io_ctx_, handshake_timeout_,
5                             [c = conn.get()] { c->close(); });
6
7     auto prefix = co_await net::read_prefix(std::move(conn), 1);
8     if (!prefix)
9         co_return std::nullopt;
10
11     std::unique_ptr<net::Connection> stream = std::move(prefix->conn);
12
13     // Обвивката, а не сокетът под нея, е това, от което сега се чете.
14     deadline.replace([c = stream.get()] { c->close(); });
15     // ...
16 }
```

Листинг 6: Прозорецът като обхват

Цената е едно вмъкване в опашката от таймери на приета връзка, а не на прочетен байт. Спрямо приемането по TCP, и спрямо ръкостискането по TLS в клона, който има такова, това не е величина, която сравнението в глава VI може да раздели.

3.4.2. Сроктът следва това, върху което действа

Действието на срока е затваряне на връзката, но „връзката“ се сменя до три пъти вътре в прозореца. Прочитането на префикса връща обвивката, която връща прочетените байтове на следващия читател, а не сокета под нея. Клонът за TLS обвива още веднъж. Проверката за преамбюл на HTTP/2 дочита буфера и връща трета обвивка.

Затварянето на обекта, с който срокът е започнал, не би затворило нищо, от което някой чете, и партньорът щеше да остане свързан след изтичането на собствения си

срок. Затова обектът има операция, която пренасочва действието, и тя се извиква след всяко обвиване. Операцията умишлено не прави нищо, ако срокът вече е изтекъл или е обезвреден: изтекъл прозорец не се отваря отново.

Ръкостискането по TLS е вътре в прозореца по решение, а не по недоглеждане. Партньор, който отвори с 0x16 и след това спре, е същата атака като този, който не казва нищо.

3.4.3. Обезвреждането трябва да значи „обезвредено и не се изпълнява в момента“

Действието се изпълнява на работна нишка, а обектът, върху който то действа, се притежава от кадър на корутина, който може да се изпълнява на друга нишка. Флаг не е достатъчен: кадърът може да унищожава връзката точно между проверката на флага и извикването.

Затова състоянието е споделено и се пази от mutex. Това превръща обезвреждането в обещание, а не в намек: действието или вече е приключило, или никога няма да започне. Споделеното състояние надживява обекта при всяка връзка, обслужена по-бързо от срока, тоест при всяка здрава връзка, и това е причината то да има собствено време на живот.

3.4.4. Срокът при бездействие е декоратор, а не четири реализации

Вторият прозорец се затваря като връзка, която обвива друга връзка, тоест над backend-ите, а не вътре в тях. Мотивът е измерването, а не удобството. Реализация вътре в цикъла означава структура със срокове на дескриптор на четири места, три от които не се компилират на машината, на която е писан кодът. Резултатът би бил срок, който се държи различно при всеки backend, а точно това сравнението между интерфейс по готовност и интерфейс по завършване не може да си позволи: разликата, която то мери, трябва да идва от механизма за вход и изход, а не от това как всеки от тях е решил да брой времето.

Над backend-а реализацията е една: същият клас се компилира за четирите backend-a. Платформено остават две неща — доставянето на callback-a, което минава през post на съответния цикъл, и часовникът, който под Linux е грубият монотонен, а другаде steady_clock. Моделът вече съществува в кодовата база: обвиващата връзка на TLS и обвивката с връщане на байтове от раздел 3.1 работят по същия начин, и новата се композира с тях, а не се конкурира.

Стойността продължава да се предава надолу. Backend, който по-късно получи

истинска реализация, ще се съгласи с тази, вместо да се състезава с нея.

3.4.5. Бездействие, а не срок на операция, и каква грешка вижда извикващият

Срок на всяка операция би означавал планиране при всяко четене. Вместо това таймерът се зарежда сам отново и сравнява спрямо момента, в който за последно е преминал байт. Всяко четене и всяко писане записва този момент. Оттук следват две свойства: цената на четене е едно прочитане на часовника и едно записване на атомарна променлива с най-слабата наредба, тоест величина, която не се вижда до системното извикване до нея — но само защото часовникът под Linux е грубият монотонен; с обикновения монотонен часовник прочитането само по себе си е системно извикване и е по-скъпата от двете части. Връзка, която се използва, изобщо не докосва опашката от таймери, защото планиране има веднъж на период на бездействие, а не веднъж на заявка.

Четене, което е пропаднало, след като таймерът е затворил сокета, се съобщава като изтекъл срок, а не като това, което backend-ът прави от затворен дескриптор. Без тази подмяна извикващият влиза в клона си за грешки и се опитва да запише 400 в сокет, който вече го няма, вместо в клона, който казва, че клиентът е замълчал. Тоест подмяната не е козметика: тя е това, което прави недостижимия клон достижим.

3.4.6. Какво е проверено и какво остава

Проверката е функционална и е направена от край до край, а не само с единични тестове. Тя е изпълнима и се намира в `tests/integration/connection_deadlines.py`, тоест числата по-долу се получават наново, а не се цитират по памет.

Проверката пуска сървъра сама, защото сроковете са конфигурация на изпълнението, и обхваща пет твърдения. При зададен срок за ръкостискане от една секунда мълчалив партньор се затваря след около една секунда, а при срок нула е още отворен след четири секунди. При срок за `keep-alive` от 1,5 секунди бездействаща връзка се затваря след около 1,5 секунди, а при нула е още отворена след пет секунди. Три последователни заявки минават по една връзка и при двата включени срока.

Последното твърдение е това, което би уловило прекалено усърдна поправка: срок, който затваря и връзка, по която тече работа, минава първите четири проверки и се проваля на петата. Допускът е широк по нарочно съображение. Точната стотна от секундата зависи от натоварването на машината и няма отношение към твърдението, докато срок, който

изобщо не сработва, се проваля при всякакъв допуск.

Тези числа са проверка на поведение, не измерване на производителност, и не се докладват като резултат. Правилата за допустимост в глава V важат и тук.

Две ограничения се заявяват на същото място. Първо, включването е решение при конструиране: последващо задаване на стойност променя периода, но не може да пусне таймер, който никога не е бил зареден. Второ, това затваря двата прозореца, които архитектурата добавя, но не поправя `set_timeout` в четирите backend-a. Това остава промяна на backend, а не на слоя над тях.

3.5. Опашката от таймери

И двата срока искат едно и също нещо от цикъла на събития: callback след изтекло време. Методът за това вече съществуваше в интерфейса. Три от четирите backend-a го реализираха, като пускаха отделна нишка, която спи за исканото време и след това публикува callback-a.

За няколко еднократни таймера това е поносимо. За каквото и да е на връзка е разорително: срокът върху класификацията щеше да струва по една спяща нишка на приета връзка, тоест щеше да е по-скъп от атаката, която затваря. Backend-ът върху IOCP вече имаше истинска опашка, така че общият механизъм е тя, извадена навън, а не нов дизайн.

Архитектурно опашката има три свойства, и всяко от тях е решение.

Една нишка на контекст, пусната при първа употреба. Контекст, който никога не планира нищо, не плаща за нея. Това е важно, защото контекстите не са един: сравнението между backend-ите ги създава по един на конфигурация.

Callback-ите се доставят през цикъла. Нишката на таймера не изпълнява callback, а го подава на същия механизъм за публикуване, който използва всичко останало. Следствията са три. Нишката на таймера никога не докосва състоянието на цикъла. Callback, който планира нов таймер, не блокира сам себе си, защото mutex-ът на опашката не се държи през подаването. И наредбата, която callback-ът вижда, е тази на цикъла, а не на втори планировчик до него.

Часовникът е монотонен. Използва се `steady_clock`, а не `system_clock`, каквото беше в реализацията върху ИОСР. Таймер не бива да се мести, когато стенният часовник бъде коригиран от NTP или от потребител. Срок за ръкостискане от десет секунди, който корекция на часовника превръща в десет минути, е прозорец за атака от вида `slowloris`, който се появява от нищото и не може да бъде възпроизведен.

Callback-ите, останали в опашката при спиране, се изхвърлят, а не се изпълняват. Това също е решение, а не пропуск: `callback`, който задължително трябва да се случи при спиране, не принадлежи на таймер. Нишката на таймера се присъединява преди освобождаването на ресурсите на цикъла, а не след него, защото обратният ред позволяваше `callback`, задействан по време на спиране, да публикува към вече затворен обект на ядрото.

3.6. Какво архитектурата отказва да прави

Липсваща възможност изглежда еднакво, независимо дали е била отхвърлена или пропусната. Разликата обаче е съществена, затова три отказа се записват тук заедно с мотивите си.

Отделна абстракция за сокет. В заглавния файл на цикъла стои декларация напред за клас `Socket`. Такъв клас няма никъде в дървото: декларацията е останка от слой, който е бил предвиден и не е построен.

Не е построен, защото нищо над цикъла не иска сокет. Иска се или връзка, тоест четене, писане, затваряне, срок и адрес на партньора, или слушател, тоест приемане. Междинният слой би имал по една реализация на `backend` и нито един извикващ над себе си, а трите декоратора върху връзка, TLS обвивката, обвивката с връщане на байтове и срокът при бездействие, щяха да трябва да решат кое от двете обвиват. Това е избор без правилен отговор, добавен без причина.

Същият отказ важи и над UDP. Сокетът за дейтаграми умишлено не е API за UDP от общ вид: формата му е тази, която е нужна на крайна точка на QUIC, тоест пакетно получаване с прикачен локален адрес и изпращане, което може да носи размер на сегмент за GSO.

Пренаписване на HTTP/2 върху сесийния слой на `nghttp2`. Библиотеката вече е зависимост на сборката, тоест това е налична възможност, а не хипотеза. От нея се използва

единствено HPACK.

Критерият, по който се взема или отказва зависимост за протокол, е формулиран при HTTP/3 и се прилага и тук: взема се библиотека, която не притежава нито сокет, нито цикъл на събития, защото такава сяда върху вече съществуващия цикъл, вместо да се състезава с него за дескриптора. `ngtcp2` и `nghttp3` минават този критерий, и това е причината да бъдат избрани, а не удобството.

Сесийният слой на `nghttp2` е от другия вид: той поема цикъла на рамките и таблицата на `stream`-овете, тоест точно кода, който тук вече е написан и проверен. Печалбата би била по-малко собствен код по механичната част. Цената е втора миграция на път, критичен за съвместимостта, при която целият набор от проверки трябва да се пусне отново срещу друга машина на състоянията. Защо HPACK е взет, а рамкирането не, е обосновано в глава IV.

Server push. HTTP/2 позволява на сървъра да изпрати ресурс, който клиентът не е поискал. Тук той не се поддържа, и отказът е обявен, а не мълчалив: настройката `SETTINGS_ENABLE_PUSH` се изпраща със стойност нула, а `PUSH_PROMISE`, дошъл от клиент, получава `GOAWAY` с `PROTOCOL_ERROR`, защото такъв кадър от клиента е нарушение на протокола [9].

Причината да не се поддържа е архитектурна. Push изисква `handler`-ът да може да каже „изпрати и това“, тоест интерфейс върху маршрутизатора и веригата от `middleware`. HTTP/1.1 няма такова понятие. Един и същ `handler` би получил възможност, която се изпълнява по два от трите протокола и мълчи по третия, а премахването точно на такова разклонение е това, което раздел 3.1 постига.

3.7. Изводи

Трите решения имат обща форма, която си струва да се назове: всяко от тях премества решение, взимано преди дадена операция, до момента след нея, когато вече има информация да бъде взето правилно.

Демултиплексирането отлага избора на протокол до след приемането на връзката, вместо да го кодира в избора на слушащ сокет. Разпределянето по идентификатор отлага избора на работна нишка до разчитането на самия идентификатор, вместо да го остави на хеш върху четворка, която QUIC има право да смени. Отложеното получаване отлага

изпращането на стойността до пристигането ѝ, вместо да задържи цялата страница.

Измеримото следствие е едно кратко описание на дескрипторите: под Linux един слушащ дескриптор за TCP на работна нишка, а за UDP по един на работна нишка само при backend `io_uring` и един общо при `epoll`; под macOS и Windows един споделен слушащ дескриптор за TCP и нито един за UDP — и във всички случаи независимо от броя на обслужваните протоколи (Табл. 5), раздел 6.12.

Механизмите за сроковете от раздели 3.4 и 3.5 имат друга обща форма, и тя също се назовава: грешката се прави невъзможна, а не малко вероятна. Прозорецът е обхват, за да не може изход да забрави обезвреждането. Срокът при бездействие е един декоратор, за да не могат четири backend-а да се разминат. Обезвреждането е под `mutex`, за да не може „обезвредено“ да означава „обезвредено, но вече започнало“.

Три твърдения обаче остават непроверени на този етап. Цената на класификацията е измерена в глава VI чрез два режима на един и същи двоичен файл. Другите две не са: глава VI отчита и дела на пакетите, изискващи препращане, и разликата между изчакано и отложено изобразяване сред измерванията, които тази кампания не е направила — първото, защото средата няма конфигурация за HTTP/3 и предизвикана миграция на един адрес е невъзможна, второто, защото `loadgen` не изобразява страница, раздел 6.13. До тогава те са твърдения за архитектура, а не за производителност.

IV. РЕАЛИЗАЦИЯ

Тази глава описва как решенията от глава III са изпълнени, и отделя непропорционално място на два класа грешки. Причината и в двата случая е емпирична, а не композиционна.

Първият клас е надпревара при подновяване на корутина. Същата грешка беше допусната три пъти на три различни места в кода, което я прави свойство на интерфейса, а не на невниманието.

Вторият клас излезе от насоченото фъзване и от подготовката за него. Десет дефекта в един работен цикъл, на пет различни повърхности, се оказаха с една и съща форма: ограничение, което съществува някъде в системата, но не и по пътя, по който минава непознат вход. Разделът 4.3 ги описва поотделно и после формулира формата, защото тя е по-полезна от кой да е от тях.

4.1. Един интерфейс, четири механизма за вход и изход

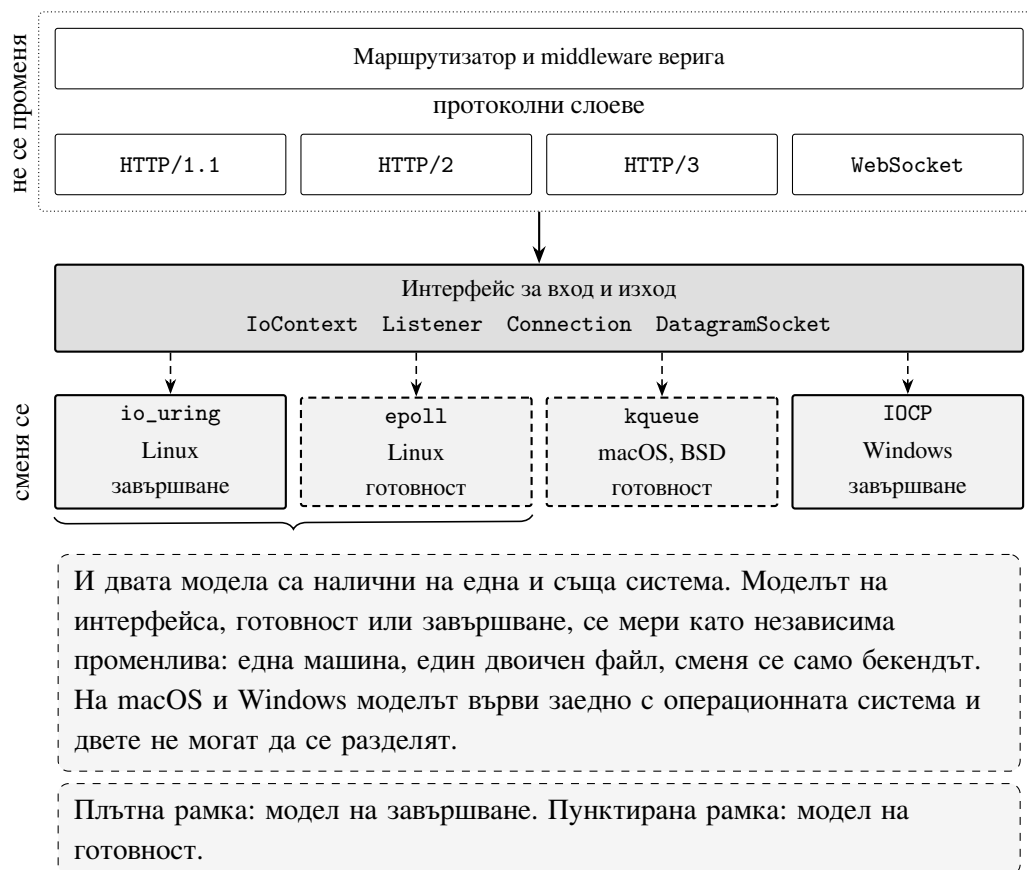
Четири механизма за вход и изход стоят зад един абстрактен интерфейс от около 160 реда. Той е умишлено тесен: колкото по-малко може да се изрази през него, толкова по-малко може да се различава между платформите по начин, който да се появи в измерванията. Какво остава от двете страни на шева е показано на (Фиг. 9).

Механизмите се делят на два вида, и делението е съществено за глава VI.

Интерфейсите по готовност съобщават, че даден дескриптор е готов, а самото системно извикване се извършва от викация. Такива са `epoll` и `kqueue`. *Интерфейсите по завършване* приемат заявка за операция и съобщават нейното завършване. Такива са `io_uring` и `IOCP`.

Делението има и история, която обяснява защо е точно това. Интерфейсите по готовност произлизат от установеното, че `select` не се мащабира с броя дескриптори, тъй като предава цялото множество при всяко извикване, и че е нужен механизъм, който съобщава само промените [43]. Интерфейсите по завършване тръгват от друга посока: не съобщаването да се съкрати, а самото системно извикване да бъде избегнато, като заявката и резултатът се разменят през споделена памет [44]. Двата вида решават различни задачи и затова не се подреждат един над друг по принцип, а само при измерване.

Под Linux са налични и двата, като изборът се прави при конфигуриране, а не



Фигура 9. Шев на абстракцията за вход и изход. Над шева стои всичко, което не зависи от бекенда: маршрутизаторът, middleware веригата и протоколните слоеве. Под шева стоят четирите реализации. Понеже io_uring и epoll се предлагат на една и съща операционна система, моделът на интерфейса може да се измери отделно от нея.

по платформа. Това е и причината да съществува реализацията върху `eroll`: без нея механизмът за вход и изход е напълно съвпадащ с операционната система и не може да бъде изследван като независима променлива. Почти всички публикувани сравнения между готовност и завършване сравняват две различни програми и следователно измерват програмите.

Едно място, на което разликата между двата вида не е количествена. Дълбочината на приемащия резерв не се пренася между тях, и това беше открито късно, при пренасянето на кампанията върху `macOS`.

При интерфейс по завършване всяка приемаща операция е отделна: `IOCP` държи резерв от едновременно висящи `AcceptEx` — по две на работна нишка и не по-малко от четири — всяка със собствено състояние, и дълбочината е действителна. При интерфейс по готовност наблюдението се определя от двойката дескриптор и филтър, а не от операцията: колкото и регистрации да бъдат направени върху един слушащ дескриптор, те се събират в едно наблюдение, а всички корутини освен една спират и никой не ги подновява. Реализацията върху `eroll` затова заявява дълбочина единица явно. Реализацията върху `kqueue` беше пренесена от `IOCP` заедно с този израз за дълбочината, който на нея не дава толкова наблюдения, колкото операции, а едно.

Поправката е дълбочината да бъде заявена като единица и там. По-дълбок резерв би изисквал по един дублиран дескриптор на позиция, за да се различават наблюденията, срещу по едно празно връщане на всяка позиция освен първата при всяко пристигане. Това е промяна в измерваната система и не се прави по време на кампания.

Следствието за глава VI е ограничение на обхвата, а не грешка в измереното: измерванията с устойчиви връзки не зависят от дълбочината, защото връзките се установяват веднъж, докато измерване по темп на пристигане на връзки би зависело и на двете платформи би описвало различна архитектура на приемане.

4.1.1. Една опашка от таймери вместо нишка на таймер

Тесният интерфейс има цена: всяко свойство, което не е изразено в него, се появява четири пъти или нула пъти. Планирането на отложено събитие е такъв случай.

Три от четирите реализации изпълняваха `schedule()`, като пораждаха откачена нишка, която спи за исканото време и после публикува. Това е поносимо за няколко еднократни таймера и разорително за каквото и да е на връзка: крайният срок за

ръкостискане, заради който изобщо потрѣбва планиране, би струвал по една спяща нишка на приета връзка, тоест повече от бавната атака, която този срок затваря.

Реализацията върху IOCP вече имаше истинска опаска, така че поправката е нейното изнасяне, а не нов дизайн. Една нишка на контекст, стартирана при първото използване, тоест контекст, който никога не планира, не плаща нищо.

Две отделни корекции дойдоха със същото преместване, и двете си струва да се назоват, защото нито една не е за производителност. Часовникът е `steady_clock`, а не `system_clock`: корекция на стенния часовник не бива да разтяга краен срок, а поправка от NTP, която превръща десетсекунден прозорец за ръкостискане в десетминутен, е прозорец за бавна атака, който се появява от нищото и не се възпроизвежда. И нишката на таймера вече се присъединява *преди* затварянето на порта за завършвания, докато предишната наредба я присъединяваше след него, тоест обратно извикване по време на спиране публикуваше към вече несъществуващ манипулатор.

4.1.2. Таймаут, който всеки backend пазеше и никой не спазваше

Същият тесен интерфейс произведе и по-неприятен случай, който тук се описва като дефект, а не като липсваща възможност.

Функцията `handle_connection` винаги е извиквала `set_timeout` с тридесет секунди за поддържане на връзката и винаги е имала клон за `Error::is_timeout()`. Нито едното не правеше нищо: всеки от четирите backend-а съхраняваше стойността и нито един не действаше по нея. Клонът беше недостижим, а клиент, който завърши заявка и после замълчи, държеше корутината, докато сам не се разкачи. Викацията беше написан срещу интерфейс, който не работи.

Поправката е декоратор над backend-а, а не четири реализации в него. Спазването на срока вътре в четирите цикъла на събития означава структури с крайни срокове на дескриптор на четири места, три от които не се компилират на тази машина, а резултатът би бил таймаут, който се държи различно при различните backend-и. Точно това сравнението между готовност и завършване не може да си позволи, защото би внесло разлика между сравняваните величини, която не е предмет на сравнението. Над backend-а реализацията е една: същият клас се компилира за четирите backend-а. Платформено остават две неща — доставянето на обратното извикване, което минава през `post` на съответния цикъл, и часовникът, който под Linux е грубият монотонен, а другаде `steady_clock`.

Срокът е за бездействие, а не за операция: таймерът се превъоръжава спрямо

последния преместен байт, тоест натоварена връзка изобщо не докосва опашката от таймери, а цената на четене е едно прочитане на часовника и едно отпуснато атомарно записване. От двете прочитането е скъпото: затова под Linux то минава през грубия монотонен часовник, който не е системно извикване, а там, където платформата няма такъв, през `steady_clock`. Четене, което се проваля, след като таймерът е затворил сокета, се докладва като таймаут, а не като каквото backend-ът прави от затворен дескриптор. Без това викацията влиза в клона за грешка и записва 400 в сокет, който вече го няма.

Проверено от край до край: при `--keep-alive-ms 1500` бездействащата връзка се прекъсва на 1,52 s, при 0 е още отворена след 5 s, а три заявки минават по една връзка при включен таймаут.

4.2. Опасността при подновяване от вътрешността на `await_suspend`

Тук се описва грешка, която беше открита три пъти: веднъж наблюдавана, два пъти по разсъждение. Тя заслужава отделен раздел, защото формата ѝ се повтаря при всеки асинхронен интерфейс и защото поправката е промяна на интерфейса, а не на реализацията.

4.2.1. Формата на грешката

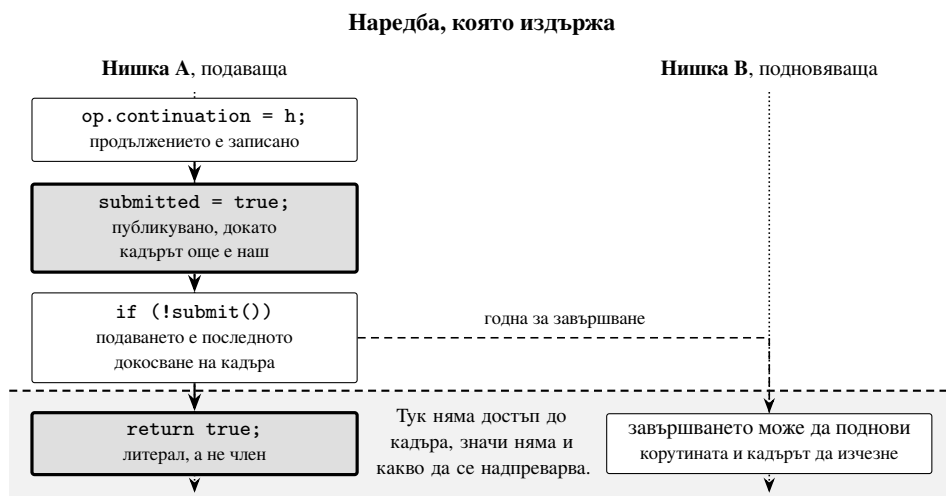
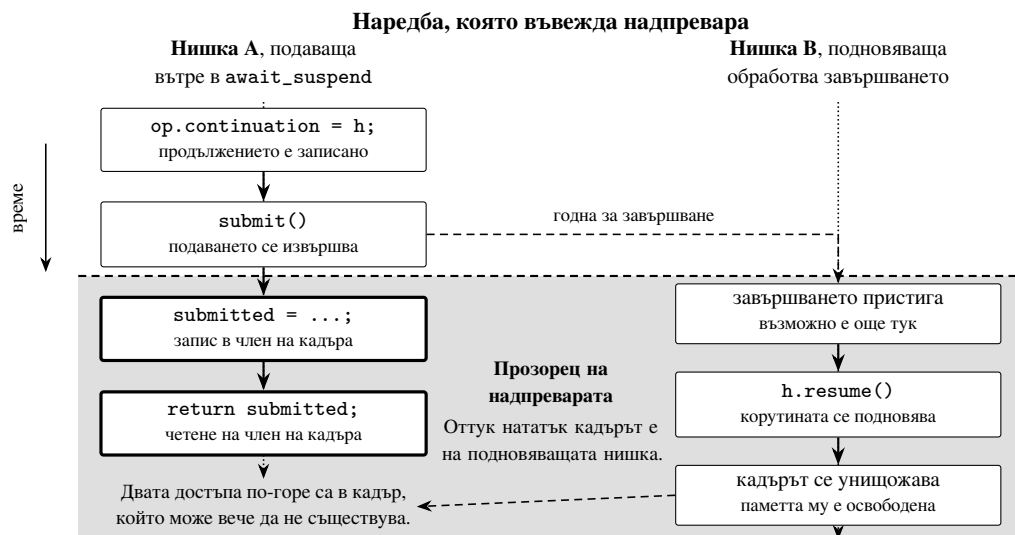
Асинхронната операция в модел с корутини има две части: записване на продължението, тоест дескриптора на спряната корутина, и подаване на самата операция. Очевидната наредба е подаване, после изчакване:

```
1 submit_operation(&op);           // операцията вече може да завърши
2 co_await SomeAwaiter{op};        // едва сега се записва продължението
```

Листинг 7: Наредба, която изглежда правилна

Между двата реда съществува прозорец. Ако операцията завърши вътре в него, а за дейтаграма, която вече чака в опашката на сокета, това се случва редовно, обработващият завършването намира празно продължение, изразходва завършването и корутината не се подновява никога. Операцията увисва. Прозорецът е защрихован на (Фиг. 10).

Естествената поправка е подаването да се извърши вътре в `await_suspend`, тоест след записването на продължението. Тя затваря прозореца и въвежда втори, по-коварен.



Фигура 10. Надпреварата за собственост върху кадъра на корутината, раздел 4.2. Горے: подаването прави операцията годна за завършване, затова всеки следващ достъп до член на кадъра от подаващата нишка е достъп до евентуално освободена памет. **Долу:** поправката публикува стойността преди подаването и връща литерал. Пътят на неуспеха, който записва `false` повторно, е безопасен по устройство, защото щом нищо не е подадено, нищо не може да подновява.

```

1 bool await_suspend(std::coroutine_handle<> h)
2 {
3     op.continuation = h;
4     submitted = submit();    // подаването позволява завършване ВЕДНАГА
5     return submitted;        // ... а дотук кадърът може вече да не съществу
    ↪ ва
6 }

```

Листинг 8: Поправка, която въвежда втора надпревара

Извикването `submit()` е точно това, което прави операцията годна за завършване. Работна нишка може да я обработи и да поднови корутината, докато `submit()` още се връща. Присвояването на `submitted` тогава се надпреварва с подновяването, което самото то е предизвикало, и губи: `await_resume` прочита флага, докато той още е `false`, и съобщава неуспешно подаване за операция, подадена съвсем успешно. По-лошото е, че присвояването попада в кадър, който може вече да е унищожен, както и връщаната стойност след него.

4.2.2. Поправката е в наредбата, не в заключването

Правилната наредба публикува оптимистичната стойност *преди* подаването, докато нишката още владее кадъра, и записва повторно само по пътя на неуспеха. Този път е безопасен по устройство: щом нищо не е подадено, нищо не може да подновява.

```

1 bool await_suspend(std::coroutine_handle<> h)
2 {
3     op.continuation = h;
4
5     submitted = true;        // публикувано, докато кадърът още е наш
6     if (!submit())
7     {
8         submitted = false;   // безопасно: нищо не е подадено, нищо не подно
    ↪ вява
9         return false;        // подновяване веднага, завършване няма да има
10    }
11
12    return true;              // литерал, а НЕ 'return submitted':
13                               // кадърът може вече да не съществува

```

Листинг 9: Наредба, която издържа

Наредбата се формулира в три реда, и те са цялото съдържание на поправката. Първо се записва всичко, което кадърът ще трябва да съдържа след подаването. Второ се

подава. Трето се връща стойност, чието изчисляване не докосва кадъра. Между втория и третия ред кадърът принадлежи на друга нишка, а не на текущата, макар че текущата още изпълнява инструкции в тялото на функцията.

Връщането на литерал вместо на член е частта, която прави разликата, и си струва да се каже защо. Присвояването `submitted = true` е записване в кадър, който в този момент още е наш, тоест то е безопасно. Изразът `return submitted` е *четене* от кадър, който вече може да не съществува, тоест той е достъп до освободена памет дори когато стойността, която би прочел, е правилната. Разликата не е в резултата, а в това дали изобщо има обект, от който да се чете. Затова инвариантът се формулира като забрана, а не като нареждане: след успешно подаване никой израз в тази функция не се позовава на член. Забраната е проверяема с четене, а правилната стойност не е.

Оттук следва и защо `await_suspend` тук връща `bool`, а не `void`. Вариантът с `void` няма как да изрази „не спирай“: щом функцията е започнала, спирането вече е факт и единственият изход от него е подновяване, тоест точно действието, което е опасно да се извърши отвътре. Вариантът, който връща дескриптор на корутина, има същия недостатък, защото дескрипторът, който би върнал, се чете от кадъра. Логическият тип е единственият от трите, чиято стойност може да бъде литерал.

4.2.3. Трите места

Грешката беше открита в `io_uring` чрез проявление: изпълнение на тестовите отказваше приблизително веднъж на десет пъти със съобщение за неуспешно подаване на операция, която в действителност беше подадена. Диагностиката премина през две други хипотези, всяка от които беше поправена и не се оказа причината: липсващо заключване на опашката за подаване, която е с един производител, и връщане от цикъла за изчакване без изчистване на опашката за завършвания. И двете поправки са правилни сами по себе си и са отбелязани като такива, но не обясняваха наблюдението.

Същата форма присъства в реализациите върху `epoll` и `kqueue`, където регистрацията предхожда изчакването, а еднократната регистрацията означава, че пропуснатото събитие не се повтаря никога. Там грешката е поправена **по разсъждение и не е била възпроизведена**: двадесет изпълнения не я предизвикаха, включително с изкуствено разширен прозорец. Защо не се възпроизвежда, остава неустановено. Бърз път, който да пропуска регистрацията, когато данни вече чакат, няма в нито една от двете реализации — нито сега, нито преди поправката: и двете регистрират безусловно при

всяко четене, а сокет с вече чакащи данни е по `loopback` обичайният случай, а не редкият.

Това се заявява така, а не като „поправени три грешки“. Едната е наблюдавана; другите две са затворени дупки в разсъжденията.

4.2.4. Същата форма при отложените стойности

Отложената стойност от раздел 3.3 среща същия въпрос. Интерфейсът ѝ за изчакване връща *дали* обратното извикване е било записано, вместо да го извика: `true`, ако стойността още липсва и извикването ще се случи по-късно, `false`, ако стойността вече е налична, при което извикването не се прави изобщо. Изчакващият връща тази стойност направо от `await_suspend`, тоест „вече разрешено“ става „не спирай“, без никакъв достъп до кадър, който може да е изчезнал.

Има и втора функция, `on_ready`, която извиква безусловно и се използва от изобразяването, а не от корутина. Двете съществуват поотделно нарочно: изобразяващият иска да бъде извикан и тогава, когато стойността вече е пристигнала, а изчакващият иска точно обратното. Един общ интерфейс, който върши и двете, би принудил едната от страните да проверява готовността преди записването, което е самата надпревара.

Изводът, който си струва да се пренесе: при интерфейс, чиято употреба съдържа такава надпревара, поправката е интерфейсът да прави грешката невъзможна, а не документацията да я описва. Три повторения са достатъчно доказателство, че помненето не работи.

4.3. Дефекти, открити при насочено фъзване

Този раздел описва десет дефекта, открити в един работен цикъл, и после формулира тяхната обща форма. Подредбата е такава нарочно: отделните дефекти са поправени и биха били просто списък, докато формата им е твърдение за интерфейси, което важи и извън тази реализация.

4.3.1. Какво се фъзва и какво не

Правилото за включване е тясно: чисти функции, които приемат байтове от неударововерен насрецен участник. Всичко, което изисква връзка или цикъл на събития, не се фъзва добре, защото входът не е един буфер, и мястото му е в набор за съответствие с протокола, а не тук.

Изборът на фъзър и на детектори следва от същото стеснение. Фъзването с обратна връзка по покритие насочва мутациите към входове, достигащи нов код, и е ефективно точно когато входът е един буфер и функцията е чиста [45]. Детекторът за грешки в паметта е задължителен спътник, защото без него фъзърът намира само сриговете, а не и достъпите извън границите, които не се сригат [46]: неоткритият достъп извън границите не спира изпълнението и следователно не се брои за намерен вход.

По това правило са осем повърхности, изброени в (Табл. 6). Библиотеката се компилира с инструментване за покритие, а самите харнеси се свързват с libFuzzer. Заедно с ASan работи и UBSan в прихващащ вариант, ограничен до знаково препълване, изместване и деление на нула. Прихващащият вариант е избран, защото средата за изпълнение на UBSan е слаба на целевата платформа Windows, а на един фъзър прихващането му стига: libFuzzer го докладва като сриг така или иначе. Знаковото препълване е точно случаят, който има значение тук, а ASan не го вижда.

Таблица 6. Харнесите и какво намери всеки от тях

Харнес	Повърхност	Резултат
request_head	начален ред и полета на заявка по HTTP/1.1	нищо при 1 821 407 изпълнения
range	заглавно поле Range	два дефекта
chunk_size	редът с размера пред всяко парче	два дефекта
hpack	цикълът около инфлатора на nghttp2	нищо при случаен вход, един дефект при ръчно построен блок
h2_frame	заглавната част на рамка по HTTP/2	нищо, но дефект в самия харнес
urlencoded	процентно декодиране и параметри	нищо при 300 000 изпълнения
multipart	съставно тяло с граница	нищо при 300 000 изпълнения
cookie	заглавно поле Cookie	нищо при 300 000 изпълнения

Най-ценната повърхност не е в таблицата и това е ограничение, а не пропуск. Това е класификаторът на QUIC пакети от раздел 3.1: първият код, който изобщо докосва дейтаграма от когото и да било. Той изисква компилация с HTTP/3, каквато тази машина не произвежда, така че за него няма нито харнес, нито твърдение.

4.3.2. Range: приемане на непълен вход и на обърнат интервал

Харнесът за Range не проверява само липсата на срыв. Той проверява и числата, преди и след привеждане спрямо размер на файл, защото историческите провали на тази повърхност са аритметични, а не сривове.

В рамките на 200 000 изпълнения той намери клас дефекти: позицията се преобразуваше с `std::stoll`, който спира на първия символ, който не разбира, и докладва успех. Затова байтова позиция можеше да носи произволни последващи байтове. Стойността `bytes=1;-0` се разчиташе като интервала от 1 до 0, а `bytes=5abc-9` като от 5 до 9. Сега позицията е цифри и нищо друго, а препълването е отказ, а не завъртане.

Това изведе наяве втория дефект. Обърнат интервал се разчиташе безпроблемно и `bytes=100-1` произвеждаше обект, който съобщаваше дължина `-98`. Функцията `normalize()` го отказваше, тоест handler-ът за статични файлове беше в безопасност, но всеки викащ, който чете резултата от разчитането, без да го привежда, наследяваше отрицателната дължина. RFC 9110 нарича такъв интервал невалиден, и сега анализаторът също.

4.3.3. HPACK: граница върху входа вместо върху изхода

HPACK е единственото място в HTTP/2, където малък вход по замисъл произвежда голям изход [3], тоест интересното при него не е разчитането, а натрупването. Затова харнесът следи отношението между подаденото и разпределеното, а не само отсъствието на срыв. Той подава на един декодер няколко блока последователно, защото нов декодер на всеки вход би проверявал единствено случая с празна динамична таблица.

Случайното фъзване не намери нищо при 300 000 изпълнения, и това е слабо доказателство, а не чисто свидетелство: бомбата изисква таблицата първо да бъде напълнена и после посочена, до което случайна мутация не се добира. Отговорът дойде от ръчно построен блок, и той е отрицателен за реализацията.

Блок от 16 384 октета, тоест такъв, който се побира в една рамка HEADERS при подразбиращия се максимален размер, се декодира до 49 528 379 октета, ако вмъкне един запис от 4000 октета и после го посочва с по един октет на повторение. Отношението е около 3023 към 1. При 8 KB, колкото допуска пътят през CONTINUATION, изходът е 16,7 MB и отношението е 2045 към 1. Числата са измерени, а не оценени.

Слоят на връзката *ограничаваше* нещо: натрупания компресиран блок, по

`max_header_list_size`. Това обаче е друга величина. RFC 9113, раздел 6.5.2, определя размера на списъка от полета върху *декодираните* полета, име плюс стойност плюс 32 октета за всяко [9], а HPACK разширява по замисъл. Ограничаването на входа следователно не ограничава изхода. Сървърът обявяваше в собствения си SETTINGS граница от 8192 и не я спазваше.

Проверката вече е в самия цикъл на декодиране, преди копирането на всяко поле, тоест октетите, които биха превишили границата, никога не се разпределят. Освен това връзката подава на декодера точно стойността, която обявява, вместо да разчита подразбиращата се стойност да съвпадне по стечение на обстоятелствата.

4.3.4. Размер на парче: препълване и липса на таван

Двата дефекта в `parse_chunk_size` излязоха в рамките на секунди след насочването на харнес към нея.

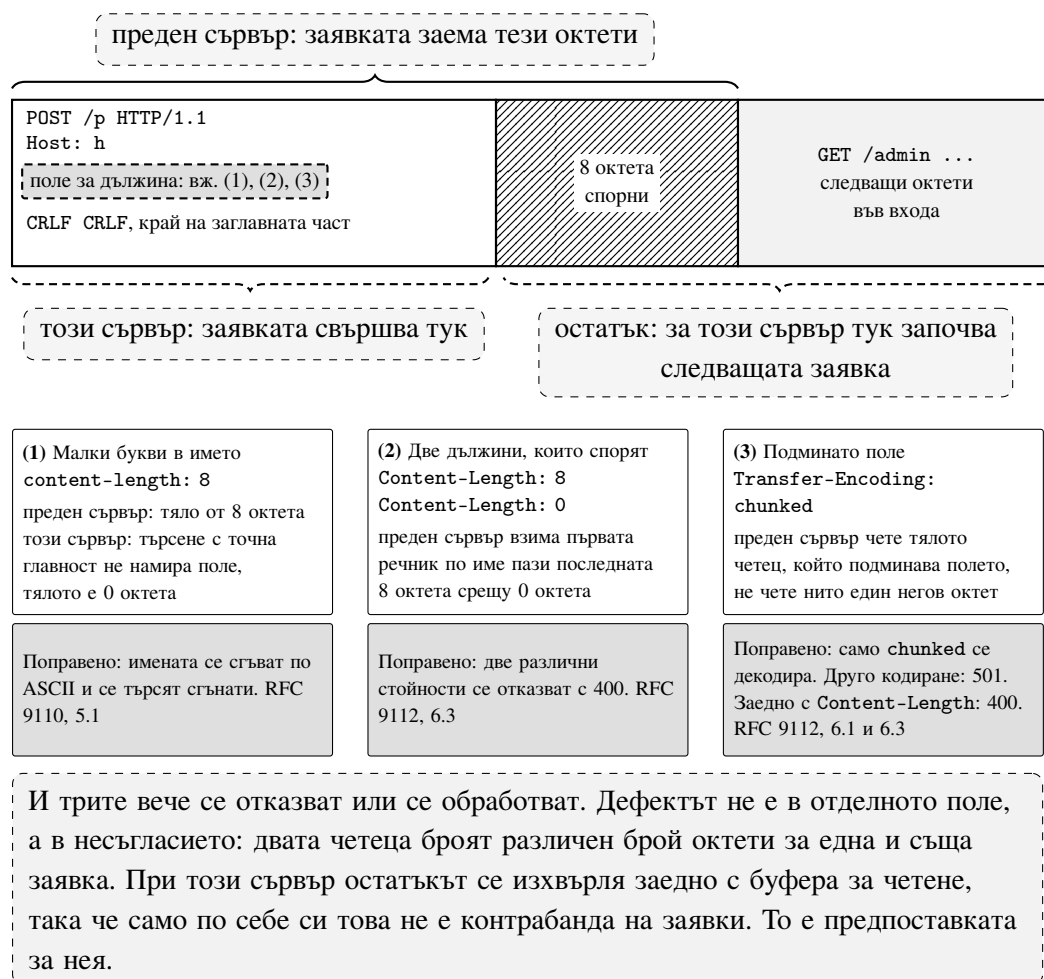
Размерът се натрупваше в `int64_t` с обикновено умножение и без граница, тоест ред с двадесет шестнадесетични цифри препълваше знаково цяло число. Това е неопределено поведение, а не просто грешен резултат, и разликата има значение: викащият проверява единствено за отрицателен резултат, а компилатор, който има право да приеме, че препълването не може да се случи, има право да премахне тази проверка. Натрупването сега е беззнаково, където завъртането е поне определено, и е ограничено, преди да се доближи до завъртане.

Таванът е вторият дефект, защото такъв нямаше. Шестцифрен шестнадесетичен размер минаваше непроменен до четене: демонстриращият вход е BB00A0, тоест 12 255 392 октета, при положение че същият сървър отказва тяло с `Content-Length` над 10 485 760. Двете рамкирания описват една и съща величина, а два пъти, които не са съгласни колко дълго може да бъде едно тяло, са точно формата, която приема контрабандата на заявки. Сега и двата използват една и съща граница.

И двата демонстриращи входа са в корпуса под описателни имена, `overflow-20-hex-digits` и `over-body-limit`, а не под съдържателен хеш. Причината е в раздел 4.3.9.

4.3.5. Рамкиране по HTTP/1.1: несъгласие, а не контрабанда

Три начина, по които този сървър можеше да не се разбере с клиент или с каквото стои пред него, за това колко октета заема една заявка. Трите са показани на (Фиг. 11).



Фигура 11. Две реализации, които не се разбират къде свършва една заявка по HTTP/1.1. Лентата е един и същ вход. Преденият сървър приписва спорните октети на тялото, а този сървър ги смята за начало на следващата заявка. Трите случая отдолу са трите начина полето за дължина да стане двусмислено, и трите водят до една и съща картина. Поправките ги отказват или ги обработват.

Имената на полета се търсеха с точна главност, с коментар на място, който казваше, че безразличието към главността би било по-добро. Имената на полета в HTTP са безразлични към главността, а HTTP/2 задължава малките букви, тоест шлюз, който превежда надолу към HTTP/1.1, изпраща `content-length` и този сървър не виждаше поле за дължина изобщо. Имената вече се сгъват при вписване и при търсене.

Два реда `Content-Length`, които не си съответстват, мълчаливо запазваха последния, защото полетата живеят в речник по име. RFC 9112, раздел 6.3, изисква съобщението да бъде отказано, а не едната стойност да бъде избрана, и сега е така.

Полето `Transfer-Encoding` не беше споменато никъде по този път, тоест заявка с парчета оставаше без прочетено тяло. Класът `ChunkedBodyReader` съществуваше и не се извикваше отникъде.

Тежестта се заявява така, както беше измерена, а не както беше предположена. Очакването беше непочетените октети да бъдат разчетени като втора заявка по същата връзка, което би било контрабанда в чист вид. Не са. Те се изхвърлят заедно с буфера за четене, тоест и трите случая произвеждат точно по един отговор преди поправката и след нея. Това, което дефектите представляват, е сървър, който не е съгласен с правилен анализатор къде свършва заявката, а това е предпоставката, от която преден посредник има нужда, за да се разсинхронизира с него, а не проникване само по себе си.

4.3.6. Процентно декодиране: приет невъзможен escape

Тези два дефекта не бяха намерени от фъзър, а при прочитане на кода, докато той се подготвяше за фъзване, и това си струва да се каже, защото е част от резултата.

Начинът на валидиране беше подаване на двойката на `strtol` и приемане, когато `strtol` е изконсумирал два символа. Функцията `strtol` обаче прескача водещи интервали и приема знак, тоест `%+1` и `% 1` се декодираха и двете до `0x01`. Декодер, който приема последователности, каквито граматиката не произвежда, разширява множеството от начини да се закодира даден октет, а по пътища октетите, които си струва да се закодират, са тези в . . . Сега и двата символа трябва да са шестнадесетични цифри.

Правилото „плюс означава интервал“ се прилагаше безусловно. То идва от RFC 1866 и се отнася за полета на формуляр, а RFC 3986 няма такова правило за URI. Път, съдържащ буквален плюс, се декодираше до интервал и се маршрутизираше като различен път от поискания. Сега това е параметър: истина за параметрите на заявката и за телата на формуляри, лъжа за пътя.

4.3.7. Общата форма на десетте дефекта

Дотук разделът е списък. Тази подточка е твърдението, заради което списъкът е тук.

Всеки един от изброените дефекти е ограничение или правило, което *вече съществуваше* някъде в системата, но не и по пътя, по който минава непознат вход. Не липса на предвидена защита, а защита, поставена на място, до което нападателят не стига.

- `normalize()` отказваше обърнат интервал, а `parse()` го раздаваше. Защитата беше в привеждането, а не в разчитането, и всеки викащ, който не привежда, минаваше покрай нея.
- `max_header_list_size` ограничаваше входа, а определението на величината е върху изхода. Числото беше правилно и обявено на насрещната страна; ограничаваната величина беше друга.
- Границата за тяло ограничаваше едното рамкиране и не ограничаваше другото. Едно и също твърдение за максимална дължина беше вярно по единия път и невярно по другия.
- `set_timeout` се съхраняваше от всеки backend и не се спазваше от нито един. Викацията имаше клон за изтекъл срок и този клон беше недостижим.
- Търсенето на поле за дължина беше правилно за имената, които сървърът очакваше, и празно за имената, които HTTP/2 задължава.

Общото между петте е, че никое от тях не изглежда като липсваща проверка при преглед на кода. Проверката е налице, тя е на един екран разстояние и е правилна там, където стои. Затова прегледът на кода систематично ги пропуска, а вход, който минава по друг път, ги намира веднага. Това е и практическият извод: за такъв клас дефекти автоматизираното подаване на вход не е допълнение към прегледа, а различен по вид инструмент, защото задава въпроса „по кой път се стига дотук“, който прегледът не задава.

От десетте дефекта четири бяха намерени пряко от фъзър, един от ръчно построен вход, който проверката за отношение в харнеса наложи като въпрос, и пет при прочитане

на код, който беше преработван, за да стане достижим от харнес. Последното число не е страничен ефект и не бива да се приписва на фъзването: то е резултат от изискването функцията да бъде чиста и достижима без сокет. Началният ред и полетата на заявката живеяха вътре в `App::parse_request`, корутина, която чете от връзка, тоест най-голямата повърхност в дървото, управлявана от нападател, не можеше да се провери без сокет. Всичко от началния ред до последното поле работи върху `string_view` и няма нужда от нищо друго; само четенето на тялото имаше. Разделянето беше предпоставка за харнеса и се оказа причина кодът да бъде прочетен.

4.3.8. Какво фъзването не намери

Отрицателният резултат се заявява като резултат, а не като мълчание.

Харнесът за началния ред и полетата беше изпълнен 1 821 407 пъти за четири минути срещу натрупания корпус, достигайки 494 ребра на покритие и 2571 характеристики, без находка. Това е твърдение с тежест точно защото шест дефекта излязоха от по-плитки изпълнения върху други повърхности: анализатор, преживял 1,8 милиона входа, е свидетелство за самия анализатор, а не само за изразходвано усилие.

Харнесите за `urlencoded`, `multipart` и `cookie` не намериха нищо при по 300 000 изпълнения. За `HPACK` същото число не е свидетелство, поради причината от неговата подточка.

Харнесът за рамките по `HTTP/2` имаше дефект в себе си, поправен преди да бъде подаден: той сравняваше `serialize(parse(x))` с `x`, докато най-старшият бит на идентификатора на потока е запазен и `RFC 9113` казва, че получателят го пренебрегва [9], тоест `serialize` нарочно не го възпроизвежда. Инвариантът е устойчивост при повторно разчитане, а не съвпадение байт по байт с входа. Това се отбелязва, защото харнес, който проверява твърде силен инвариант, произвежда находки, които не са дефекти, и това е най-скъпият начин да се изразходва внимание.

4.3.9. Как това остава поправено

Десет дефекта, намерени за един цикъл, и нищо, което ги изпълнява автоматично, означава, че следваща промяна може да върне който и да е от тях, без някой да забележи.

Задачата в интеграцията прави две неща и само първото е бариера в обичайния смисъл. Възпроизвеждането на записания корпус е детерминирано: всеки вход, който някога е бил интересен, се изпълнява по веднъж, включително трите наименувани входа,

които възпроизвеждат бомбата в НРАСК и двата дефекта в размера на парчето. Краткото изследователско изпълнение след него не е детерминирано и не се очаква да намери нищо при повечето промени; то е там, защото корпус остава полезен само ако нещо продължава да го попълва.

Проверено беше, че бариерата наистина спира, а не се предполага, че спира. Връщането на тавана върху размера на парчето и възпроизвеждането на корпуса се проваля с код 77 върху входа, който първоначално го намери; възстановяването на тавана минава отново. Регресионен тест, който никога не е бил виждан да се проваля, е надежда.

Корпусът беше сведен до входовете, които действително допринасят покритие. За началния ред и полетата това е 620 файла вместо 2404, при еднакво покритие. Корпус, който носи излишни входове, струва на всяко бъдещо изпълнение време, което то би могло да прекара някъде другаде.

4.4. Реализации на протоколите

Трите протокола са реализирани по различни стратегии, и разликите са обосновани, а не исторически.

4.4.1. HTTP/1.1: собствен анализатор, защото там живеят правилата

Синтактичният анализ е собствен. Протоколът е достатъчно прост, за да не оправдава зависимост, а анализаторът е и мястото, където се прилагат правилата срещу контрабанда на заявки, тоест собствената реализация дава пряк контрол върху приемливото.

Какво означава това конкретно, след поправките от раздел 4.3:

- Имената на полета се съгват по ASCII при вписване и при търсене, тоест `content-length` и `Content-Length` са едно и също поле.
- Две различни стойности за `Content-Length` отказват съобщението с 400, вместо едната да бъде избрана.
- Приема се точно кодирането `chunked` и нищо друго. Анализатор, който избира познатото кодиране от списък, рамкира тяло, което не е разбрал; тук непознато кодиране получава 501.

- Двете рамкирания едновременно, тоест Content-Length заедно с Transfer-Encoding, се отказват с 400.
- Размерът на парче е ограничен от същата граница, която важи и за тяло с обявена дължина в октети.

Отказът стига до клиента със състоянието, което грешката носи, а не с 400 за всичко. Първоначално `handle_connection` строеше всеки провал при разчитане с `Response::bad_request` и изхвърляше кода, тоест клиент не можеше да различи „заявката ти е неправилна“ от „този сървър няма да направи това“, а това са различни неща, по които се действа различно.

Четенето на тяло с парчета има една подробност, заради която беше отлагано два пъти. Четенето, което намира празния ред, край на заглавната част, обикновено носи със себе си и част от тялото, а тези октети вече са свалени от сокета. Четец, който умее да чете само от сокет, следователно започва по средата на първото парче. Затова четецът ги приема при конструиране. Проверено чрез разделяне на тяло от две парчета на всяка от неговите 27 позиции, с изискване за еднакъв резултат, плюс случая, в който цялото тяло е пристигнало заедно със заглавната част и сокетът изобщо не се докосва.

Ограничение, което се заявява, а не се крие. След завършващото парче четецът може да държи октети, принадлежащи на следващото изпратено от клиента, а `parse_request` владее буфера си за едно извикване и няма къде да ги остави. Затова връзката се затваря след заявка с парчета. Пренасянето им както трябва означава цикълът на връзката да държи остатък между извикванията, което е и условието, при което пътят с Content-Length би поддържал конвейеризиране на заявки. Днес той също не я поддържа.

4.4.2. HTTP/2: собствено рамкиране, чужд HPACK

Рамкирането е собствено, а от `nghttp2` [47] се използва само HPACK. Разделянето минава точно по границата между това, което се проверява добре, и това, което не се проверява добре.

Рамкирането е механично: заглавната част на рамката е девет октета с фиксирано разположение, изходът е функция на входа и няма състояние между рамките освен номера на потока. Такъв код се покрива с харнес, който проверява устойчивост при повторно разчитане, и това беше направено.

НРАСК не е такъв. Той носи динамична таблица, тоест състояние, което се пренася между блокове, и правилността на един блок зависи от всички преди него. Дефект в такъв код не е дефект, а уязвимост: погрешно състояние на таблицата означава, че полета от една заявка се появяват в друга. Затова алгоритъмът е чужд.

Разделянето обаче не прехвърля отговорността, и това е урокът от раздел 4.3. Чуждата библиотека декодира правилно; какво се прави с декодираното, е задължение на викация, и точно там беше дефектът. Границата върху размера на списъка от полета не е свойство на НРАСК, а на НТТР/2, тоест тя не може да бъде делегирана заедно с алгоритъма. Изводът, който си струва да се пренесе: делегирането на алгоритъм не делегира ограниченията, които протоколът налага върху неговия резултат.

4.4.3. НТТР/3: две машини на състоянията, които не притежават нищо

Тук се използват външни библиотеки, `ngtcp2` и `nghttp3`, и изборът им е архитектурен, а не удобство. И двете са само машини на състоянията: не притежават нито сокет, нито цикъл на събития, нито таймер. Точно това прави възможен дизайнът от глава III. Библиотека, която управлява собствен цикъл, би наложила втори, и единственият слушащ дескриптор щеше да отпадне, преди изобщо да бъде реализиран.

Какво означава „не притежават нищо“ на практика, тоест как се задвижват:

- **Вход.** Дейтаграмата, получена от сокета, който *сървърът* притежава, се подава на `ngtcp2_conn_read_pkt` заедно с пътя, тоест локалния и отдалечения адрес. Библиотеката не чете; тя ѝ се дава.
- **Изход.** Обратната посока е две извиквания в определена наредба. Извикването `nghttp3_conn_writev_stream` казва какви данни на приложно ниво чакат по кой поток, а после `ngtcp2_conn_writev_stream` превръща описаното в дейтаграма. Изпращането отново извършва сървърът, не библиотеката.
- **Време.** Библиотеката не разполага с таймер и не иска такъв. Тя обявява най-ранния си краен срок чрез `ngtcp2_conn_get_expiry`, а endpoint-ът обхожда връзките си и извиква `ngtcp2_conn_handle_expiry` за изтеклите. Тоест срокът за повторно предаване се обслужва от същия цикъл, който обслужва и четенето, без отделна нишка и без отделна опашка.

- **Ръкостискане.** TLS минава през `ngtcp2_crypto` и обратните извиквания, които той предоставя, а не през BIO на OpenSSL. Обектът SSL носи препратка към връзката като данни на приложението. Тоест ръкостискането се задвижва от QUIC, а не обратното, и няма втори слой, който да иска да чете от сокет.

Едно следствие от това, че библиотеката не е реентрантна: изпразването на изхода е защитено с флаг, а не с мутекс. Второ извикване, докато първото тече, не чака, а само отбелязва, че има още работа, и първото я поема, преди да се върне. Изчакването тук би означавало две корутини вътре в `ngtcp2` едновременно, което е точно състоянието, което флагът предотвратява.

Съответно връзката по HTTP/3 наслагва три машини на състоянията: `ngtcp2` за транспорта, SSL за ръкостискането, управлявано от `ngtcp2`, и `nghttp3` за рамкирането и QPACK. Заявката излиза оттам като обикновен обект заявка и се връща като обикновен обект отговор, тоест минава през същия маршрутизатор, същата верига от middleware и същите handler-и както останалите два протокола. Добавянето на протокол не разклонява приложението.

4.5. Типово безопасни параметри на маршрута

Параметрите на маршрута пристигат като текст и почти винаги се използват като нещо друго. Обичайният подход оставя преобразуването на handler-a: той получава асоциативен масив от низове, изважда "id", преобразува го към цяло число и се надява. Три неща могат да се объркат, и трите остават невидими до изпълнение: името може да е сгрешено, преобразуването може да не успее, а типът може да не съответства на това, което handler-ът очаква.

Регистрацията тук приема типовете като шаблонни аргументи, а изискването върху handler-a е изразено като ограничение:

```
1 template <typename... Args, typename F>
2     requires std::invocable<F, Args..., Request&>
3 void get(std::string pattern, F&& handler);
4
5 // Употреба: типовете се обявяват, handler-ът ги получава готови.
6 app.get<int, std::string>("/users/{id}/posts/{slug}",
7     [](int id, std::string slug, Request& req) -> Task<Response>
8     {
9         co_return Response::ok(std::to_string(id) + ":" + slug);
10    });
```

Листинг 10: Регистрация с обявени типове на параметрите

Ограничението `std::invocable` премества една от трите грешки от изпълнение към компилиране: `handler`, чиито параметри не съответстват на обявените типове, не се компилира. Диагностиката е на мястото на регистрацията, а не под формата на страница шаблонни съобщения от вътрешността на библиотеката.

Преобразуването е през една точка за разширение, специализирана по тип. За целочислените типове тя използва `std::from_chars`, което не разпределя памет и не зависи от локала. Общият случай минава през поток, но с изрична проверка, че целият вход е бил изконсумиран: без нея "12abc" се преобразува до 12 и остатъкът се губи мълчаливо, което е точно видът приемане, който по-късно се оказва разлика между два посредника.

Тази проверка не е предпазливост, а същият дефект, който `Range` показва в раздел 4.3: анализатор, който спира на първия неразбран символ и докладва успех, приема вход, който граматиката не описва. Разликата е само в това, че тук той беше очакван, защото беше срещнат преди това другаде.

Неуспешното преобразуване не е изключение и не е стойност по подразбиране. То е грешка, носеща състояние 400, тоест невалиден параметър се превръща в отговор към клиента, а не в нула, която `handler`-ът приема за истина.

Какво остава за изпълнение. Съответствието между *броя* на обявените типове и броя на заместителите в шаблона на маршрута не се проверява при компилиране. Маршрут с два заместителя и `handler` с един параметър се открива при първата заявка. Затварянето на тази последна пролука изисква анализ на низа на шаблона по време на компилиране и е записано като тема за отделна публикация, а не като реализирано свойство.

4.6. Реализация на отложената стойност

Раздел 3.3 описва какво представлява отложената стойност и защо запълнителят е сървърната страна на `Promise`, а не маркер за подмяна. Тук се описва как това е изпълнено, защото две от решенията не следват от описанието.

4.6.1. Събиране без деклариране

Въпросът е как сериализацията разбира кои полета още нямат стойност, без моделът да ги обявява.

Очевидният подход е моделът да ги изброи. Той е отхвърлен, защото повтаря това, което преобразуването към JSON така или иначе обхожда, и защото остарява при първото добавено поле, което някой е забравил да впише в списъка, тоест дава грешен резултат мълчаливо.

Приетият подход е събирач, активен за времето на едно изобразяване. Той е нишково локален, защото едно изобразяване принадлежи на една нишка през цялото си времетраене, а споделен събирач би смесил слотове от едновременни заявки. Активирането е с област на видимост, а не с двойка „задай и изчисти“: така изключение по средата на изобразяването не оставя увиснал събирач, което е разликата между грешка в една заявка и повреден процес.

Свързването с конкретния тип става чрез свободна функция за преобразуване към JSON, намирана по правилата за търсене по аргумент. Тоест библиотеката не изисква нищо от модела: щом полето е `Deferred<T>`, преобразуването на модела минава през тази функция, без моделът да знае за нея. Тя има три случая, и третият е решение, а не пропуск:

- **Разрешена стойност.** Записва се самата стойност. Няма какво да се отлага и страницата не плаща нищо.
- **Неразрешена, при активен събирач.** Записва се обект с един ключ, а събирачът връща номера на слота, който да бъде вписан в него. Регистрацията се случва точно когато запълнителят се записва, тоест двете не могат да се разминат.
- **Неразрешена, без активен събирач.** Записва се `null`. Щом никой не предава, запълнителят би бил обещание, което страницата никога няма да изпълни, а дупка, която остава празна завинаги, е по-лоша от честна липса.

Ключът на запълнителя е `__coroute_deferred`. Името е избрано достатъчно отличително, че истинско поле със същото име да е съзнателно действие, а не съвпадение.

4.6.2. Границата на екраниране

Стойността се вгражда в елемент `script` и това е граница на сигурност. Разделът тук казва точно кои символи се екранират и защо точно те, защото изборът не е очевиден.

Анализаторът на HTML търси затварящия етикет вътре в елемента `script`, преди JavaScript изобщо да види съдържанието. Стойност, съдържаща затварящ етикет за скрипт, следователно прекратява блока предсрочно и всичко след нея се разчита като разметка. Достатъчен е низ, предоставен от потребител.

Екранират се символите, които са невидими за `JSON.parse` и същевременно спират токенизатора на HTML: `<`, `>` и амперсандът се записват в `\u` форма. Към тях се добавят `U+2028` и `U+2029`, които прекратяват ред в JavaScript, но не и в JSON, тоест низ, който е валиден JSON, може да е синтактично неправилен JavaScript, ако бъде вграден дословно.

Изборът е такъв, а не екраниране на всичко: замяната се извършва върху всяка предадена стойност, тоест тя е по горещия път на отложената страница, а разширяването ѝ до целия набор от неалфанумерични символи би оскъпило именно случая, заради който отлагането съществува. Ограничението, което това налага, е че функцията приема вече сериализиран JSON, а не произволен текст.

4.6.3. Средата на страницата и отказът

Началният скрипт се вгражда веднъж и е нарочно малък и без зависимости: отложена страница, която първо трябва да изтегли рамка, за да покаже дупка, си е изразходвала предимството.

Мястото му е ограничено от два края. Той трябва да предхожда всеки код на страницата, който иска слот, защото поточно предаваният документ изпълнява скриптовете в реда на пристигането им. И не може просто да бъде първи: скрипт преди декларацията на типа на документа поставя браузъра в режим на съвместимост, което променя оформлението на страница, която не е искала това. Затова той влиза непосредствено след началото на `head`, където има такова, непосредствено след началото на `body` иначе, и най-отпред само при фрагмент без нито едно от двете, където няма декларация, пред която да застава.

Има и трети скрипт, за отказ, и той не е удобство. `Handler`, който е предизвикал изключение, трябва да го съобщи. Без него обещанието остава неизпълнено, докато разделът е отворен, което изглежда точно като бавна заявка и е най-лошият възможен начин да се провали: без грешка, без изтичане на срок и без начин да се разбере.

Запазена е и резервна възможност без JavaScript: елемент с атрибут `data-coroute-slot` получава попълнен текст, тоест страница, която не използва собствен скрипт, продължава да работи. Тя е *в допълнение* към типизирания вариант, а не вместо

него: без него приносът от раздел 3.3 не съществува, защото подмяната на съдържание не е Promise.

4.7. Изводи

Главата описва четири вида решения, и си струва да се различат, защото се защитават по различен начин.

Първият вид е избор между налични средства с обосновка. Собствен анализатор за HTTP/1.1, защото там живеят правилата срещу контрабанда на заявки; чужд HPACK, защото динамичната таблица със състояние превръща дефекта в уязвимост; чужди машини на състоянията за QUIC, защото те не притежават нито сокет, нито цикъл, нито таймер, а библиотека със собствен цикъл би отменила архитектурата от глава III, преди тя да бъде реализирана.

Вторият вид е преместване на грешка от изпълнение към компилиране. Обявените типове на параметрите правят несъответстващия handler некомпilierуем, и разделът казва изрично коя пролука остава отворена.

Третият вид е поправка на грешка, чиято форма се повтаря. Разделът 4.2 описва надпревара, допусната три пъти на три места, от които една беше наблюдавана, а две бяха затворени по разсъждение и не бяха възпроизведени. Изводът е за интерфейса, а не за конкретния код: когато обичайната употреба на интерфейса съдържа такава надпревара, поправката е интерфейсът да прави грешката невъзможна. Три повторения са достатъчно доказателство, че разчитането на внимание не работи.

Четвъртият вид е дефект, който прегледът на кода не намира по устройство. Разделът 4.3 описва десет такива в един работен цикъл, и общото между тях е формулирано в раздел 4.3.7: ограничението съществува, то е правилно там, където стои, и не е по пътя, по който минава непознат вход. Четири от десетте бяха намерени от фъзър, един от ръчно построен вход, а пет при прочитане на код, преработван, за да стане достижим от фъзър. Последното число не се приписва на инструмента.

Три ограничения се заявяват заедно с резултатите, а не отделно. Класификаторът на QUIC пакети, който е най-ценната повърхност за фъзване, няма харнес, защото тази машина не произвежда компилация с HTTP/3. Връзката се затваря след заявка с парчета, защото няма къде да се пренесат октетите след завършващото парче. И конвейеризирането на заявки не се поддържа по нито едно от двете рамкирания.

Нито едно от твърденията в тази глава не е за производителност. Тя описва какво е реализирано и защо; дали реализацията струва това, което се твърди, се решава в глава VI.

V. МЕТОДИКА НА ИЗСЛЕДВАНЕТО

Тази глава определя как се измерва, и я предхожда една констатация: измерването на уеб сървъри е област, в която грешките рядко се проявяват като грешки. Неправилно проведеното измерване не отказва да работи. То произвежда числа, които изглеждат правдоподобни, подреждат се в графика и водят до заключение, изведено от нещо друго, а не от това, което авторът смята, че е измерил.

Затова главата е подредена около начините, по които измерването може да бъде тихо погрешно, и около механизмите, които правят всеки от тях невъзможен, а не невероятен.

5.1. Какво трябва да произведе методиката

Твърденията от глава III определят изискванията, а не обратното. Те са три и всяко налага различно ограничение.

Първо, цената на демултиплексирането се твърди, че е неоткриваема. Твърдение за липса на разлика е по-трудно за защита от твърдение за наличие: то изисква разделителна способност, достатъчна да покаже разликата, ако тя съществуваше.

Второ, делът на пакетите, изискващи препращане, се твърди, че е малък. Това изисква условия, при които миграцията изобщо се случва, тоест повече от един адрес на клиента.

Трето, отложеното изобразяване се твърди, че намалява времето до първия байт. Това изисква разделяне на времето до първия байт от времето до пълното изобразяване, тъй като второто не се подобрява.

5.2. Сравнени подходи

Разглеждат се три подхода, преди да се приеме единият.

Измерване през `loopback`. Най-простият: клиент и сървър на една машина, свързани през `lo`. Отхвърля се, и не като ограничение, което да се спомене, а като дисквалифициращо за централното твърдение.

Причината е, че `loopback` не е неутрален спрямо протокола. Интерфейсът `lo` има MTU 65536, тоест протоколите над TCP получават сегменти от 64 KB без сегментиране,

без контролни суми и без прекъсвания, докато QUIC по замисъл продължава да изпраща дейтаграми от 1200 до 1450 байта и плаща пълна криптография в потребителско пространство на всяка от тях. Сравнение на HTTP/1.1 и HTTP/2 срещу HTTP/3 през loopback наказва HTTP/3 по причини, които нямат нищо общо с която и да е от двете реализации.

Освен това времето за обръщане през loopback е от порядъка на десетки микросекунди, тоест всяко свойство на мултиплексирането, свързано със скриване на закъснение, става невидимо. И понеже адресът е един, миграцията на връзка е непроверима по определение.

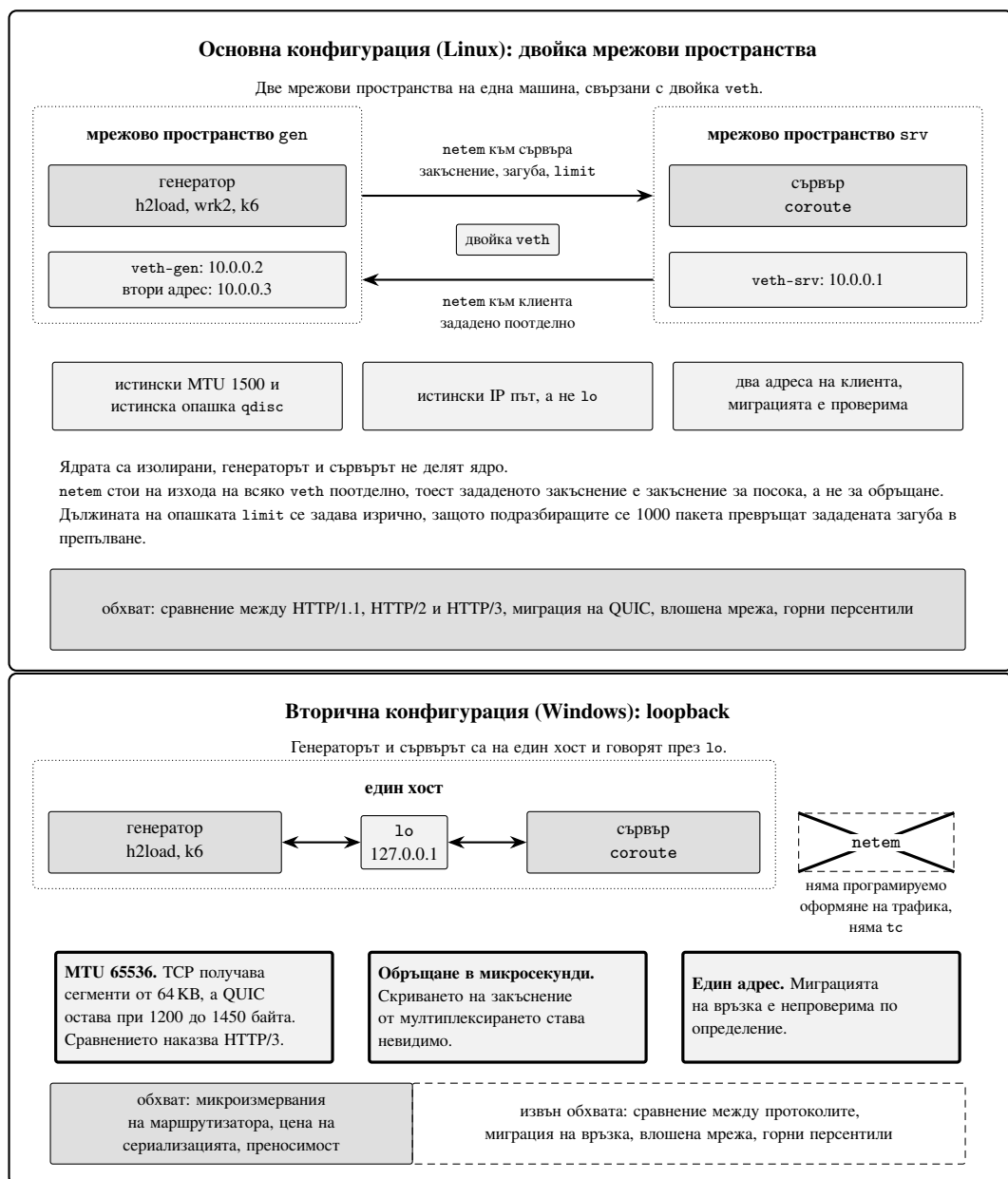
Loopback остава полезен за микроизмервания на маршрутизатора, за цената на сериализацията и за изрично обозначен „под на протоколните разходи“. За нищо друго.

Две физически машини. Най-точният, и той е и целта. Изисква втора машина и връзка, чиято пропускателна способност не се превръща сама в измерваната величина: при 1 Gbit/s таванът е около 118 MB/s, което прави всяко измерване с голям отговор измерване на кабела.

Двойка мрежови пространства. Приетият подход. Две мрежови пространства на една машина, свързани с veth, с netem по всяка посока поотделно.

Той дава истински MTU, истинска опашка, истински IP път и два адреса на клиента, което е единственото, което прави миграцията на QUIC изобщо проверяема на една машина. Приема се като основен, а измерванията на две машини се използват за проверка на валидността върху малък брой клетки.

Уточнение за netem. Прилагането на netem върху основната опашка на lo е грешка, която си струва да се назове, защото не се проявява като грешка. През loopback и заявката, и отговорът са изходящ трафик по един и същи интерфейс, тоест зададено закъснение от 50 ms доставя около 100 ms за обръщане, а загуба от 1% става около 2% на обръщане. Освен това netem по подразбиране има ограничение от 1000 пакета; при скоростите на loopback опашката прелива и „1% загуба“ на практика е неизмерено изчерпване на буфер.



Фигура 12. Двете конфигурации за измерване. Горе: основната конфигурация под Linux, две мрежови пространства на една машина, свързани с двойка veth, с netem на изхода на всяка посока поотделно и с изрично зададена дължина на опашката. Тя дава истински MTU, истинска опашка, истински IP път и два адреса на клиента, тоест миграцията на QUIC е проверяема. Долу: вторичната конфигурация под Windows, генератор и сървър на един хост през lo, без програмируемо оформяне на трафика. Трите ограничения са отбелязани изрично. Долните ленти имат еднаква обща ширина и сравняват обхвата на твърденията, които всяка конфигурация допуска.

5.3. Инструменти за натоварване

За сравнения между протоколи се използва **един** генератор: h2load, сглобен срещу ngtcp2 и nhttp3.

Мотивът не е предпочитание, а елиминиране на променлива. h2load е единственият зрял генератор, който говори HTTP/1.1, HTTP/2 и HTTP/3 с един и същ изходен формат и един и същ модел на латентността. Сравнение между протоколи, направено с три различни клиента, е объркано с три различни клиентски реализации, и няма как след това да се раздели кое на какво се дължи. h2load освен това предоставя брой отговори по код на състоянието, което затваря най-опасния пропуск, описан по-долу.

Две цени се обявяват вместо да се премълчат. Пътят за HTTP/1.1 на h2load е по-слабо оптимизиран от този на wrk, тоест абсолютната пропускателна способност за HTTP/1.1 ще изглежда по-ниска, отколкото при wrk. Това е приемливо, защото всяко сравнение се прави при един клиент, но трябва да бъде казано, и една контролна клетка с wrk определя разликата между клиентите, вместо тя да се предполага. Второ, ключът -t на h2load задава темп на пристигане на *връзки*, а не на заявки, тоест не замества wrk2.

Разпределението на задачите между три инструмента е следното: h2load за наситена пропускателна способност и за разпределения на времето за обслужване по трите протокола; wrk2 за HTTP/1.1 при постоянен темп на заявките в отворен цикъл; k6 за криви на латентността спрямо предложеното натоварване и за натоварване на WebSocket.

5.4. Генератор за кампанията на три платформи

Предходният раздел описва клиента за сравненията между протоколи, но той важи само за Linux. Кампанията обхваща Windows, Linux и macOS, а h2load не се сглобява под Windows без усилие, което само по себе си става променлива. Различен генератор на всяка платформа пък превръща всяко твърдение между платформи в твърдение между клиенти: ако измерването под Windows излезе по-бавно, разделянето на двете причини след факта е невъзможно. Изискването е същото, което налага един генератор за трите протокола, приложено към трите платформи.

Затова е написан отделен генератор, loadgen. Един генератор, един модел на латентността, три платформи. Това е по-слабият инструмент и по-силният метод.

Цената се обявява. loadgen говори само HTTP/1.1, тоест сравненията между

протоколи продължават да изискват h2load [47] и остават в кампанията под Linux. В обхвата му остава точно това, от което се нуждаят двете рамена на демултиплексирането, обхождането по backlog и обхождането по брой работни нишки.

Загряването се филтрира по издаване, не по завършване. Разликата е методическа, а не техническа. Първата версия филтрираше по завършване, тоест заявка, издадена по време на загреването и приключила малко след него, внасяше цялото си време на чакане в измервания прозорец. Едно изпълнение показва извадки от 2,1 секунди в горния край на разпределение, чийто р99 беше 785 микросекунди. Извадките се пазят поединично, а не в кофи, по същата причина, по която не се подрязват: границите на кофите се избират, преди да се знае къде е горният край.

Затварянето на връзка не е грешка. Сървър, който затваря keep-alive връзка заради собственото си ограничение за брой заявки на връзка, прави това, за което е настроен. Първото изпълнение отчете 9502 „грешки“, които бяха именно това. Затова POLLHUP сам по себе си се брои като затваряне от отсрещната страна, а само POLLERR и POLLNVAL са повреди. Критерият за дял отговори извън 2xx от (Табл. 7) зависи от това разграничение: без него всяко изпълнение би се отхвърляло по несъществуваща причина.

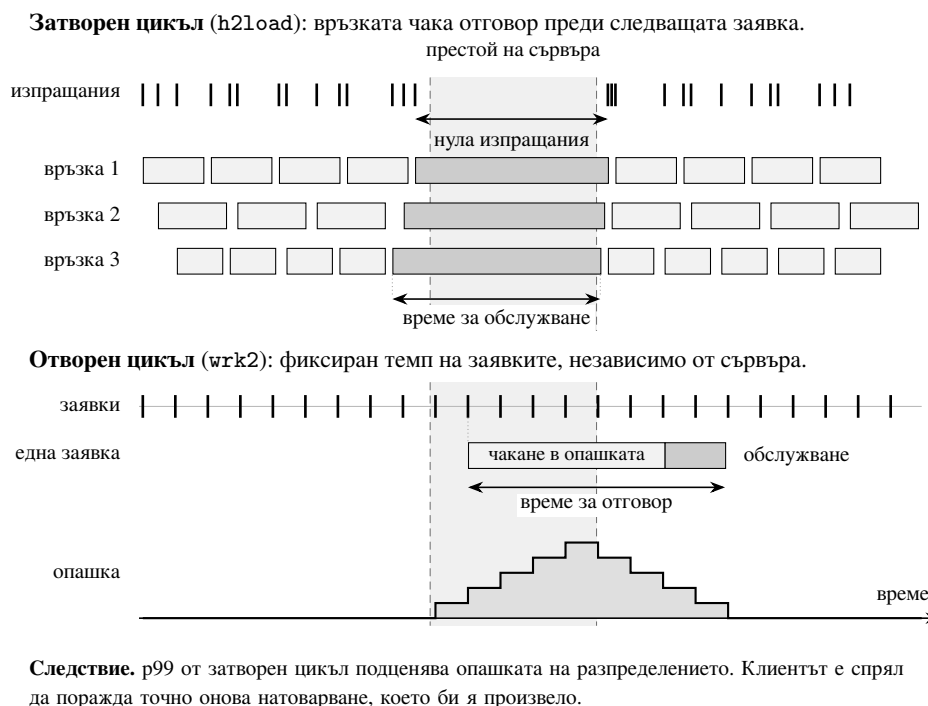
5.5. Съгласувано пропускане

Затвореният цикъл измерва време за обслужване, а не време за отговор, и разликата не е терминологична.

При затворен цикъл с C връзки всяка връзка изпраща следващата заявка едва след като е получила предходния отговор. Когато сървърът блокира, клиентът спира да изпраща. Заявките, които *биха* били забавени, никога не се изпращат и никога не се измерват. Полученият 99-и персентил е персентил на времето за обслужване на заявките, които са били допуснати, а не на времето за отговор, каквото го вижда потребител. Двата цикъла са съпоставени на (Фиг. 13).

Разликата между двата модела е изследвана и извън измерването на сървъри. Schroeder, Wierman и Harchol-Balter показват, че затворената и отворената система се държат съществено различно при едно и също средно натоварване, включително че подредбата на две системи по латентност може да се обърне при смяна на модела [48].

Оттам следва, че изборът на цикъл не е подробност от настройката на генератора, а част от това какво изобщо се измерва.



Фигура 13. Съгласувано пропускане, показано като механизъм, а не като твърдение. Горес всяка от C връзки изпраща следваща заявка едва след получен отговор, затова за целия престой на сървъра няма нито едно изпращане. Измерва се време за обслужване на допуснатите заявки. Долу заявките се предлагат с постоянен темп, престоят не ги спира, опашката расте и се източва след него. Измерва се време за отговор, тоест чакане плюс обслужване. Затова 99-ият персентил от затворен цикъл подценява горния край на разпределението: липсват именно заявките, които престоят би забавил най-много.

Протоколът е следният. Първо изкачване в затворен цикъл, за да се намери T_{max} на всяка система. След това T_{ref} се приема за минимума от T_{max} на сравняваните системи за дадената клетка. Накрая се изпълняват измервания в отворен цикъл при фиксирани нива на предложено натоварване от 25, 50, 75, 90 и 95 процента от T_{ref} , тоест всяка система получава едно и също предложено натоварване.

Числата от затворен цикъл се съобщават изрично като време за обслужване, а тези от отворен цикъл като време за отговор. Смесването на двете в една таблица е начинът, по който разликата изчезва.

Как това изглежда в генератора. В loadgen двата цикъла са два различни начина на измерване, а не два режима на съобщаване. При затворен цикъл се брои от момента, в който връзката е освободена, тоест измерва се време за обслужване. При отворен цикъл

латентността се брои от *предвидения* момент на изпращане, а не от момента, в който сокетът е приел запис. Това е цялата корекция за съгласувано пропускане, сведена до един избор кой момент да се запише. Механизмът, който я прави възможна, е че предвиденият момент се придвижва с периода независимо от това дали има свободна връзка: заявките, които забавянето би скрило, не изчезват от плана, а получават закъснението, което ги е догонило.

Заявен пропуск. За HTTP/3 няма зрял генератор с постоянен темп на заявките в отворен цикъл. wrk2 говори само HTTP/1.1, k6 и vegeta не говорят HTTP/3, а управлението на темпа при h2load е по връзки. Следователно латентността в горните персентили за HTTP/3 се съобщава само от затворен цикъл, изрично обозначена като време за обслужване, а ограничението се заявява, вместо да се заобикаля.

5.6. Квантуване на таймера и корекция на темпа

Тази грешка беше допусната, а не избегната, и стои тук, защото намереното е методическо: генератор с отворен цикъл може да измерва собствения си часовник по начин, който изглежда като резултат.

Ранна версия на loadgen задаваше на poll фиксиран изчаквателен срок от една милисекунда. Измереното беше p50 от 7,2 ms при предложени 10 хиляди заявки в секунда, което пада до 88 микросекунди при 75 хиляди. Двете стойности са от калибрирането на самия генератор, а не от кампанията, и са записани в историята на хранилището заедно с поправката, която ги отстрани. Латентност, която намалява, докато натоварването расте, е обратна на всичко, което теорията на масовото обслужване предвижда.

Причината е в платформата. Под Windows подразбиращата се разделителна способност на системния таймер е 15,6 ms, тоест poll, помолен да изчака една милисекунда, чака цял тик. Генераторът се събуждаше около шестдесет и четири пъти в секунда, издаваше наведнъж всички слотове, станали дължими междуременно, и всяка заявка носеше разстоянието от собствения си дължим момент до този тик. Разпределението излизаше равномерно между нула и 15,6 ms: p50 около 7,2 ms и p99 около 14,5 ms при предложени 10 хиляди. Падането следва от същия механизъм: при 75 хиляди отговорите пристигат непрекъснато, poll се връща веднага заради готов сокет и почти никога не изчаква тик, тоест артефактът изчезва точно там, където натоварването

е най-голямо. Кривата беше на това колко често цикълът успява да не заспи.

Правилото, което остава: латентност, която пада с растежа на натоварването, не е находка, а инструмент, и се проверява, преди да се обяснява. Изчаквателният срок на poll е решение за измерване, а не подробност от реализацията.

Поправката е двойна. Изчакването се ограничава от следващия дължим слот, вместо да бъде константа, и последната отсечка преди него се изминава със завъртане, а не със заспиване, защото никое заспиване на нито една от трите платформи не е точно до десетките микросекунди, които отвореният цикъл при тези темпове изисква. Под Windows процесът освен това заявява таймер от една милисекунда чрез `timeBeginPeriod` и го освобождава при изход. Цената се плаща в правилата за допустимост по-долу: генератор, който поддържа темпа със завъртане, вече не може да бъде съден по процесорно време.

5.7. Едно рамо и друго рамо от един и същ двоичен файл

Всяко сравнение между два режима на настоящата система се управлява с флаг при изпълнение, а не с опция при компилиране. Това е ограничение върху реализацията, наложено от методиката, и то е причината флаговете от глава III да съществуват.

Мотивът е количествен. Разликите между отделни сборки, дължащи се на подредбата на кода, на реда на свързване и на размера на средата, са документирани в порядъка от пет до десет процента [49], докато разликите между изпълнения на една и съща контролирана машина са порядъчно по-малки. Следователно разлика от три процента между два поотделно компилирани двоични файла е неразличима от това обектните файлове да са били свързани в друг ред.

Едно и също изпълнимо, два флага. Проверява се, че двете рамена наистина произхождат от един файл, чрез сравнение на контролната му сума.

Съществува и обратният подход: вместо да се избягва зависимостта от подредбата, тя се разбърква нарочно при всяко изпълнение, така че влиянието ѝ да влезе в разсейването, вместо да остане в системната грешка [50]. Той е по-общ и е приложим там, където двете рамена по необходимост са два двоични файла, например при сравнение с чужд сървър. Тук е избран по-простият, защото двете рамена на X1 могат да бъдат едно изпълнимо, и разбъркването не купува нищо, когато различието е премахнато, а не осреднено.

Еднакви опции на сокетите за всяко рамо. Правилото за един двоичен файл се отнася и за онова, което всеки заден край прави с приетия сокет. Проверка на четирите задни края показва, че те го конфигурираха различно: `io_uring` задаваше `TCP_NODELAY`, `TCP_QUICKACK` и изходен буфер от 256 KiB, `epoll` само `TCP_NODELAY`, а `kqueue` и `IOCP` нищо, тоест `macOS` и `Windows` работеха с включен алгоритъм на Нейгъл, а `Linux` с изключен. Всяко сравнение между рамена по механизъм за вход-изход дотогава е сравнявало отчасти и опции на сокета. Политиката е една функция, извиквана от конструктора на всяка връзка, така че никой път за приемане не може да я пропусне, и е проверена под `strace`, а не изведена от кода. Собствената ѝ цена, измерена върху `epoll` при ниско натоварване, е нула в рамките на шума, и това се съобщава като такова: объркващият фактор беше реален, размерът му в тези клетки не е. Това не освобождава от повторение на междураменните проекти, защото пропускателната способност, установяването на връзки и таблиците със системни извиквания не са проверени по този начин.

5.8. Средата като контролирана променлива

Кампанията продължава седмици. През това време ядрото се обновява, управлението на честотата се връща на пестелив режим след приспиване, зависимост се преизгражда. Всяко от тези събития прави изпълненията преди и след него несравними, и нито едно от тях не се обявява. Числата остават правдоподобни, което е и проблемът.

Затова средата се записва и от нея се извлича отпечатък, а програмата, която управлява кампанията, отказва да добави изпълнения към кампания, чийто отпечатък се е променил. Продължаването въпреки това изисква изричен ключ, тоест решение, което някой е взел, а не случайност.

Интересната преценка не е как се хешира, а какво се хешира. Записва се всичко; в отпечатъка влиза само това, чиято промяна прави сравнението невалидно: ядро, модел на процесора, брой физически и логически ядра, режим на управление на честотата, настройки за големи страници, компилатор, версия на кода и версии на зависимостите. Средното натоварване и времето от стартирането се записват и *не* влизат, защото хеширане на всичко би направило всяко изпълнение отделна кампания, а предпазна мярка, която се задейства постоянно, е предпазна мярка, която някой изключва.

Отказът назовава полето, което се е променило. Съобщение „средата се промени“ се пренебрегва от раздразнение; съобщение „режимът на честотата премина от `performance`

към powersave“ води до действие.

5.9. Конфигурацията под Windows и какво тя не позволява

Кой компилатор стои зад „Windows“. Двоичните файлове под Windows са построени с MinGW-w64 g++, а не с MSVC. Разликата не е формална: двата генерират различен код, свързват различни стандартни библиотеки и получават OpenSSL от различни пакети. Читател, срещнал „Windows“ без уточнение, ще предположи MSVC, а всяко измерване тук е снето с другия компилатор. Записът го казва вместо прозата: полето `toolchain.compiler` във файла на средата съдържа точния низ на компилатора, а той влиза и в отпечатъка, тоест кампания, продължена с друг компилатор, се отказва вместо да се слее. Твърденията оттук са за тази реализация под този компилатор и не се пренасят върху MSVC без измерване.

Кампанията под Windows върви на един хост. Той има шест физически ядра и дванадесет логически, и трябва да побере и сървър, и генератора. Затова двата се закачат за несъвпадащи множества ядра: сървърът за логически 0 до 7, тоест четири физически ядра, генераторът за логически 8 до 11, тоест две. Множествата не се пресичат, за да не си отнемат работа взаимно.

Маската се прилага от процеса към самия себе си при стартиране, преди да е създадена която и да е нишка, иначе съществува прозорец, в който нишките се изпълняват другаде и после мигрират. Под macOS няма преносим интерфейс за закачане; изпълнението записва маската, която е поискана, и дали е приложена, така че читателят да различи двата случая, вместо да предположи единия.

Следствието се заявява, а не се премълчава: нито сървърът, нито генераторът разполагат с машината. Това изключва абсолютната пропускателна способност като твърдение, тоест числата от този хост не се сравняват с публикувани числа от други машини. Конфигурацията поддържа сравнения: двете рамена виждат еднакви условия и единственото, което се различава, е изследваният фактор. Предложените темпове са 10, 25, 40, 55 и 70 хиляди заявки в секунда, а горната граница не е избрана, а измерена: над около 75 хиляди с две генераторни нишки на този хост закъснението на графика при 99-ия персентил излиза от десетките микросекунди.

Хостът върви през `loopback` и наследява ограниченията, заради които той беше отхвърлен в началото на главата. Затова твърденията оттук са ограничени до HTTP/1.1

и до сравнения в рамките на един протокол: сравнение с QUIC не се прави, ефектите от скриване на закъснение остават невидими, а миграцията на връзка е непроверима, защото адресът е един. И трите въпроса принадлежат на кампанията под Linux с двойката мрежови пространства.

5.10. Правила за допустимост, обявени предварително

Всеки критерий за отхвърляне е определен преди събирането на данни, и този ред е същината. Критерии, избрани след като резултатите са видени, са неразличими от избиране на резултати: винаги съществува защитимо на пръв поглед основание да отпадне точно изпълнението, което разваля тенденцията.

Критериите са малко на брой и всеки от тях назовава конкретна механична повреда, а не мнение дали числото изглежда правилно.

Таблица 7. Предварително обявени критерии за отхвърляне на изпълнение

Условие	Праг	Какво означава
Дял отговори извън 2xx	над 0,1%	Сървърът е отказвал, а не обслужвал.
Натоварване на генератора, затворен цикъл	над 85%	Измерва се клиентът, не сървърът.
Закъснение на графика при r99, отворен цикъл	над 1 ms	Генераторът не е спазил темпа, който записва.
Постигнат дял от предложения темп	под 99%	Предложеното натоварване не е било предложено.
Отклонение на честотата	над 2%	Дроселиране по температура.
UdpRcvbufErrors,	всяко	Ядрото е изпуснало работа,
TcpExtListenOverflows	нарастване	преди сървърът да я види.
Виртуализация	открита	Планирането и таймерите не са на машината.
Работно дърво с некоммитнати промени	открито	Записаната версия не описва двоичния файл.
Грешки на сокет	всяка	Отговорът липсва, а не закъснява.
Поискано закачане, което не е приложено	поискано и отказано	Изпълнението няма изолацията, която постановката описва.
Захранване от батерия	открито	Тактовете и разпределението по ядра не са тези, които кампанията описва.
Ограничение на честотата под 100%	в началото или в края	Хостът е бил дроселиран.

Първият критерий е най-важният. Без него сървър, който се срина под натоварване, печели по пропускателна способност, защото отказът е по-евтин от обслужването.

Последните четири критерия и защо стоят тук предварително. Те са добавени, преди да бъде проведена която и да е кампания извън Windows, а не след като нейните числа са били видени. Редът има значение: критерий, избран след като резултатите са известни, не се различава от избиране на резултатите.

Грешките на сокет се броят отделно от отговорите извън 2xx, защото връзка, която се е скъсала, не е произвела код на състоянието, който да бъде класифициран. Без отделно правило изпълнение, в което една трета от връзките са умрели, съобщава чист дял грешки върху оцелелите.

Останалите три произтичат от свойства на хостове, каквито настолната машина няма. Закачането за ядра не съществува като програмен интерфейс под macOS, тоест маска може да бъде поискана и тихо да не бъде приложена; правилото отказва такова изпълнение, защото то не притежава изолацията, която постановката описва. Затова кампанията под macOS не иска маска изобщо, и записът казва, че изолация не е била поискана, вместо че е била поискана и отказана. Двата случая не са едно и също, и разликата между тях е точно това, което този раздел обещава на читателя да различи.

Батерията и температурното дроселиране са за преносима машина това, което виртуализацията е за сървър: условие, при което числото описва нещо друго, а не кода. При процесор с разнородни ядра разреждането измества разпределението към икономичните ядра и понижава тактовете, а нито едното, нито другото се вижда в който и да е от предходните критерии. Проверката за отклонение на честотата чете път в `sysfs`, който под macOS не съществува, тоест без тези две правила преносима машина, дроселирала по средата на тричасова кампания, минава през всички останали.

Кой критерий може да се задейства на коя платформа. Таблица 7 изброява критериите. Тя не казва дали всеки от тях има с какво да бъде проверен на всяка платформа, а отговорът не е еднакъв и никога не е бил. Подготовката на първата кампания извън Windows, на 1 септември 2026 г., установи, че пет от тези критерии съществуваха в кода, бяха именувани като критерии и не отхвърляха нищо на поне една платформа. Проверката за виртуализация извикваше единствено `systemd-detect-virt`, който съществува само под Linux, а липсващият изпълним файл се четеше като „не е открита“; тоест всеки запис от Windows и macOS в проекта беше отбелязан като снет на физическа машина от проверка, която никога не е поглеждала. Проверката за хранване четеше `pmset`, който съществува само под macOS. Проверката за отклонение на честотата четеше път в `sysfs`, който съществува само под Linux. Равнището на диспечирание в маршрутизиращия експеримент не внасяше модула за допустимост изобщо, и полето `accepted` в него означаваше единствено, че процесът е завършил с код нула.

Всичките пет са поправени в същия ден. Таблица 8 казва какво чете всеки критерий на всяка платформа след поправката. Под Windows виртуализацията се разпознава по идентификаторите в SMBIOS, а не по `HypervisorPresent`: последният е истина и на физическа машина, щом WSL2 е включен, и би отхвърлил точно хоста, произвел повечето данни в тази работа.

Таблица 8. Какво чете всеки критерий за средата на всяка платформа, след поправките от 1 септември 2026 г.

Критерий	Windows	Linux	macOS Intel	macOS Apple Silicon
Виртуализация	SMBIOS	systemd-detect-virt, иначе SMBIOS	kern.hv_ vmm_present	kern.hv_ vmm_present
Захранване	Win32_Battery	/sys/class/power_supply	pmset -g ps	pmset -g ps
Отклонение на честотата	брояч на производителността	/proc/cpuinfo	няма	няма
Температурно дроселиране	няма	няма	CPU_Speed_Limit	няма: стойност не се публикува
Некоммитнати промени	да	да	да	да
Грешки, закъснение на графика, постигнат дял	да	да	да	да

Три следствия принадлежат тук, а не в бележка под линия.

Първо, твърдението, че постановката гарантира физическа машина, има дата. То е вярно за записи от ревизия d9b376bb5 нататък и не е било вярно преди това за платформите, които са произвели данните. Тъй като всяка кампания в тази работа започва от нула измервания след тази дата, това не струва нищо; изречение, пренесено от по-стар текст, обаче би било невярно.

Второ, неизвестното не е чисто. Сонда, която не може да бъде изпълнена, вече връща стойност, започваща с unchecked, и тя се отхвърля от същото правило, което отхвърля именуван хипервайзор. Машина, която не може да отговори на въпроса, не е с това физическа машина.

Трето, нито една платформа няма всички критерии. Windows и Linux не могат да открият температурно дроселиране. Apple Silicon не публикува нито ограничение на честотата, нито такт на отделно ядро, тоест там липсват и двата критерия за такта; Intel Мас има температурния. Това е реално ограничение, и заявяването му тук е по-евтино от откриването му от читател. Че четири отказа, пропускащи всичко, са открити в един ден в код, именуван така, както се именува предпазна мярка, е констатация за постановката, а не поредица от несвързани дефекти. Таблица, произведена преди тези поправки, е

произведена от постановка, която описваше себе си неточно, и никаква грижа в анализа не би го показала.

Къде се взема първото отчитане на честотата. Правилото за отклонение пита дали тактовата честота се е задържала *докато* изпълнението е текло, защото машина, дроселирала по средата, произвежда забавяне, което принадлежи на стаята, а не на кода. Затова и двете отчитания се вземат вътре в измервания прозорец: първото след загреването, а не преди съвърхът изобщо да е стартирал.

Разликата не е формална. Взето преди натоварването, отчитането сравнява машина в покой с машина под натоварване, което е покачване на честотата, а не дроселиране, и точно за това покачване съществува загреването. На хост, чийто режим на честотата я мащабира, това отхвърляше всяко изпълнение с отклонения между три и десет процента, докато самите изпълнения постигаха предложения темп без нито една грешка; по-дългото загреване влошаваше резултата, защото установяването, което то купува, се отчиташе срещу правилото вместо да бъде изключено от него.

Прагът и формулировката на правилото са непроменени. Преместено е само мястото на първото отчитане, така че сравняваната величина да бъде тази, която правилото назовава. Разликата между двете е записана тук, а не оставена в кода, защото промяна в предварително обявен критерий е промяна в методиката дори когато прагът остава същият.

Кой часовник обслужва измерването. В отпечатъка влиза и източникът на време на ядрото, защото той определя какво струва едно отчитане на часовника. При TSC vDSO отговаря на `clock_gettime` в потребителско пространство за около двадесет наносекунди. При HPET това е невъзможно, тоест всяко извикване е системно повикване плюс четене през паметта: измерено на един от хостовете 1931 ns срещу 5 ns за грубия часовник, стократно, и по едно системно повикване на извикване там, където TSC хост не прави нито едно.

Следствието не е малко. При около 1,7 отчитания на заявка това е близо 3,3 микросекунди на заявка допълнителна цена, която принадлежи на фърмуера на машината, а не на кода, и която влиза както в измерените закъснения, така и в броя системни повиквания. Ядрото дисквалифицира TSC, на който не се доверява, а при някои процесори на AMD прави това погрешно; поправката е параметър при зареждане, тоест хост, който

съобщава href, е хост за поправяне, а не число за коригиране.

Затова стойността се записва и участва в отпечатъка: две машини, различаващи се само по нея, не са сравними, а нищо друго в записа не би го казало, и двете кампании биха се слели в една.

Кое ядро се чете. Отчитането е на най-бързото ядро, а не средното. Средното е очевидната величина и е грешната: ядро без работа се паркира на няколкостотин мегахерца, тоест при слабо натоварена машина средното съобщава главно колко ядра са спали в мига на отчитането, и се мести с десетки процента между две отчитания, докато всяко заето ядро държи честотата си. Измерено на преносима машина с шестнадесет ядра: правилото отхвърляше изпълнения при 26% отклонение при ниско предложено натоварване и 2 до 4% при високо, което е обратната посока на температурното дроселиране и правилната за средно, доминирано от спящи ядра.

Дроселирането при тези процесори е ограничение на целия пакет, тоест машина, на която е казано да забави, забавя и най-бързото ядро, докато машина, която просто няма работа, не го прави. Затова отчитането на най-бързото ядро не изисква знание за маската на закачане, не се мести при паркиране на ядра и греши в приемливата посока: може да пропусне дроселиране, пощадило едно ядро, но не може да измисли такова, което не се е случило. Правило, отхвърлящо изправни изпълнения, е правило, което някой изключва.

Защо двата цикъла имат различен критерий за насищане. Прагът за натоварването на генератора важи само за затворения цикъл, и това е пряка последица от поправката на темпа. Отвореният цикъл поддържа темпа със завъртане, тоест е на пълно процесорно натоварване независимо от това дали успява да догони плана си. Съден по процесорно време, всяко валидно изпълнение с отворен цикъл би било отхвърлено и нито едно прието: процесорното време просто престава да бъде сигнал за насищане на клиента.

Сигналът за отворен цикъл е закъснението на графика, тоест разстоянието между момента, в който заявката е била дължима, и момента, в който тя е достигнала сокета, взето при 99-ия персентил. Генератор, изостанал с милисекунди от собствените си дължими моменти, предлага друго натоварване, а не записаното, и разпределението съдържа неговата собствена опашка, а не само тази на сървъра. Прагът е обявен предварително като всички останали, а генераторът връща ненулев код при недопустимо изпълнение, така че управляваща програма, която не чете JSON изхода, пак да не може да го запише за данни

по невнимание.

Обратното също е част от правилото: **бавното изпълнение се приема**. Бавно е резултат, и критерий, който отхвърля бавни изпълнения, отхвърля находката.

Отхвърлените изпълнения се запазват, а броят им се съобщава заедно с резултатите. Методика, която е отхвърлила една трета от изпълненията си, е различна методика от такава, която не е отхвърлила нито едно, и само съобщаването прави разликата видима.

5.11. Когато отказите не са разпределени еднакво между рамената

Отказът по обявено правило не замърсява изпълненията, които са минали: правилото е работило точно както е било обявено, и приетата извадка е това, което е останало. Но има един случай, в който изхвърлените изпълнения носят информация за самото сравнение, и той трябва да се разпознае, а не да се подмине.

Ако отказите са разпределени неравномерно между двете рамена, приетата извадка на едното рамо е обусловена по-силно от тази на другото. Тогава разликата, пресметната от оцелелите, не е оценка на търсената величина, а граница, и посоката на границата е известна: щом изхвърлените изпълнения са по-бавните, разликата подценява цената на рамото, което е загубило повече от тях. Правилото следователно е двойно. Първо, делът на отказите се съобщава *по рамо*, а не общо за клетката. Второ, ако тези дялове се различават, текстът назовава посоката: разликата е долна граница, а не измерена стойност. Разлика, намерена въпреки това, е вярна поне до този размер; липсата на разлика при такава асиметрия е слабо доказателство за отсъствие.

Мрежовото рамо на кампанията даде точно такъв случай. При 400 установявания в секунда рамото с TLS отказа три изпълнения от петдесет, а рамото в явен вид нито едно от петдесет. Трите отказа не са опашка на разпределението: най-голямата приета стойност при това натоварване е 665 μ s, а трите отказа са 2030, 2110 и 2170 μ s, тоест три точки в рамките на 140 μ s една от друга след празнина от 1365 μ s. Това е втора мода, а не край на същото разпределение. Очевидното обяснение, изместен от планировчика генератор, се проверява и не издържа: делът на процесорно време на приетите изпълнения е с медиана 0,95 в двете рамена, а трите отказа носят 0,88, 0,947 и 0,948, тоест две от трите изостават с две милисекунди при напълно обичаен дял. Модата и асиметрията са установени, механизмът не е, и точно така се съобщават.

5.12. Праг за отказ по натоварване, а не общо за проекта

Правило, което спира проект при надхвърлен общ дял откази, смесва натоварвания, за които постановката е годна, с натоварвания извън обхвата ѝ, и се задейства от аритметика, а не от доказателство. В мрежовия проект общият дял беше 11 %, но 41 от 44-те отказа бяха при едно натоварване, за което предишна стълбица вече беше показала, че е извън достижимото; при трите по-ниски натоварвания отказите бяха 0, 0 и 3 от по 100. Прагът затова се прилага *по натоварване*. Натоварване, което го надхвърля, се съобщава като измерен таван на постановката, а не като свойство на сървъра, и остава в записва заедно с това, което тавана изглежда: масови откази по темп и в един случай пълен отказ с нула доставени заявки.

5.13. Ред на изпълнение

Очевидният ред е всички изпълнения на система А, след това всички на система В. Той е и единственият ред, гарантирано погрешен: машината се загорява, и тогава всяка разлика между първия и втория час се приписва на разликата между А и В.

Затова планът се обхожда на проходи. Всеки проход посещава всяка клетка по веднъж, системите се редуват, а редът вътре в прохода е разбъркан, при това различно за всеки проход. Загоряването продължава да съществува, но се разпределя приблизително по равно, а индексът на прохода го прави видимо: ако пети проход е по-бавен от първи за всичко, това е температура и може да се види, вместо да се извежда.

Разбъркването е с фиксирано семе. Кампания, която не може да бъде повторена в същия ред, не е възпроизводима, а „случаен ред“ не е основание за неповторимост.

5.14. Статистика

Съобщават се медиани, а не средни стойности. Резултатите на ниво изпълнение не са симетрични: едно температурно неудачно изпълнение измества средната стойност и почти не помръдва медианата. Латентните извадки **не се подрязват** при никакви обстоятелства, тъй като горните персентили са самото явление; следователно обобщаващата статистика трябва да е устойчивата.

Доверителните интервали се получават чрез бутстрап с корекция за отклонение и ускорение [2]. Процентилният бутстрап е по-кратък за реализиране и е грешен по начин,

който тук има значение: той предполага, че бутстрап разпределението е без отклонение и с постоянна дисперсия, а за медиана от пет или десет изпълнения то не е нито едното.

Разлика между системи се съобщава като такава само когато изпълни **две** условия: доверителният интервал на разликата изключва нулата, *и* относителната разлика е поне пет процента. Статистическата различимост не е същото като значимост: при достатъчно изпълнения разлика от 0,4% става значима и остава безинтересна.

Броят на повторенията не се обявява, а се определя: една клетка се изпълнява двадесет пъти, за да се измери променливостта между изпълнения, и от нея се избира *n*.

Заедно с всеки резултат, при който хипотеза не е отхвърлена, се съобщава и *разделителната способност* на постановката, тоест полуширочината на доверителния интервал върху разликата като дял от медианата. Без нея непотвърждаването е твърдение за броя изпълнения, а не за измерването: всяка хипотеза остава непотвърдена при достатъчно малка извадка. Изискването броят повторения да бъде обоснован, а не приет, е формулирано и в литературата за оценяване на производителност [51], и настоящата методика го прилага, като допълнително съобщава и какво този брой е бил в състояние да различи.

5.15. Производна величина назовава изпълненията, от които идва

Отпечатъкът на средата не допуска сливане на две кампании, чиито машини се различават. Той обаче не вижда вътре в анализа: нищо не пречи на човек да вземе едно число от един файл и друго от втори и да ги раздели. Полученото не е измерване на нито едно от двете и не носи следа, по която това да се забележи.

Случаят, който наложи правилото, е от настоящата работа. Отношение между два вида натоварване беше пресметнато като 4,176, като общият брой системни повиквания беше взет от едно изпълнение, а броят на отчитанията на часовника — от друго. Пресметнато изцяло върху изпълнението, което произвежда сравнението, същото отношение е 4,116. Разликата е малка и затова опасна: тя не изглежда като грешка, а като число.

Затова всяка производна величина в глава VI назовава изпълненията, от които е получена, и величина, получена от повече от едно изпълнение, се обявява като такава или не се съобщава. Механизмът, който пази кампаниите една от друга, не пази анализа от себе си; това го прави правило за писане, а не за програма.

И изваждането на съставка не е точно. Свързано следствие от същия случай. Отношение, от което е извадена цената на инструмента, предполага, че извадената съставка е независима от останалите. Измерено: премахването на 4,308 отчитания на часовника на връзка премахна 5,945 системни повиквания, тоест около 1,64 повикване, което не е отчитане, си отиде заедно с тях. Аритметичното изваждане на един ред следователно не е същото измерване като премахването на извикванията и повторното изпълнение, и четирите защитими стойности на едно и също отношение се разпростират от 3,84 до 4,82. Съобщава се тази, която описва непроменения код на изправен хост, заедно с довода защо е тя.

5.16. Кога обща за двете рамена цена наистина се съкращава

Сравнение в рамките на един хост често се защитава с довода, че постоянна допълнителна цена, обща за двете рамена, се съкращава в разликата. Доводът е верен само при равен *брой* на плащанията и следователно е твърдение за натоварването, а не за инструмента. Проверява се за всяка форма поотделно, а не веднъж за цялата кампания.

Случаят, който го наложи, е поучителен и в двете посоки. На хост, чийто източник на време е НРЕТ, едно отчитане на часовника струва около 1931 ns вместо около 20 ns. При форма с преизползвани връзки двете рамена правят 2,0015 и 2,0018 отчитания на заявка, тоест равни до четвъртата значеща цифра, и цената наистина е обща и се съкращава напълно. При форма с установяване на връзка на всяка заявка същите две рамена правят 9,2928 и 8,6246 отчитания на връзка. Разликата от 0,668 отчитания е действителна работа, но инструментът я умножава по около деветдесет и седем: истинска разлика от около 13 ns на връзка се появява в записа като 1,3 μ s и сочи към едното рамо.

Следствието е общо. Обща цена се съкращава само там, където броят е равен, а където броят се различава, скъпият инструмент увеличава разликата пропорционално на собствената си цена. И понеже увеличената разлика изглежда точно като резултат, тя не се самоизобличава.

5.17. Разлагане на измерена величина на съставки

Няколко от резултатите в глава VI имат вида „тази величина се състои от част, пропорционална на заявките, и част, пропорционална на времето“. Такова твърдение е модел, а не отчитане, и затова се проверява по същия начин, по който се проверява всяко

друго измерване.

Две правила се спазват. Първо, разлагане с k неизвестни изисква поне $k + 1$ измервателни точки, защото при точно k точки остатъкът е нула по построение и съответствието не носи сведение. Второ, две величини, които са се изменили заедно, не са доказателство поотделно за нито една от тях: съответствие между два независими източника, обработени по един и същ объркан начин, потвърждава единствено, че объркването е било еднакво.

И двете правила бяха приложени, след като бяха нарушени. Оценка по две точки, в която темпът и формата на връзката се измениха едновременно, приписа цялата разлика на съставка, независеща от темпа, и съгласието на две отделни реализации беше представено като независима проверка. Трета точка при задържана форма опроверга разлагането за двадесет минути: величината се оказва постоянна на 2,00 при петкратна промяна на темпа, докато твърдяната независеща от темпа съставка би я направила спадаща от около 3,5 до 2,3.

5.18. Изводи

Методиката се свежда до няколко механизма, всеки от които прави определен вид тиха грешка невъзможна, вместо да я прави малко вероятна.

Двойката мрежови пространства прави сравнението между протоколите честно, което loorback не може. Един генератор прави сравнението между протоколите сравнение между протоколи, а не между клиенти, а същият довод, приложен към трите платформи, е причината за собствения генератор. Отвореният цикъл разделя времето за отговор от времето за обслужване, като брои от предвидения момент на изпращане. Един двоичен файл с флагове прави разликата между две рамена по-голяма от шума между сборки. Отпечатъкът на средата спира кампанията, вместо да смеси две популации. Предварително обявените критерии премахват свободата резултатите да бъдат избирани след като са видени, а закъснението на графика заменя процесорното време като сигнал за насищане там, където процесорното време вече не значи нищо. Последният от тези механизми дойде от грешка, а не от план: фиксираният изчаквателен срок на poll беше превърнал разделителната способност на системния таймер в измерена латентност.

Четири ограничения се заявяват тук, а не се откриват по-късно. За HTTP/3 няма зрял генератор с постоянен темп на заявките в отворен цикъл, тоест горните персентили

за него идват само от затворен цикъл. Под Windows няма програмируем еквивалент на `tc`, тоест измерванията при влошена мрежа не се извършват там, а разликите в UDP сегментирането правят твърденията за производителност на HTTP/3 валидни само за Linux. Измерванията на две машини се ограничават до малък брой клетки, използвани за проверка на валидността, а не за отделна матрица. И кампанията под Windows дели един хост между сървъра и генератора, тоест поддържа сравнения при еднакви условия, но не и абсолютни числа.

VI. РЕЗУЛТАТИ И АНАЛИЗ

Главата прилага методиката от глава V и представя това, което тя произведе. Числата в нея не са преписвани: скаларите се вмъкват по име от `generated/results.tex`, а таблиците и графиките се четат направо от CSV файловете в `data/`. Стойност, за която липсва измерване, се отпечатва като червен маркер, а не като правдоподобно число, и такива маркери в текста по-долу са умишлени.

Главата се чете в определен ред. Първо какво позволява средата, защото това ограничава кои твърдения изобщо могат да бъдат направени. После колко изпълнения бяха отхвърлени и по коя причина, защото кампания, изхвърлила една трета от изпълненията си, е различна кампания от такава, която не е изхвърлила нито едно. Едва след това резултатите.

6.1. Средата и какво тя позволява

Измерванията в тази глава са направени на една машина под Windows, неvirtуализирана, с шест физически ядра. Отпечатъкът на средата е записан заедно с изпълненията и кампанията отказва да дописва към файл, чиято машина се е променила.

Едната машина е ограничението. Генераторът и сървърът се изпълняват на един и същ хост и се конкурират за едни и същи ядра. Затова те са закрепени към несечащи се множества: сървърът към четири физически ядра, генераторът към две. Нито един от двата не разполага с цялата машина.

Следствието е върху вида на твърденията, а не върху точността им. Абсолютната пропускателна способност, измерена така, е свойство на разделянето на ядрата, а не на сървъра, и не се съобщава като характеристика на реализацията. Това, което конфигурацията поддържа, е *сравнение*: две рамена, които виждат едни и същи условия, при които се различава само изследваният фактор. Хипотеза X1 е точно такова сравнение и затова е измерима тук.

Loopback не е мрежа. Интерфейсът има MTU 65536 и време за обръщение от порядъка на микросекунди. Първото облагодетелства протоколите над TCP спрямо QUIC, тъй като QUIC изпраща дейтаграми от порядъка на 1200 байта по замисъл. Второто прави невидими свойствата, заради които мултиплексирането съществува. И понеже адресът е

един, миграцията на връзка не може да бъде предизвикана изобщо.

Затова тук няма сравнение между протоколи, няма измерване при влошена мрежа и няма нито един резултат за HTTP/3. Те не са пропуснати, а изключени от средата, и се провеждат в кампанията под Linux (Фиг. 12).

Обхватът на генератора. loadgen говори само HTTP/1.1, в явен вид и през TLS. Всичко в тази глава е следователно за HTTP/1.1, а таблиците за X1 са за връзки в явен вид.

6.2. Една кампания, три проекта, и защо се обработват поотделно

Данните идват от една кампания, проведена на 2 септември 2026 г., с три проекта в нея, и различаването на проектите не е счетоводна подробност. h1-deer изпълнява само клетките, от които зависи X1, в явен вид. transport изпълнява четирите рамена по транспорт: с и без класификация, в явен вид и през TLS. churn измерва установяването на връзки при темп на пристигане. Таблиците и графиките за време в тази глава четат h1-deer, а броенията в раздел 6.12 не зависят от нито един проект; другите два проекта се появяват по име там, където техни ключове носят сведение за твърдение в главата.

Обхождането на трите фактора около базовата конфигурация, брой работни нишки, размер на отговора и дължина на опашката за приемане, е предвидено при седем повторения на клетка и не е проведено в тази кампания. Клетките му нямат измерване, и ключовете му, като `[?campaign.sweeps.runs]`, се отпечатват червени по замисъл, както и таблиците в съответните раздели по-долу.

Повторенията в h1-deer са двадесет и пет на клетка: 250 изпълнения. Броят повторения следва от прага, при който X1 обявява разлика за докладваема, пет процента, и от това, че полуширочината на доверителния интервал върху разликата се свива като корен квадратен от броя повторения: при седемте повторения, които инструментът приема по подразбиране, тя би била около два пъти по-голяма. Постигнатата разделителна способност е измерена, а не предположена: полуширочината на интервала върху разликата между двете рамена е между 2.47% и 5.63% от медианата, според предложеното натоварване. Тоест при двадесет и пет повторения постановката разграничава разликата, която сама е обявила, че търси, при четири от петте натоварвания, а при 25 000 заявки в секунда не я разграничава. Това е констатация за постановката, а не за резултата, и лекарството е повторения.

Трите проекта са изпълнени на една и съща машина, от един и същ двоичен файл, и носят един и същ отпечатък на средата. Обработват се поотделно не защото механизмът от глава V отказва да ги слее, а защото са отделни проекти и се обобщават поотделно: таблицата за X1 идва изцяло от h1-deep и никоя нейна клетка не е допълвана от другите два проекта, дори там, където те предлагат същото натоварване. Отказът от глава V остава в сила и би сработил, ако някой от проектите бъде допълван от друго дърво. Трите обхождания нямат кампании, от които да дойдат.

6.3. Допустимост: колко изпълнения бяха отхвърлени

Малко, и всяко със записана причина. От 250 изпълнения в проекта h1-deep са отхвърлени 6 и са приети 244. Всяко отхвърляне е за изоставане на графика над границата или за отклонение на честотата на процесора над допустимото, и нито едно не е за отговор извън 2xx или за грешка на сокет. От 500 изпълнения в transport е отхвърлено 1, а от 400 в churn са отхвърлени 6. Обхожданията не са проведени и нямат какво да бъде отхвърлено.

Малък брой отхвърлени изпълнения е твърдение, което трябва да бъде защитено, а не отбелязано. Прочита се по два начина: или изпълненията са били чисти и правилата са хванали малкото нечисти, или правилата са били толкова хлабави, че почти нищо не би ги задействало. Разликата се вижда само ако се съобщи докъде е стигнал всеки сигнал, а не само дали е преминал границата.

Таблица 9. Правила за допустимост и най-лошата достигната стойност

Сигнал	Граница	Обхождания	X1 (h1-deep)
Изоставане на графика, p99	$< 1000 \mu s$	[?campaign.sweeps.p99 > 1000 μs]	979 809 μs
Постигнат дял от предложеното	$\geq 0,99$	[?campaign.sweeps.shade > 0.99]	0.9969
Отговори извън 2xx	0,1%	[?campaign.sweeps.non2xx]	0
Грешки на сокет	0	[?campaign.sweeps.socket-errors]	0

Колоната за обхожданията е червена, защото те не са проведени. Колоната за X1 казва това, което броят на отхвърлянията не може, и се чете с уговорката, че стойността в нея е най-лошата над всички изпълнения, включително отхвърлените. Изоставането на графика в нея е 979 809 μs: отхвърлено изпълнение при 55 000 заявки в секунда, при което генераторът е изостанал от собствения си график с почти секунда и дялът му процесорно

време е паднал под обичайния за проекта. Границата е премината само в отхвърлените изпълнения, а сред приетите най-лошата стойност остава далеч под нея (Табл. 10). Тоест правилото не е бездействащо: сработи там, където генераторът е най-близо до тавана си, и приетата съвкупност остава далеч от границата.

Защо натоварването на генератора не е правило тук, и какво все пак казва. Делът процесорно време на генератора е 0.999 2 от закрепените му две ядра, тоест практически единица при всички натоварвания в тази постановка. Това не е признак за насищане. Генераторът върти празен цикъл през последните няколко микросекунди преди всеки насрочен момент, защото разделителната способност на изчакването под Windows е твърде груба за темп от десетки хиляди заявки в секунда (Фиг. 13). Като критерий за отхвърляне той следователно не носи сведения тук, и правилото важи само за затворения цикъл; сигналят за насищане при отворен цикъл е изоставането на графика, което е величина за самия график, а не за машината под него.

Обратното обаче не следва, и е грешка, която тази работа направи и след това си взе обратно. Делът е практически единица *докато генераторът е насит.* При достатъчно нисък предложен темп той наистина има малко работа и делът пада: в проекта с установяване на връзки той не надхвърля 0.857 7 при нито едно натоварване и е най-нисък при двадесет и пет установявания в секунда, срещу 0.999 2 в h1-deep и 0.999 2 в transport. Тоест като критерий за отхвърляне той е безполезен, а като диагностика е точно разграничителят, който отделя незасита постановка от засита. Разликата между двете е разликата между правило и наблюдение и двете не се използват по един и същ начин.

Докъде стига генераторът. Изоставането на графика е измерено при всяко изпълнение и се съобщава тук по предложено натоварване, а не само като едно число за цялата кампания. Причината е, че единственото число не различава генератор, който е далеч от границата си навсякъде, от генератор, който я приближава точно там, където се правят твърденията.

Таблица 10. Изоставане на генератора от собствения му график, 99-и персентил в микросекунди

Предложени	с кл. медиана	най-лошо	без кл. медиана	най-лошо
10,000	48	60	47	171
25,000	47	54	47	57
40,000	51	72	52	78
55,000	61	130	60	90
70,000	80	125	80	157

Медианата расте с натоварването, тоест генераторът се затруднява точно там, където се очаква, и остава на порядък от границата си. Най-лошата единична стойност сред приетите изпълнения е при най-ниското натоварване, в рамките без класификация, и остава далеч под границата. Най-лошата стойност над всички изпълнения, 979 809 μ s от таблица 9, е отхвърленото изпълнение при 55 000 и не е в тази таблица, защото тя съдържа само приетите.

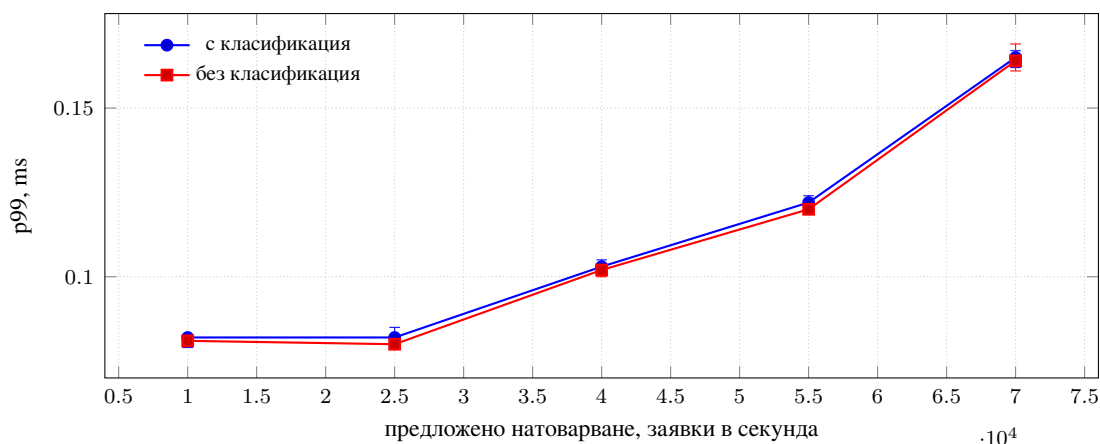
Предложените натоварвания спират на 70 000. Над този темп в тази кампания не е измервано и за поведението на генератора там не се прави твърдение. Това, което проектът сам показва за границата, е следното: всички негови отхвърляния за изоставане на графика са при 40 000 и нагоре, а медианата на изоставането расте с темпа (Табл. 10). Измерване над последния ред на таблицата би описвало генератора, а не сървъра.

6.4. X1: цената на класификацията след приемането

Хипотезата гласи, че разпознаването на протокола по първия октет не води до измеримо влошаване спрямо същия сървър без класификация. Двете рамена са един и същ двоичен файл, разграничен с флаг при изпълнение, при еднакво предложено натоварване в отворен цикъл.

Таблица 11. Пропускателна способност и латентност с и без класификация; n е броят приети изпълнения в клетката

Предложени	n	с кл. заявки/s	p50 (ms)	p99 (ms)	n	без кл. заявки/s	p50 (ms)	p99 (ms)
10,000	25	10,000	0.06	0.082	24	10,000	0.06	0.081
25,000	25	24,999	0.059	0.082	25	24,999	0.059	0.08
40,000	25	39,998	0.057	0.103	23	39,998	0.057	0.102
55,000	25	54,998	0.07	0.122	24	54,998	0.07	0.12
70,000	24	69,998	0.066	0.165	24	69,997	0.066	0.164



Фигура 14. Латентност при 99-и персентил спрямо предложеното натоварване, двете рамена на хипотеза X1. Лентите са 95% ВСа доверителни интервали върху медианите на изпълненията.

Как се чете. Хипотезата се отхвърля само ако доверителният интервал на разликата изключва нулата и относителната големина надхвърля пет процента. И двете условия са обявени в глава I, преди събирането на данните. Разликата се получава чрез повторни извадки върху самата разлика, а не чрез сравняване на застъпването на двата интервала: непокриващи се интервали наистина означават разлика, но покриващи се не означават нейната липса, и проверката по застъпване е по-консервативната от двете без принципно основание за това.

Таблица 12. Разлика между двете рамена по p99, с класификация минус без нея

Предложени	Разлика (ms)	долна	горна	% от медианата
10,000	0.001	−0.001	0.003	1.23
25,000	0.002	−0.004	0.005	2.5
40,000	0.001	−0.003	0.004	0.98
55,000	0.002	−0.0015	0.0085	1.67
70,000	0.001	−0.0055	0.0055	0.61

Резултат. При всяко от петте предложени натоварвания доверителният интервал на разликата съдържа нулата и относителната разлика е под пет процента. Тя е между 0.61% и 2.50%, тоест най-много половината от прага. **X1 не се отхвърля.** Пропускателната способност на двете рамена съвпада до една заявка в секунда при всяко натоварване, което е очаквано в отворен цикъл: генераторът предлага заявки по график, и докато сървърът смогва, той обслужва точно предложеното.

Твърдението има покритие при четири от петте натоварвания. Полуширочината

на интервала върху разликата е между 2.47% и 5.63% от медианата. При 10 000, 40 000, 55 000 и 70 000 заявки в секунда тя е под прага, тоест постановката различава разлика, по-малка от тази, която сама е обявила. При 25 000 тя е над прага и тази клетка не би могла да отдели от нулата разлика от пет процента; непотвърждаването там е твърдение за броя изпълнения, а не за сървър.

Какво точно е ограничено и какво не. Тук трябва да се направи разграничение, което таблицата сама не прави и без което резултатът звучи по-широк, отколкото е.

Класификацията има две цени. Едната е на връзка: едно четене преди всичко останало и една проверка на първия октет. Другата е на заявка: обвивката `PrefaceConnection` остава в пътя за четене през целия живот на връзката и добавя по едно виртуално извикване на четене (Фиг. 5).

Проектът поддържа шестдесет и четири устойчиви връзки за двадесет секунди и през тях преминават от двеста хиляди заявки при най-ниското натоварване до милион и четиристотин хиляди при най-високото. Тоест цената на връзка се плаща шестдесет и четири пъти, а цената на заявка от двеста хиляди до над милион пъти. При такова съотношение първата е под разделителната способност на измерването, каквато и да е тя, а втората е това, което таблицата ограничава.

Следователно резултатът гласи: *цената на класификацията по заявка е между 0.61% и 2.50% от p99 като точкова оценка при петте натоварвания, с интервал, който при всяко от тях съдържа нулата* (таблица 12). Той не гласи нищо за цената на връзка. За нея е нужно натоварване с темп на *пристигане на връзки*, а не на заявки, и този проект не го произвежда. Проектът `churn` от същата кампания го произвежда, 400 изпълнения, и резултатите му остават за попълване в тази глава; повторението му през мрежов път е записано като изискване в раздел 6.14.

6.5. Разпределението на латентността по цялата си дължина

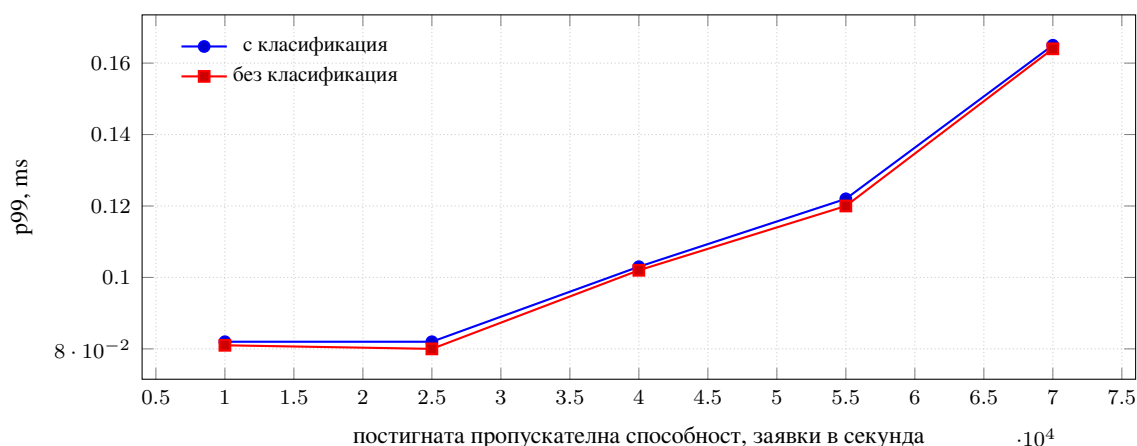
Таблиците дотук показват два персентила. Хипотезата се решава от тях, но сървър, описан с два персентила, е описан наполовина: разликата между рамената може да е нула в средата и различна в опашката, и обратното.

Таблица 13. Персентили на латентността, двете рамена, в милисекунди

Предл.	с класификация				без нея			
	p50	p90	p99	p99.9	p50	p90	p99	p99.9
10,000	0.06	0.071	0.082	0.126	0.06	0.07	0.081	0.124
25,000	0.059	0.07	0.082	0.14	0.059	0.07	0.08	0.138
40,000	0.057	0.084	0.103	0.157	0.057	0.083	0.102	0.158
55,000	0.07	0.09	0.122	0.217	0.07	0.089	0.12	0.205
70,000	0.066	0.127	0.165	0.291	0.066	0.128	0.164	0.298

Формата на разпределението е една и съща в двете рамена. Медианата остава между 0.06 и 0.07 ms при всяко натоварване. Деветдесетият персентил следва медианата на разстояние от около една стотна до 25 000 заявки в секунда, отделя се от нея при 40 000 и при 70 000 е около два пъти медианата. Деветдесет и деветият се отделя по същия начин и на същото място. Тоест до 25 000 заявки в секунда опашката се дължи на отделни събития, а от 40 000 нагоре към тях се добавя натрупване в опашката за обработка, което расте с темпа. Обхождането по backlog в раздел 6.11, което би проверило това от другата страна, не е проведено.

Персентилът 99,9 не следва тази форма и не бива да се чете от тази таблица като величина. Раздел 6.8 обяснява защо.



Фигура 15. Опашка спрямо постигната пропускателна способност. Двете криви са неразличими, което е твърдението на X1, представено така, както се чете при избор на работна точка.

6.6. Процесорно време и памет

Латентността и пропускателната способност описват какво вижда клиентът. Процесорното време и паметта описват какво струва това, и се съобщават отделно, защото равна латентност при различна цена е различен резултат от равна латентност при равна цена.

Таблица 14. Процесорно време на сървъра за изпълнение, в секунди, и връх на работното множество, в MiB

Предл.	с кл.	долна	горна	без кл.	долна	горна	памет
10,000	9.34	8.86	9.67	9.04	8.82	9.39	9.75
25,000	23.92	23.55	24.06	23.03	22.9	23.11	9.77
40,000	37.02	35.3	39.38	36.55	35.03	39.16	9.77
55,000	50.11	49.38	50.12	49.06	48.92	49.34	9.89
70,000	59.66	59.09	59.88	58.66	58.12	58.88	9.79

Паметта не зависи от натоварването. Върхът на работното множество остава около десет мебибайта при всяко предложено натоварване, тоест при седемкратна разлика в обема обслужен трафик. Това е съгласувано с твърдението от глава II за произхода на цената за връзка: при корутини без стек тя се определя от текста на програмата, а броят връзки тук е постоянен. Наблюдението не е проверка на това твърдение, защото броят връзки не е обхождан; то е съгласуваност, а не потвърждение, и обхождането по брой връзки остава за кампанията под Linux.

Процесорното време расте почти пропорционално. От 9.34 секунди при 10 000 заявки в секунда до 59.66 при 70 000 в рамото с класификация, тоест малко над шест пъти при седемкратно натоварване. Разликата между двете рамена е разгледана в следващия раздел.

6.7. Правилото за докладваемост върху процесорното време

Правилото за докладваемост беше приложено и към процесорното време на сървъра, и при нито едно от петте натоварвания то не сработи. Медианата в рамото с класификация е по-висока от тази в рамото без нея при всяко натоварване, и при 25 000, 55 000 и 70 000 заявки в секунда интервалите на двете рамена в (Табл. 14) не се застъпват: при 25 000 те са 23.92 срещу 23.03 процесорни секунди. Относителната разлика обаче е под пет процента навсякъде, тоест по буквата на обявеното правило тя не е докладваема.

Наблюдението е записано тук по две причини. Правилото е обявено предварително и трябва да се съобщи какво е дало, независимо в чия полза. И посоката на разликата е една и съща в петте клетки, при постоянен брой връзки и равна латентност, тоест ако класификацията има цена на заявка, процесорното време е мястото, където тя би се проявила преди опашката; постановката не може да я отдели като докладваема при този брой повторения и не я обявява за такава.

Разсейването вътре в едно рамо е указание какво прави машината при всяко натоварване: при 10 000 заявки в секунда то е около 1,5 пъти между най-малката и най-голямата стойност, докато при 70 000 е около 1,1 пъти. Тоест при ниско натоварване хостът е в област, в която управлението на честотата и разпределянето на ядрата под Windows все още се колебаят, а при високо вече не се колебаят.

6.8. Разпределението на p99.9 е двумодално и затова не се използва

Персентилът 99,9 е в таблиците, но не се използва за сравнение, и това изисква обяснение, защото опашката е явлението, а не смущението.

От 244 приети изпълнения в h1-deep 61 имат p99.9 над една милисекунда, а останалите имат p99.9 между 0,1 и 1 ms с медиана около 0,15 ms. Двете групи не са разделени с празнина, а с рядко населена ивица: между 0,3 и 3 ms попадат малко изпълнения, но има такива. Над една милисекунда p99.9 е разпределен непрекъснато от около 1 до около 45 ms с медиана около 15 ms. Най-голямата наблюдавана единична латентност е около 60 ms и се среща в изпълнение от долната група.

Три наблюдения определят как това се тълкува.

Дялът е от един и същ порядък в двете рамена. Броенето е по клетки, а не общо, защото общият брой би скриел зависимост от натоварването, ако такава има.

Таблица 15. Изпълнения с p99.9 над една милисекунда, от приетите в клетката

Предложено натоварване	с класификация	<i>n</i>	без класификация	<i>n</i>
10,000	6	25	4	24
25,000	8	25	6	25
40,000	5	25	4	23
55,000	9	25	5	24
70,000	7	24	7	24

Дялът не следва натоварването: при 40 000 заявки в секунда е по-нисък, отколкото

при 25 000, а при 55 000 и 70 000 е същият, какъвто е при 25 000. Конкуренция за процесорно време, която расте с натоварването, не го обяснява. Разликата между двете рамена в отделна клетка достига четири изпълнения от двадесет и пет, при 55 000 заявки в секунда, и е нула при 70 000. Рамото с класификация има повече изпълнения във високия режим при четири от петте натоварвания. При дял около една четвърт стандартното отклонение на броя в клетка от двадесет и пет е около две изпълнения, тоест четири е в обхвата на разсейването и постановката не може да го отдели като ефект.

Дялът се мени с времето, но не монотонно. Проектът продължава около сто и три минути; шест от двадесет и петте повторения нямат нито едно изпълнение във високия режим, а четиридесет и девет от 61 са в десетте повторения между тридесет и третата и седемдесет и петата минута. Първото се появява във втората минута на първото повторение, последното в предпоследното повторение. Тоест не е нагряване и не е дрейф, а прозорец от около четиридесет минути, в който хостът е бил в друго състояние, и двете рамена попадат в него еднакво.

Големината съответства на планировчика, а не на сървъра. Спиране от една до около шестдесет милисекунди при сървър, чиято медиана е 0,06 ms, е от един до три порядъка над всичко останало в пътя на заявката, а горният му край е от порядъка на кванта на планировчика под Windows.

Изводът е за приложимост, а не за резултат: на този хост p99.9 не е използваем за сравнение между две рамена, защото режимът, в който попада едно изпълнение, се определя от нещо извън сървъра и не се усреднява при двадесет и пет повторения. Персентилът 99 е едномодален и устойчив и затова е основната величина в тази глава. Персентилът 99,9 се съобщава, за да е видимо защо не се използва.

6.9. Мащабиране по работни нишки

`[?data/workers.csv]`

Сървърът е закрепен към четири физически ядра, тоест над четири работни нишки се очаква насищане, а не подобрение. Стойността на измерването е в това дали кривата се държи както предвижда закрепването, което е проверка на самата постановка, а не на реализацията.

Обхождането, което проверява това, не е проведено, и на мястото на таблицата по-горе стои червен маркер по замисъл. Планът му е една, две, четири и осем нишки

при 40 000 заявки в секунда, с включена класификация, при подразбиращите се седем повторения на клетка. Очакването, което то ще провери, е следното: при отворен цикъл под насищането пропускателната способност е една и съща при четирите стойности, защото сървърът обслужва предложеното и при една нишка, а разликата е в процесорното време, което расте с броя нишки при една и съща свършена работа. Единствените измерени стойности при една, две, четири и осем нишки в тази глава са броенията в раздел 6.12, които не измерват време.

Обхождането ще бъде ограничено по два начина, които се казват преди провеждането му. Първо, при 40 000 заявки в секунда сървърът не е при насищане, тоест то няма да намери къде мащабирането се чуи, а само дали под насищането то е нужно. Второ, при седем повторения интервалът върху разликата е около два пъти по-широк, отколкото при двадесет и петте на h1-deer. Затова таблицата ще може да поддържа твърдението „не се подобрява“ и няма да може да поддържа твърдението „не се влошава“.

6.10. Размер на отговора

`[?data/payload.csv]`

Размерът на отговора е включен, защото цената на класификацията е на връзка, а цената на отговора е на заявка. Ако първата изобщо е разграничима, съотношението между двете трябва да се мени с размера, и това е втори, независим начин да се погледне същият въпрос.

Обхождането не е проведено, и на мястото на таблицата стои червен маркер по замисъл. Планът му е три размера на отговора, 256, 1024 и 8192 байта, около базовия празен отговор, при 40 000 заявки в секунда, с включена класификация, при подразбиращите се седем повторения. Очакването, което то ще провери, следва от раздел 6.4: при устойчиви връзки класификацията не е на заявка и затова не бива да се появява в разлика, която расте с размера на отговора.

Под loorback обхождането ще покаже ограничението на интерфейса, а не свойство на сървъра: осем килобайта през интерфейс с MTU 65536, без сегментиране, без контролни суми и без прекъсвания, не струват това, което биха стрували през реална мрежа. Затова същото обхождане е предвидено и за кампанията под Linux, където границата на сегмента е около 1500 байта и трите размера попадат от двете ѝ страни.

6.11. Дължина на опашката за приемане

[?data/backlog.csv]

Подразбиращата се стойност в интерфейса за вход и изход беше 128, което е малко при няколко хиляди едновременни връзки, затова параметърът е предвиден за обхождане, а не приет за константа: 128, 512 и 4096 при 40 000 заявки в секунда и шестдесет и четири устойчиви връзки. При това натоварване и брой връзки опашката е плитка и не се очаква разлика. Обхождането не е проведено, тоест това остава допускане, а не резултат.

Дори проведено, обхождането ще отговори само на половината въпрос. Опашката за приемане има значение при пристигане на много връзки едновременно, а постановката на обхождането установява шестдесет и четири връзки веднъж и после не установява повече. Тоест то може да покаже, че параметърът не вреди при устойчиви връзки, и нищо за случая, заради който съществува. Това е същата слепота, установена в раздел 6.4. Установяването с темп е измерено в същата кампания от отделен проект, churn, с 400 изпълнения при темп на пристигане на връзки, но той държи backlog постоянен.

6.12. Преброяване на слушащите дескриптори

Предложението от раздел 2.5 е твърдение за брой, а не за скорост, тоест то се проверява чрез броене, а не чрез натоварване. Броенето е и единственото измерване в главата, което не зависи от генератора.

Броенето се прави от операционната система, а не от сървъра. Процесът се пуска, изчаква се портът да отговори, и след това ядрото се пита колко слушащи крайни точки притежава този процес. Сървър, който съобщава собствения си брой дескриптори, потвърждава очакването на автора си, а не състоянието на машината.

Таблица 16. Слушащи дескриптори на работещ процес, отчетени от ядрото

Работни нишки	Класификация	TCP	UDP	Общо
1	1	1	0	1
2	1	1	0	1
4	1	1	0	1
8	1	1	0	1
1	1	1	0	1
2	1	1	0	1
4	1	1	0	1
8	1	1	0	1
1	0	1	0	1
2	0	1	0	1
4	0	1	0	1
8	0	1	0	1
1	0	1	0	1
2	0	1	0	1
4	0	1	0	1
8	0	1	0	1
1	1	1	0	1
1	1	1	0	1
1	1	1	0	1
1	1	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1

Първото, което таблицата казва. Броят не се мени при включване и изключване на класификацията. Това е твърдението, заради което броенето съществува, и то се потвърждава: обслужването на HTTP/1.1, HTTP/2 и WebSocket от един и същ дескриптор не струва дескриптор.

Второто, което таблицата казва, опровергава по-ранна формулировка. Броят не се мени и с работните нишки: един слушащ дескриптор при една, две, четири и осем. По-ранната редакция на предложението пишеше $L = W \cdot |T|$ безусловно, тоест

толкова слушателя, колкото са работните нишки. Това е вярно за модел на приемане със `SO_REUSEPORT`, при който всяка нишка има собствен слушател, но не е вярно за модел с един споделен слушател, какъвто са IOCP под Windows и `kqueue` под macOS (Фиг. 1). Затова предложението е преформулирано като $L = A \cdot |T|$, в което A е броят приемащи дескриптори, изискван от модела на приемане на платформата: W при първия модел, единица при втория.

Преформулирането не отслабва твърдението, което работата защитава. То е за независимостта от броя обслужвани приложни протоколи, тоест за отсъствието на $|P|$ в израза, и тя се запазва. Множителят пред нея беше сгрешен, и измерването е това, което го хвана.

Защо няма UDP слушател. Двоичният файл за измерването е компилиран без HTTP/3 и не отваря UDP сокет, тоест колоната е нула по конфигурация, а не по резултат. Числото за $|T| = 2$ идва от кампанията под Linux.

Същото броене под macOS. Преброяването е повторено върху `kqueue`, тоест върху втори модел на приемане, на Apple M4 Pro под macOS 26. Резултатът е един слушач дескриптор при една, две, четири и осем работни нишки, с включена и с изключена класификация, с и без TLS. Във всяка колона, която е твърдение, слушащи дескриптори по TCP и UDP и установени връзки, двете броения съвпадат във всички клетки; различават се само в трите колони, които са свойство на платформата: портове за събития, нишки в покой и нишки след сондата.

Таблица 17. Слушащи дескриптори под macOS и kqueue, отчетени от ядрото. Отпечатаните колони съвпадат с (Табл. 16); файлът се различава от този под Windows само в колоните, които са свойство на платформата. Това е броене върху втори модел на приемане, а не второ независимо измерване на същата величина.

Работни нишки	Класификация	TCP	UDP	Общо
1	1	1	0	1
2	1	1	0	1
4	1	1	0	1
8	1	1	0	1
1	1	1	0	1
2	1	1	0	1
4	1	1	0	1
8	1	1	0	1
1	0	1	0	1
2	0	1	0	1
4	0	1	0	1
8	0	1	0	1
1	0	1	0	1
2	0	1	0	1
4	0	1	0	1
8	0	1	0	1
1	1	1	0	1
1	1	1	0	1
1	1	1	0	1
1	1	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1
1	0	1	0	1

Какво това добавя и какво не. Добавя следното: броят вече не е свойство само на IOCP. Той е преброен на две платформи с различни ядра, различни компилатори и различни механизми за вход и изход, и е един и същ, тоест $A = 1$ е следствие от модела с един споделен слушател, а не особеност на едната реализация. Изворният текст на kqueue backend-a го предсказва, защото създава точно един слушащ дескриптор и никъде не използва SO_REUSEPORT, но предсказание не е измерване, а сега измерване има.

Не добавя следното: това не е второ независимо наблюдение в статистическия смисъл и не се представя като такова. Двете таблици съвпадат във всяка клетка, която

е твърдение, тоест няма разсейване, което да бъде оценено, и n не е две. Броенето е детерминирано: една и съща конфигурация дава един и същ брой при всяко изпълнение, и точно затова величината е доказуема, а не оценима. Твърдението, което двете таблици заедно поддържат, е за обхвата на предложението, а не за неговата точност.

Не добавя и нищо за *едновременността* на приемането, която е друга величина. Под крещеие тя също е единица, но по причина, която не е замисъл: интерфейсите по готовност определят наблюдението по двойката дескриптор и филтър, тоест няколко приемащи операции върху един слушащ дескриптор се събират в едно наблюдение. Това е записано в раздел 4.1 и е ограничение на реализацията, а не резултат.

6.13. Какво тази кампания не измерва

Три от четирите хипотези остават непроверени тук, и причината за всяка е различна.

X2, делът на препратените QUIC пакети. Изисква HTTP/3, а тази среда няма конфигурация за него, и изисква предизвикана миграция, която на един адрес е невъзможна. Провежда се под Linux.

X3, изчакано срещу отложено изобразяване. Изисква генератор, който изобразява страница и измерва време до първия байт отделно от времето до последния. `loadgen` не прави това. Функционалната проверка, че обвивката тръгва преди стойността, съществува и е описана в глава VII, но тя е проверка на поведение, а не измерване.

X4, времето за маршрутизиране при нарастващ брой маршрути. Изисква обхождане по брой регистрирани маршрути, което не е част от тази кампания. То не изисква Linux и може да бъде проведено на същата машина. Пропускът е в разпределението на времето, а не в средата, и се отбелязва като такъв.

6.14. Запланувани измервания под Linux и macOS

Разделът съществува, за да е ясно какво липсва и къде ще застане, а не за да обещава. Ключовете по-долу са същите, които текстът ще използва, когато кампаниите бъдат проведени. Дотогава те се отпечатват като червени маркери, тоест липсата е видима в самия документ и се намира с търсене, вместо да бъде премълчана.

Таблица 18. Какво остава за измерване, на коя платформа и защо там

Измерване	Платформа	Защо не тук
Цена на класификацията на връзка	Linux	Под Windows е измерена в същата кампания, 400 изпълнения при темп на пристигане на връзки; остава повторението ѝ през мрежов път, което под Windows беше спряно от защитната стена на хоста.
Сравнение между HTTP/1.1, HTTP/2 и HTTP/3	Linux	Изисква h2load и мрежов път с реален MTU. Loopback наказва QUIC по причина, която няма нищо общо с реализацията.
Готовност срещу завършване, epoll срещу io_uring	Linux	Двата механизма съществуват на една и съща операционна система само там, което прави вида на интерфейса независима променлива.
Дял на препратените QUIC пакети (X2)	Linux	Изисква предизвикана миграция между два адреса в отделни мрежови пространства.
Влошена мрежа: закъснение, загуба, разбъркване	Linux	Под Windows няма програмируем еквивалент на tc.
kqueue като трети механизъм	macOS	Единствената платформа, на която съществува. Нативно изпълнение, не във виртуална машина.
Проверка на две машини	Linux	Отделя ефекта от това, че клиент и сървър делят една машина, от измерването.

Ключовете, които ще бъдат попълнени. Пропускателна способност под Linux с класификация: [\[?coroute-linux.h1.protocol-detection-on.40000.4.0.1024.rps\]](#) заявки в секунда, при p99 [\[?coroute-linux.h1.protocol-detection-on.40000.4.0.1024.p99\]](#) ms. Същото под macOS: [\[?coroute-macos.h1.protocol-detection-on.40000.4.0.1024.rps\]](#). Делът на препратените QUIC пакети: [\[?coroute-linux.h3.fwd-in\]](#) от [\[?coroute-linux.h3.rx\]](#) получени.

Всеки от тези маркери е червен в отпечатания документ. Това е предвиденото състояние до провеждането на съответната кампания и е за предпочитане пред число, което изглежда измерено.

6.15. Изводи

Всичко в тази глава е за HTTP/1.1, върху една машина под Windows, при която сървърът и генераторът делят хоста и са закрепени към несечащи се ядра; таблиците за X1 са за връзки в явен вид. Абсолютните стойности са свойство на това разделяне. Сравненията не са.

X1 не се отхвърля. При пет предложени натоварвания от 10 000 до 70 000 заявки в секунда разликата в р99 между рамото с класификация и рамото без нея е между 0.61% и 2.50%, интервалът ѝ съдържа нулата при всяко от 5-те натоварвания, а разделителната способност на постановката е между 2.47% и 5.63%, под прага при четири от петте натоварвания и над него при 25 000. Тоест при четири натоварвания непотвърждаването е твърдение за сървър, а при едно за броя изпълнения. Ограничението на това твърдение е обхватът му: то се отнася за цената по заявка. Цената на връзка не се измерва от този проект, защото при шестдесет и четири устойчиви връзки и от двеста хиляди до милион и четиристотин хиляди заявки тя е амортизирана под всяка разделителна способност; измерена е от проекта churn в същата кампания и резултатите му остават за попълване в тази глава.

Предложението за дескрипторите се потвърждава като наблюдение и се поправя като формулировка. Броят слушащи дескриптори не зависи от това дали класификацията е включена, което е твърдението, което работата защитава. Той не зависи и от броя работни нишки под Windows, което опровергава множителя W в по-ранната редакция и води до $L = A \cdot |T|$, в което A идва от модела на приемане на платформата.

X2, X3 и X4 остават непроверени, и причината е различна за всяка: X2 изисква Linux и два адреса, X3 изисква генератор, който loadgen не е, а X4 не изисква нищо, което липсва, и е въпрос на неразпределено време.

Две наблюдения за методиката излизат от кампанията, а не от плана. Правилото за докладваемост, приложено към процесорното време, не сработи при нито едно натоварване, но рамото с класификация е по-скъпо в петте клетки и в три от тях интервалите не се застъпват, вижте раздел 6.7. И персентилът 99,9 на този хост е двумодален по причина извън сървър, което го прави неизползваем за сравнение дори при двадесет и пет повторения, вижте раздел 6.8. И двете са записани, защото кампания, която съобщава само онова, което е потвърдило очакването ѝ, не е проверяема.

VII. ПРИЛОЖЕНИЕ И ВАЛИДАЦИЯ

Библиотека може да бъде бърза и въпреки това неизползваема. Тази глава проверява втората половина: дали архитектурните решения от глава III остават поносими, когато с тях се пише приложение, а не измерителен стенд.

Тези решения не са неутрални спрямо кода на приложението. Демултиплексирането обещава, че добавянето на протокол не разклонява приложението; отложеното получаване въвежда нов тип в модела на изгледа. И двете твърдения се проверяват най-добре, като се напише нещо с тях.

7.1. Примерното приложение

Приложението е списък със задачи с потребители. То се състои от 1790 реда на C++ в `examples/Project`, 206 реда шаблони и 846 реда JavaScript и CSS за страницата. Броенето е с `wc -l` върху изходните файлове, тоест включва коментари и празни редове. Организацията е по предназначение, а не по тип на файла.

Таблица 19. Организация на примерното приложение

Директория	Съдържание
<code>src/app</code>	Конфигурация и съставяне на сървъра.
<code>src/handlers/api</code>	API маршрути за задачи и потребители.
<code>src/handlers</code>	Маршрути за страници, тоест изгледи.
<code>src/handlers/websocket</code>	Двупосочен канал за известия при промяна на задача.
<code>src/middleware</code>	Журналиране на заявките и удостоверяване.
<code>src/models,</code> <code>src/services</code>	Данни и логика, независими от протокола.
<code>templates</code>	Шаблони за страници и общо оформление.
<code>static</code>	CSS и JavaScript на страницата.
<code>tests</code>	Празна. Виж бележката по-долу.

Редът за `tests` е записан такъв, какъвто е. Директориите съществуват, съдържат само `.gitkeep`, а блокът в `CMakeLists.txt`, който би построил тези тестове, е коментиран заедно със списъка на осемте файла, които е трябвало да съдържа. Приложението няма собствени тестове. Това е липса, а не решение, и се брои като такава в раздел 7.5.

Разделянето на `handlers/api` от `handlers` за страници не е наложено от библиотеката като директория, а следва от разделянето на маршрутите: API маршрутите

и маршрутите за изгледи се регистрират поотделно и се обслужват от отделни маршрутизатори. Приложението просто отразява това разделение в структурата си.

7.1.1. Маршрутите

Маршрутите са тринадесет HTTP и един WebSocket, регистрирани от четири места, плюс статичните файлове, които минават през middleware, а не през маршрутизатора. Пълният списък е в таблица 20, защото твърденията по-надолу се позовават на конкретни редове от него.

Таблица 20. Маршрутите на примерното приложение

Маршрут	Какво прави	Иска сесия
GET /	Изобразява pages/index.html.	не
GET /login	Изобразява формата; пренасочва към /, ако вече има сесия.	не
GET /logout	Пренасочва и изтрива двете бисквитки.	не
POST /api/login	Проверява данните и поставя user_id и username.	не
POST /api/logout	Изтрива същите бисквитки.	не
GET /api/me	Текущият потребител по user_id.	да
GET /api/users	Списък за падащото меню при възлагане.	не
GET /api/stats/tasks	Броячи по състояние.	не
GET /api/tasks	Списък с филтри и странициране.	не
POST /api/tasks	Създава задача.	да
GET /api/tasks/{id}	Една задача или 404.	не
PUT /api/tasks/{id}	Променя задача.	да
DELETE /api/tasks/{id}	Изтрива задача.	да
WS /ws	Каналът за известия.	не
GET /static/...	Статични файлове, през middleware.	не

Един ред от таблицата е следа от ограничение на библиотеката, а не избор на приложението. Статистиката се обслужва от /api/stats/tasks, а не от естественото /api/tasks/stats, и изходният текст записва причината на самото място: за да не се сблъска с /api/tasks/{id}. Тоест авторът е преименувал маршрут, вместо да разчита, че постоянният сегмент има предимство пред параметъра. В тестовите на маршрутизатора

няма случай, който да фиксира кой от двата печели, тоест не е установено дали маршрутизаторът не може да ги различи, или просто не е бил опитан. Това е находка, която се появява само когато някой напише приложение.

7.1.2. Услуги и модели

Услугите държат състоянието и не знаят нищо за HTTP. `TaskService` и `UserService` го пазят в `std::unordered_map` под `std::mutex` и предлагат обикновени методи: `create`, `find`, `list`, `update`, `remove`, `authenticate`. Няма база от данни, тоест никаква услуга не изчаква истински вход-изход. Това е най-сериозното ограничение на примера.

Моделите носят три функции: `to_json`, `from_json` и `validate`. Последната връща `std::optional<std::string>`, тоест празна стойност при валидни данни и съобщение при невалидни, което `handler`-ът превръща в отговор 400. Така проверката на входа стои в модела, а превръщането ѝ в отговор стои в `handler`-а. `User::to_json` изрично пропуска `password_hash`, с коментар на реда. Филтрирането е също модел: `handler`-ът чете параметрите на заявката и попълва `TaskFilter`, а сортирането и странирането се извършват в услугата.

Съставянето става в конструктора на `Server` и редът има значение, отбелязан с коментар в изходния текст: първо `TaskHub`, после `UserService`, накрая `TaskService`, който получава `hub`-а по препратка. Тази препратка е единствената връзка между HTTP частта и канала за известия.

7.1.3. Каналът за известия и връзката му с HTTP маршрутите

Каналът не е паралелен на API-то, а е закачен зад него. `TaskService::create`, `update` и `remove` извикват съответно `broadcast_created`, `broadcast_updated` и `broadcast_deleted`, след като са освободили мутекса. Тоест известието тръгва от услугата, а не от `handler`-а, и се появява независимо от това по кой маршрут е дошла промяната.

`TaskHub` не изпраща сам. Той поставя съобщението в опашка за всяка от отворените връзки, а изпращането се извършва от корутината на самата връзка: тя изпразва опашката си, преди да изчака следващото съобщение. Следствието е неудобно и трябва да се каже. Мълчалив клиент получава известието едва когато неговият цикъл се върне от изчакването, тоест едва когато сам изпрати нещо. Кодът на страницата решава това, като изпраща `ping`

на всяка секунда, и коментарът в `static/js/app.js` го заявява дословно: ping-ът е там, за да накара сървъра да изпрати чакащите съобщения. Тоест каналът, наречен „в реално време“, на практика се допитва с честота 1 Hz. Известието не се разпространява от сървъра, а се изтегля от клиента. Това е дефект в примера, не свойство на библиотеката, но е дефект, който примерът не показва сам.

7.1.4. Middleware

Регистрирани са три компонента, в реда, в който се изпълняват. `request_logger` е най-външният: той засича времето преди и след извикването на `next` и отпечатва състоянието на отговора, продължителността, метода и пътя. `coroute::compression` следва. `coroute::static_files` с префикс `/static`, изключено изброяване на директории и `max_age` от 3600 секунди е третият.

Четвъртият и петият не са регистрирани. `src/middleware/auth.cpp` съдържа `require_auth` и `load_user`, написани и компилирани, но `Server` не извиква нито едното. Вместо това всеки `handler`, който изисква сесия, повтаря локално една и съща проверка върху бисквитките. Трите маршрута, които променят задача, я правят с еднакъв код, а `/api/me` има свой вариант на същото.

Това е находка, а не оформление. Приложението е предпочело повторение пред `middleware`, защото `app.use` прилага компонента върху всички маршрути, а приложението няма механизъм за прикачване на `middleware` само към една група. Тоест удостоверяването не се вписва в единствения предвиден начин за композиция и авторът го заобиколи, без да отбележи, че го заобикаля. Точно това ограничението в раздел 7.5 предупреждава, че се случва.

7.2. Трите твърдения, проверени в код

Че добавянето на протокол не се вижда в приложението. `Handler`-ите в `src/handlers` не съдържат условия по протокол. Търсене на `http_version`, HTTP/2 и HTTP/3 в целия `examples/Project/src` не връща нито едно съвпадение. Един и същи `handler` обслужва заявка, пристигнала по HTTP/1.1, по HTTP/2 или по HTTP/3, защото и трите достигат до него през един и същ маршрутизатор.

```
1 app.get("/api/tasks/{id}",
2     [&task_service](Request& req) -> Task<Response>
3     {
```

```

4      auto id_result = req.param<int64_t>(0);
5      if (!id_result) co_return json_error(400, "Invalid task ID");
6
7      auto task = task_service.find(*id_result);
8      if (!task) co_return json_error(404, "Task not found");
9
10     co_return Response::json(task->to_json().dump());
11 }

```

Листинг 11: Handler без нито едно условие по протокол

Листингът е съкратен по фигурните скоби на едноредовите тела и по пълните имена на пространствата; останалото е дословно. В него няма нищо, зависещо от транспорта: нито версия, нито заглавна част, нито поточен идентификатор. Твърдението от глава III е проверено от страната на потребителя, а не на реализацията, и потвърдено от проверката в раздел 7.4, където един и същи handler отговаря и по трите протокола.

Че разделянето на API от изгледи не води до дублиране. Изгледът получава данните си от същия маршрут, който обслужва и API клиентите, и извикването не напуска процеса.

```

1  app.view<ListingVm>("/",
2    [&app](Request&) -> View<ListingVm>
3    {
4      // Същият маршрут, който обслужва и външните клиенти.
5      Response resp = co_await app.fetch_get("/api/items");
6
7      std::vector<std::string> items;
8      if (resp.status() == 200)
9      {
10         items = nlohmann::json::parse(resp.body())
11             .get<std::vector<std::string>>();
12      }
13
14      co_return ViewResult<ListingVm>{
15         .templates = ViewTemplates{"listing"},
16         .model = ListingVm{.title = "Items from API",
17                           .items = std::move(items)}};
18 }

```

Листинг 12: Изглед, който взема данните си от API маршрут в същия процес

App::fetch съставя заявка, съпоставя я с маршрутизатора, изпълнява веригата от middleware и връща отговора. Няма сокет, няма системно извикване, няма кръгово пътуване.

Пресичането обаче не е безплатно и цената трябва да се назове точно: тялото се

сериализира до JSON от API маршрута и се разбира обратно от изгледа. Спестява се транспортът, не сериализацията. Прекият вариант, при който изгледът извиква услугата и подминава маршрута, също е възможен и спестява и двете; примерното приложение подава UserService на регистрацията на страниците именно за това, но текущият handler за / чете само бисквитката и не докосва услугата, така че параметърът стои маркиран `[[maybe_unused]]`. Тоест примерното приложение показва разделянето на маршрутите, но не и вземането на данни от услуга вътре в изглед. Другият пример, `examples/view_example`, също не го показва: там изгледът минава през `fetch_get` към собствения си API маршрут, тоест през извикване без сокет, а не през пряко обръщение към услуга. Двата пътя се различават и разликата се назовава тук, защото по-ранна редакция на този раздел ги разменяше.

Че моделът на изгледа остава прост. Моделът е обикновена структура с написана на ръка функция за преобразуване към JSON. Няма базов клас, няма макрос за регистрация и няма нищо, което да свързва структурата с библиотеката. Отложена стойност се добавя като поле от тип `Deferred<T>`. Промяната е следната, и това е целият ѝ обем:

```
1 struct Dashboard
2 {
3     std::string title;
4     int visitors_now = 0;
5     Deferred<std::string> slow_report; // първи ред
6 };
7
8 void to_json(nlohmann::json& json, const Dashboard& model)
9 {
10     json = nlohmann::json{
11         {"title", model.title},
12         {"visitors_now", model.visitors_now},
13         {"slow_report", model.slow_report}}; // втори ред
14 }
```

Листинг 13: Пълната промяна при добавяне на отложено поле

Строго погледнато редовете са два, а не един, и вторият съществува само защото преобразуването е написано на ръка. Той не носи знание за отлагането: изглежда точно като реда над себе си.

Останалото се случва без участие на модела. Преобразуването на `Deferred<T>` проверява дали стойността е готова; ако е, я записва, а ако не е, регистрира гнездо в

събирача, който е активен по време на изобразяването, и вписва запълнител. Тоест моделът не декларира кои от полетата му са отложени и не съществува списък, който да остане несинхронизиран с него.

Има и трети случай, който е избор, а не пропуск: ако стойността не е готова и няма активен събирач, полето се сериализира като `null`. Никой не предава по-късните части, така че запълнител би бил обещание, което страницата никога няма да изпълни.

7.3. Отделни примери за отделни свойства

Освен цялото приложение съществуват и единадесет малки примера, всеки от които изолира по едно свойство. Таблица 21 ги изброява заедно с размера им, защото главата твърди, че те служат за изпълним отговор на въпроса „колко код струва това“, а такъв отговор без числа не е отговор.

Таблица 21. Отделните примери и техният размер

Пример	Какво изолира	Редове
<code>static_server</code>	Статични файлове, ETag, изброяване на директории.	75
<code>raw_benchmark</code>	Цикълът за вход и изход и корутините без HTTP слоя отгоре.	85
<code>echo_server</code>	Тяло на заявката, спиране по сигнал.	87
<code>http2_server</code>	HTTP/2 по TLS и по открит текст.	88
<code>template_server</code>	Шаблони и потребителска функция в тях.	91
<code>https_server</code>	TLS със сертификат от команден ред.	95
<code>hello_world</code>	Маршрути, типизирани параметри, компресия.	114
<code>streaming_server</code>	Отговор на части.	168
<code>range_server</code>	Частични отговори върху файл от 1 MB.	185
<code>websocket_server</code>	Ехо и разпращане към няколко клиента.	225
<code>benchmark_server</code>	Минимален сървър, чиито настройки са флагове.	482

Числата са редове в `main.c` и не са мярка за нужния код. Те включват коментарите и текста, който всеки пример отпечата при пускане, а в няколко от тях този текст е съществена част от файла: `hello_world` изразходва десетина реда само за да изброи адресите, които читателят да опита. Тоест таблицата дава горна граница, при това щедро.

Долната граница е по-къса от всеки ред в таблицата. Пълен сървър, който

отговаря по HTTP/1.1, са три израза: конструиране на App, един `app.get` с ламбда, връщаща `Response::ok`, и `app.run` с номер на порт. Всичко останало в `hello_world` е допълнителните свойства, които той показва.

Два от примерите не се строят винаги: `http2_server` изисква `COROUTE_ENABLE_HTTP2`, а `https_server` изисква `COROUTE_ENABLE_TLS`. Останалите девет се строят безусловно.

Двата примера за измерване са различни по предназначение от останалите девет. `raw_benchmark` заобикаля HTTP слоя и работи направо с цикъла за вход и изход на библиотеката, чийто backend се избира при изпълнение, за да покаже колко струва слой отгоре. При `benchmark_server` всяка настройка е флаг от командния ред, а не опция при компилиране, защото различието между два отделно компилирани двоични файла е по-голямо от разликата, която се измерва. Сред флаговете е и `--no-detect`, тоест рамото, спрямо което се измерва демултиплексирането.

7.4. Три изпълними проверки

Отделно от примерите съществуват три проверки, използвани в настоящата работа като доказателство, а не като демонстрация. Всяка от тях се състои от малък сървър и скрипт, който го пуска и проверява изхода. И трите живеят в `tests/integration`.

Един екземпляр, един номер на порт, три протокола. `verify_multiprotocol.sh` строи `http3_app.crr`, който регистрира два маршрута и включва TLS и HTTP/3 върху едно повикване за пускане, и го пуска с четири работни нишки, за да се използва сегментираният път. После проверява девет неща: че `curl` по HTTP/1.1 получава 200 и тялото, че същото важи по HTTP/2 на същия порт, и че референтният клиент на `ngtcp2` получава договорен ALPN h3, състояние 200 и същото тяло по UDP. Проверява се и че заглавната част `Alt-Svc` присъства и в двата TCP отговора, защото без нея UDP страната е достижима на теория и не се използва на практика: браузър не опитва HTTP/3, ако не му е казано, че съществува [6].

Накрая скриптът брои дескрипторите с `ss` и отпечатва броя на TCP и UDP слушателите заедно с броя на работните нишки. Твърдението, което тази бройка проверява, е формулирано в самия скрипт и е по-слабо от възможното: един слушащ дескриптор на транспорт на работна нишка, независимо колко протокола се обслужват, а

не един дескриптор изобщо.

HTTP/3 срещу чужда реализация. `verify_http3.sh` прави същото за HTTP/3 сам, срещу референтния клиент на `ngtcp2`. Проверяват се три низа и всеки от тях се проваля различно: липсващият договорен ALPN значи грешен TLS контекст, липсващото състояние 200 значи, че заявката никога не е стигнала до handler, а липсващото тяло значи, че отговорът е бил оформен, но не и изпратен. Тялото е под шестнадесет знака нарочно, защото клиентът го отпечатва като шестнадесетичен дъмп с ASCII колона, широка шестнадесет байта.

Скриптът съществува вместо случай в `ctest` по причина, записана в него: нужна е втора реализация на QUIC, а тя не е зависимост при строене. Протоколен код, който никога не е завършвал ръкостискане срещу нещо, което не е писал сам, не заслужава доверие.

Че обвивката излиза преди стойността. `verify_deferred.sh` строи `deferred_app.cpp`, който изобразява страница с едно отложено поле, чието изчисляване отнема зададен брой милисекунди. Скриптът извиква `curl` с изключено буфериране, отпечатва до всеки пристигнал ред времето на пристигането му, и после сравнява два момента: този на реда с готовата част на страницата и този на реда, който предава стойността. Проверката минава, ако разликата е поне половината от зададеното закъснение.

Останалите шест проверки са върху съдържанието: че страницата носи и двата клиентски пътя, този по атрибут за страница без JavaScript и този през Promise. Един детайл в скрипта е поправка на самата измервателна процедура: търси се извикването на функцията, която предава стойността, а не самото ѝ име, съвпадащо и с дефиницията ѝ в главата на документа. Дефиницията пристига преди обвивката и първоначално правеше измерената разлика отрицателна.

Какво не доказват. И трите са скриптове, пускани на ръка под Linux, и нито един не влиза в непрекъснатата интеграция. Два от тях изискват локално построени OpenSSL, `ngtcp2` и `nghttp3` под отделен префикс, тоест не вървят в обикновената задача за тестване; проверката за отлагането не изисква нито TLS, нито HTTP/3 и минава по открит текст. При двата, които използват TLS, проверката на сертификата е изключена, защото

сертификатът е самоподписан. Всичко се случва върху локалния интерфейс, тоест без загуба на пакети, без пренареждане и без реална латентност. Проверката за отлагането минава по праг от половината закъснение, а не по точна стойност, и измерва времето с разделителна способност от милисекунда.

7.5. Ограничения на тази проверка

Използваемостта тук се преценява по един източник, и това трябва да се каже.

Приложението е написано от автора на библиотеката. Такъв автор знае кои интерфейси са неудобни и ги заобикаля, без да забележи, че ги заобикаля. Следователно настоящата глава показва, че решенията *могат* да бъдат използвани, а не че са удобни за някой друг.

Двете заобикаляния, описани по-горе, са точно този случай. Нито преименуваният маршрут, нито повтореното удостоверяване са отбелязани в изходния текст като компромис; открити са при четенето за настоящата глава, не при писането. Външен потребител не би ги заобиколил мълчаливо, а би ги съобщил като дефекти.

Приложението не изчаква истински вход-изход. Всичко е в паметта, под мутекс. Тоест случаят, в който архитектурата има най-голяма вероятност да заболи, изглед, изчакващ бавна заявка към база от данни, е изпробван само от `deferred_app.cpp`, където бавната стойност е нишка, спяща зададен брой милисекунди.

Има и по-груби липси. Приложението няма собствени тестове, макар да има директории за тях. Обработката на грешки е незапълнена: функцията, която трябва да ги регистрира, е празна, с коментар, че за това липсва поддръжка в библиотеката, тоест приложението е установило липсата и не я е поправило. Хеширането на паролите е заместител, отбелязан в изходния текст като негоден за употреба.

Истинската проверка на използваемостта изисква независим потребител, и тя не е извършена. Записва се като ограничение, а не като бъдеща работа, защото бъдещата работа звучи като обещание, а това е липса.

7.6. Изводи

Приложението показва, че трите архитектурни решения се понасят взаимно в код, който не е измерителен стенд: handler-ите не знаят по кой протокол е дошла заявката, изгледите вземат данни от същите маршрути като външните клиенти без излизане от

процеса, а отложеното поле е добавка от два реда към обикновена структура, само единият от които говори за отлагане.

Приложението намери и три неща, които не работят: маршрут, преименуван заради параметър; удостоверяване, което не се вписва в единствения предвиден начин за композиция; канал за известия, който се допитва вместо да разпраца. Нито едно от трите не се вижда от страната на реализацията.

Това е по-слабо твърдение от „библиотеката е лесна за използване“ и е точно толкова, колкото един автор може да провери сам за собствената си работа.

ЗАКЛЮЧЕНИЕ

Целта, формулирана в раздел 1.8, съдържа две части: да се разработи архитектура, при която броят на слушащите дескриптори не зависи от броя на обслужваните протоколи, и да се установи цената на това решение чрез измерване. Заключението разделя двете, защото те са в различно състояние.

Първата част е изпълнена. Втората е изпълнена частично, и разделението между направеното и ненаправеното минава на определено място, което разделят по-долу назовава точно.

По задачите

Таблица 22. Състояние на задачите от раздел 1.8

№	Задача	Състояние
1	Формална постановка на минимизацията	Доказана, раздел 2.5; поправена от измерването, раздел 6.12
2	Разпознаване на протокола след приемане	Реализирано, раздел 3.1
3	Преносимо разпределяне на QUIC връзки	Реализирано, раздел 3.2
4	Отложено получаване с типизиран договор	Реализирано, раздел 3.3
5	Втори механизъм за вход и изход под Linux	Реализирано, раздел 4.1
6	Методика за измерване	Формулирана, глава V
7	Провеждане на измерването	Проведено под Windows за HTTP/1.1, глава VI; предстои под Linux и macOS

Шест от седемте задачи са изпълнени изцяло и всяка от тях сочи мястото, на което може да бъде проверена. Седмата е изпълнена за една платформа и един протокол от три, тоест кампанията е проведена, а не описана, но обхватът ѝ е по-тесен от предвидения.

По хипотезите

X1 не се отхвърля, и този път твърдението има покритие. При пет предложени натоварвания от 10 000 до 70 000 заявки в секунда разликата в 99-ия персентил между сървъра с класификация и същия двоичен файл без нея е между 0.61% и 2.50%, а

доверителният интервал върху разликата съдържа нулата при всяко от тях. Прагът за докладваемост е пет процента и е обявен преди събирането на данните.

Значението на непотвърждаването зависи от това какво постановката е била в състояние да види, и затова разделителната ѝ способност се съобщава заедно с резултата: полуширочината на интервала върху разликата е между 2.47% и 5.63% от медианата. Тоест при четири от петте натоварвания полуширочината на интервала е под собствения праг на постановката и непотвърждаването там е твърдение за сървър, а не за броя изпълнения; само при 25 000 заявки в секунда тя го надхвърля и непотвърждаването е твърдение за извадката.

Обхватът на твърдението е по-тесен от хипотезата. То се отнася за цената по заявка. Цената на връзка остава неизмерена от този проект, защото той поддържа шестдесет и четири устойчиви връзки за двадесет секунди и през тях преминават от двеста хиляди заявки при най-ниското натоварване до милион и четиристотин хиляди при най-високото, което амортизира цената на връзка под всяка разделителна способност, вижте раздел 6.4. Тя е измерена от проекта churn на същата кампания, 400 изпълнения при темп на пристигане на връзки. Това е недостатък на постановката, открит от анализа на резултата.

X2, X3 и X4 остават непроверени, по три различни причини. X2 изисква HTTP/3 и предизвикана миграция между два адреса, което на един loopback адрес е невъзможно, и се провежда под Linux. X3 изисква генератор, който изобразява страница и разделя времето до първия байт от времето до последния, а loadgen не прави това. X4 не изисква нищо, което липсва: обхождането по брой маршрути може да се проведе на същата машина и не е проведено поради разпределението на времето.

Всяка от трите остава формулирана така, че да може да бъде отхвърлена, и измерването, което би я отхвърлило, е записано заедно с нея.

Какво е установено независимо от измерването

Част от твърденията в работата не зависят от кампанията, защото не са твърдения за производителност. Те се изброяват тук, за да не бъдат подминати заедно с онези, които зависят.

Предложението за дескрипторите е доказано, а не измерено, и е поправено от броенето. Равенството $L = A \cdot |T|$, в което A е броят приемащи дескриптори, изискван от

модела на приемане на платформата, следва от несечащите се множества от начални октети и от предположенията, изброени в раздел 2.5. По-ранната формулировка пишеше $W \cdot |T|$ безусловно, тоест толкова слушателя, колкото са работните нишки. Преброяването на дескрипторите на работещ процес я опроверга: под Windows процесът държи един слушач дескриптор при една, две, четири и осем работни нишки, защото IOCP използва модел с един споделен слушател. Твърдението, което работата защитава, е за независимостта от броя обслужвани приложни протоколи и се запазва непроменено. Сгрешен беше множителят пред нея, и измерването е това, което го хвана.

Трите протокола се обслужват от един екземпляр и един номер на порт. Проверено е от край до край, включително установяване на връзка по HTTP/3 срещу еталонен клиент. Твърдението е за функция, не за скорост, и не се променя от резултата на кампанията.

Правилото за собственост върху кадъра е правило за проектиране. Формулировката от раздел 4.2 е получена след като една и съща надпревара беше допусната три пъти на три независими места. Следствието е, че интерфейсът трябва да прави грешката невъзможна, и то важи независимо от това какво показва измерването.

Методиката е реализирана и се самопроверява. Инструментариумът съществува, изпълнява правилата, които глава V описва, и отказва да работи, когато условията му не са изпълнени. Един от отказите му е сработил през работата: отказът да се приеме число от виртуализирана машина. Правилото за докладваемост беше приложено и към процесорното време и не обяви разликата за докладваема при нито едно от петте натоварвания, което наложи да се обясни защо резултатът не е това, което изглежда, вижте раздел 6.7. Отказът да се слеят две кампании с различен отпечатък на средата остава в сила, но не е бил задействан.

Решенията са използвани в приложен код. Глава VII показва приложение, в което handler-ите не съдържат условия по протокол, изгледите вземат данни от същите услуги като API-то без излизане от процеса, а отложеното поле е добавка от един ред към обикновена структура.

Ограничения

Ограниченията се изброяват тук отново и накратко, защото заключение, което ги пропуска, ги кани като въпроси.

Кампанията е за HTTP/1.1 в явен вид и през TLS, върху една машина под Windows, при която сървърът и генераторът делят хоста; таблиците за X1 са за връзки в явен вид. Абсолютните стойности са свойство на разделянето на ядрата и не се съобщават като характеристика на реализацията. Сравненията не са, и само те се твърдят.

Проектът h1-deer не измерва цената на връзка, както е обяснено по-горе. Поправката е натоварване с темп на пристигане на връзки, а не само на заявки, и тя вече е приложена: под Windows проектът churn я проведе в същата кампания, 400 изпълнения. Остава повторението ѝ през мрежов път.

Персентилът 99,9 на този хост е двумодален по причина извън сървъра и затова не е използван за сравнение, вижте раздел 6.8. Това е ограничение на машината, а не на метода, и се съобщава, защото опашката е явлението, което един сървър трябва да описва.

Използваемостта е преценена по един източник, който е авторът на библиотеката. Независим потребител не е привлечен, тоест глава VII показва, че решенията могат да бъдат използвани, а не че са удобни.

За HTTP/3 не съществува зрял генератор с постоянен темп на заявките в отворен цикъл. Горните персентили за този протокол ще идват само от затворен цикъл и се означават като време за обслужване, а не като време за отговор.

Под Windows липсва програмируем еквивалент на tc, а сегментирането на UDP се различава съществено. Затова твърденията за производителност на HTTP/3 важат за Linux, а измерването под Windows остава функционално.

Сравнението с насочване в ядрото не е проведено. Работата твърди преносимост на метода от раздел 3.2 и предвижда да измери колко често препращането се налага, но не твърди превъзходство над eBPF.

Бъдеща работа

Четири посоки следват пряко от състоянието по-горе, и всяка от тях е дефинирана, а не пожелана.

Кампанията под Linux. Изисква неvirtуализирана машина с двойка мрежови

пространства. Планът, редът на изпълнение и правилата за допустимост са налични и вече изпълнени веднъж на друга платформа, тоест това е изпълнение, а не проектиране. Тя носи сравнението между трите протокола, готовността срещу завършването и X2.

Натоварване с темп на пристигане на връзки. Това е поправката на слепотата, открита в раздел 6.4. Тя не е промяна, а вече налична настройка от двете страни: `--max-requests 1` на сървъра и `--reconnect` на генератора. Проведена е под Windows от проекта `churn`, 400 изпълнения; остава повторението ѝ през мрежов път, което под Windows беше спряно от защитната стена на хоста.

Рамо с насочване в ядрото. След като делът на препратените пакети бъде измерен, сравнението с програма на eBPF става смислено. До това измерване то би било сравнение без известна величина от едната страна.

Генератор с постоянен темп за HTTP/3. Липсата му е ограничение на инструментариума в целия отрасъл, не само тук, и премахването му би направило горните персентили за HTTP/3 сравними с тези за останалите два протокола.

Обобщение

Работата премества едно решение от конфигурацията на сокета към момента след приемането на връзката, и показва, че това е достатъчно, за да отпадне зависимостта между броя на слушащите дескриптори и броя на обслужваните протоколи. Постановката е доказана, реализацията работи по трите протокола от един номер на порт, а броенето на дескриптори на работещ процес потвърждава независимостта от броя протоколи и поправя множителя в по-ранната ѝ формулировка.

Цената по заявка е измерена и е под прага, който работата обяви предварително: разликата е между 0.61% и 2.50% при праг от пет процента. Разделителната способност на постановката е под прага при четири от петте натоварвания и над него при 25 000 заявки в секунда. Цената на връзка е измерена от проекта `churn`, 400 изпълнения, а резултатите му остават за попълване.

СПРАВКА ЗА ПРИНОСИТЕ

Приносите са разделени по вид, а видът определя и начина, по който всеки от тях се защитава. Научните приноси се защитават с доказателство. Научно-приложните се защитават с измерване. Приложните се защитават с работеща реализация.

Затова към всеки принос е посочено къде е обоснован и *с какво*. Където обосновката е измерване, което още не е извършено, това е записано изрично, а не премълчано.

Научни приноси

П1. Формулирана и доказана е минимизацията на слушащите дескриптори при демултиплексиране след приемане на връзката. Броят на слушащите дескриптори е изразен като $L = A \cdot |T|$, тоест като функция на броя транспорти и на модела на приемане на платформата, но *независим* от броя на обслужваните протоколи на приложно ниво. Доказателството се опира на несечащите се множества от начални октети на TLS запис и на текстов метод, и е дадено в раздел 2.5.

Приносът е в постановката, не в наблюдението. Твърдението „един сокет вместо няколко“ съществува като инженерна практика; тук то е сведено до равенство с явно изброени предположения, което го прави оборимо. В същия раздел е записано и какво предложението не твърди: множителят пред $|T|$ остава и не е нула.

Състояние на обосновката: доказано, и проверено чрез броене на дескрипторите на работещ процес, отчетени от ядрото, раздел 6.12. Броенето потвърди независимостта от броя протоколи и опроверга по-ранната редакция на самото равенство, която пишеше $W \cdot |T|$ безусловно. Поправката произхожда от измерването и е пример за това, което предварително обявеното броене е предназначено да прави.

П2. Формализирана е границата на собственост върху кадъра на корутина при асинхронно подаване. Установено е, че от момента, в който подадена операция стане годна за завършване, кадърът на спряната корутина принадлежи на подновяващата страна, тоест всяко обръщение към него от подаващата страна след този момент е надпревара, която подаващата страна не може да спечели надеждно.

Формулировката е дадена в раздел 4.2, а нейната стойност е практическа: получена е след като една и съща грешка беше допусната три пъти на три независими места в кода.

Следствието е правило за проектиране на интерфейс, а не съвет за внимание.

Научно-приложни приноси

П3. Разработен е метод за разпределяне на QUIC връзки между работни нишки чрез кодиране на индекса в идентификатора на връзката, без зависимост от eBPF. Методът е описан в раздел 3.2. Той е приложим и на платформи без eBPF, тоест там, където решението на nginx и Angie е недостъпно по принцип.

Състояние на обосновката: реализиран и проверен функционално. Делът на пакетите, изискващи препращане, **още не е измерен**, тъй като изисква принудителна миграция в отделни мрежови пространства. До извършването на това измерване приносът е за наличие на преносим метод, а не за неговата цена.

П4. Разработен е типизиран договор за отложени стойности, обхващащ компилиран и скриптов език. Стойност, която още не е пристигнала, се изразява като тип от страната на сървъра и получава съответствие от страната на клиента, което кодът на страницата може да изчака и да композира. Описан е в раздел 3.3.

Разграничението спрямо известното е изрично: ранното изпращане на части от страница не е нов похват. Новото е, че незавършеността на стойността е изразена в типовата система от двете страни на връзката, а не като конвенция за подмяна на елемент.

Състояние на обосновката: реализиран и проверен функционално, включително наличието на резервен път без JavaScript. Числената разлика между изчакано и отложено изобразяване **още не е измерена**. Пречката не е машината, а генераторът: за нея е нужен клиент, който изобразява страница и разделя времето до първия байт от времето до последния, а наличният генератор измерва само заявки.

П5. Предложена е методика за измерване на многопротоколни сървъри. Методиката, изложена в глава V, съдържа няколко елемента, всеки от които премахва определен вид тиха грешка: единен генератор за сравнения между протоколи, разделяне на времето за обслужване от времето за отговор, отпечатък на средата, спиращ кампанията при промяна, предварително обявени критерии за отхвърляне и изискването двете рамена на всяко сравнение да идват от един двоичен файл.

Състояние на обосновката: методиката е формулирана, инструментариумът е реализиран, и двете са приложени върху една кампания с три проекта: 250, 500 и 400

изпълнения, глава VI. Стойността ѝ се показва от това, което тя хвана, а не от това, че съществува: отказът да приеме число от виртуализирана машина, разделителната способност, която обяви една от петте клетки за недостатъчна за собствения ѝ праг, и процесорното време, при което правилото за докладваемост не сработи при нито едно от петте натоварвания, макар медианата на рамото с класификация да е по-висока навсякъде, и това наложи да бъде обосновано, раздел 6.7.

Приложни приноси

П6. Реализирана е библиотека, обслужваща HTTP/1.1, HTTP/2 и HTTP/3 от един екземпляр, един номер на порт и един набор от маршрути. Заявките от трите протокола минават през един и същ маршрутизатор, една и съща верига от middleware и едни и същи handler-и. Проверено е от край до край.

П7. Реализиран е механизъм за вход и изход върху epoll, съществуващ съвместно с този върху io_uring на една и съща машина. Това превръща вида на интерфейса, готовност или завършване, в независима променлива, каквато той не е, когато е обвързан с операционната система. Без него сравнението между готовност и завършване би било сравнение между две различни програми. Самото сравнение не е в глава VI: тя го отчита сред планираните, раздел 6.14, защото двата механизма съществуват на една и съща операционна система само под Linux.

П8. Реализиран е инструментариум за възпроизводими измервания. Обхваща отпечатък на средата, правила за допустимост, схема на записа, ред на изпълнение и преобразуване на приетите изпълнения в стойностите, цитирани в текста, без ръчно преписване на число.

Какво не се твърди

Разделът съществува, защото преувеличеното твърдение е това, което бива атакувано.

Не се твърди, че броят на слушащите дескриптори е константа. Множителят по работни нишки остава и е независима ос.

Не се твърди, че демултиплексирането е безплатно. Твърди се, че не изисква

дескриптор. Дали цената му е откриваема в измерване, се решава в глава VI.

Не се твърди, че препращането в потребителско пространство е по-бързо от насочването в ядрото. Твърди се, че е преносимо, и се измерва колко често изобщо се налага.

Не се твърди приоритет върху алгоритъма за съпоставяне. Той е публикуван по-рано [39]; настоящата работа проверява поведението му в контекста на пълна заявка.

Не се твърди, че трите поправени надпревари са три открити дефекта. Една беше наблюдавана; две бяха затворени по разсъждение и не бяха възпроизведени. Разграничението е направено и в раздел 4.2.

ПУБЛИКАЦИИ ПО ДИСЕРТАЦИЯТА

Публикувани

1. Станков, И., Цветанов, А. *Matching Text from Start to Finish Against Multiple Regular Expressions*. 32-ра Национална конференция с международно участие „Телеком 2024“, София, ноември 2024, с. 21–22, IEEE [39].

Отношение към дисертацията: алгоритмът за съпоставяне, върху който стъпва маршрутизаторът. Изложен е в раздел 2.3, а поведението му в контекста на пълна заявка се проверява в глава VI. Приносът на настоящата работа не е върху самия алгоритъм, а върху приложението и проверката му при маршрутизиране.

Подготвяни

Списъкът се води отделно и се поддържа в съответствие с приносите от справката. Всяка тема е изведена от работа, която дисертацията извършва така или иначе, и е цитируема в нея.

2. *Демултиплексиране на няколко протокола върху минимален брой сокети.* Съответства на принос П1. Проверява твърдението, че класификацията по първия октет не струва измерима пропускателна способност или латентност спрямо отделни слушащи сокети.
3. *Преносимо разпределяне на QUIC връзки по идентификатор без eBPF.* Съответства на принос П3. Проверява доколко насочването в ядрото би спестило работа, като измерва колко често препращането изобщо се налага.
4. *Готовност срещу завършване в една кодова база: epoll и io_uring.* Съответства на принос П7. Сравнение при еднакви ядро, машина, маршрути и код, при което единствената променлива е видът на интерфейса.
5. *Изчакано срещу отложено получаване на данни при изобразяване от сървър.* Съответства на принос П4.
6. *Възпроизводима методика за измерване на многопротоколни сървъри.* Съответства на принос П5.

Забележка за състоянието. Към момента на съставяне на настоящия текст е публикувана една работа. Останалите теми са планирани и не се представят като публикации. Изискването по чл. 9(д) от процедурите се изпълнява с действително публикувани или приети за печат работи, а списъкът по-горе ясно разделя двете състояния, вместо да ги смесва.

Забележка за съавторство. Публикуваната работа е в съавторство. Съгласно чл. 9(з) член на журито не може да бъде съавтор на докторанта в публикации, включени в дисертацията, с изключение на научния ръководител. Това ограничение се отчита при съставянето на журито.

ЛИТЕРАТУРА

- [1] Stephan Friedl, Andrei Popov, Adam Langley, and Emile Stephan. Transport layer security (tls) application-layer protocol negotiation extension. RFC 7301, IETF, 2014. <https://tools.ietf.org/html/rfc7301>.
- [2] Bradley Efron. Better bootstrap confidence intervals. *Journal of the American Statistical Association*, 82(397):171–185, 1987.
- [3] Roberto Peon and Herve Ruellan. Hpack: Header compression for http/2. RFC 7541, IETF, 2015. <https://tools.ietf.org/html/rfc7541>.
- [4] Mike Bishop. HTTP/3. RFC 9114, Internet Engineering Task Force, June 2022.
- [5] Jana Iyengar and Martin Thomson. QUIC: A UDP-based multiplexed and secure transport. RFC 9000, Internet Engineering Task Force, May 2021.
- [6] Mark Nottingham, Patrick McManus, and Julian Reschke. HTTP alternative services. RFC 7838, Internet Engineering Task Force, April 2016.
- [7] R. Fielding, M. Nottingham, and J. Reschke. HTTP/1.1. Request for Comments RFC 9112, Internet Engineering Task Force, 6 2022. Отменя RFC 7230, което на свой ред отменя RFC 2616.
- [8] Mike Belshe, Roberto Peon, and Martin Thomson. Hypertext transfer protocol version 2 (http/2). RFC 7540, IETF, 2015. <https://tools.ietf.org/html/rfc7540>.
- [9] Martin Thomson and Cory Benfield. Http/2. RFC 9113, IETF, 2022. <https://tools.ietf.org/html/rfc9113>.
- [10] Ian Fette and Alexey Melnikov. The websocket protocol. RFC 6455, IETF, 2011. <https://tools.ietf.org/html/rfc6455>.
- [11] Martin Thomson and Sean Turner. Using TLS to secure QUIC. RFC 9001, Internet Engineering Task Force, May 2021.
- [12] Caddy. Automatic https. <https://caddyserver.com/docs/automatic-https>. Достъпен през август 2026.

- [13] nginx. Module ngx_http_core_module: the listen directive. https://nginx.org/en/docs/http/ngx_http_core_module.html#listen. Достъпен през август 2026.
- [14] nginx. Configuring https servers: a single http/https server. https://nginx.org/en/docs/http/configuring_https_servers.html#single_http_https_server. Достъпен през август 2026.
- [15] Microsoft. Binding <binding>: Iis configuration reference. <https://learn.microsoft.com/en-us/iis/configuration/system.applicationhost/sites/site/bindings/binding>. Достъпен през август 2026.
- [16] Angie. Http core module: the listen directive. <https://en.angie.software/angie/docs/configuration/modules/http/>. Достъпен през август 2026.
- [17] H2O. Http/3 directives. https://h2o.example.net/configure/http3_directives.html. Достъпен през август 2026.
- [18] H2O. Base directives: the listen directive. https://h2o.example.net/configure/base_directives.html. Достъпен през август 2026.
- [19] Tao An. Drogon wiki: configuration file. <https://github.com/drogonframework/drogon/wiki/ENG-11-Configuration-File>. Достъпен през август 2026.
- [20] Michael Kerrisk. The SO_REUSEPORT socket option. LWN.net, March 2013. Достъпен на 29.08.2026.
- [21] Aurélien Buchet and Cristel Pelsser. An analysis of QUIC connection migration in the wild. *ACM SIGCOMM Computer Communication Review*, 55(1):3–9, 2025.
- [22] Jan Rüth, Ingmar Poesse, Christoph Dietzel, and Oliver Hohlfeld. A first look at QUIC in the wild. In *Passive and Active Measurement (PAM 2018)*, pages 255–268. Springer, 2018.
- [23] Tao An. Drogon: A c++14/17/20 based http web application framework. <https://github.com/drogonframework/drogon>, 2018. Accessed: 2024.
- [24] CrowCpp. Crow: A fast and easy to use microframework for the web. <https://crowcpp.org/>, 2014. Accessed: 2024.
- [25] CrowCpp. Crow guides: Ssl. <https://crowcpp.org/master/guides/ssl/>. Достъпен през август 2026.

- [26] Stryzhevskiy Leonid. Oat++: Modern web framework for c++. <https://oatpp.io/>, 2018. Accessed: 2024.
- [27] Stryzhevskiy Leonid. Oat++: simple api and async api. <https://github.com/oatpp/oatpp>. Достъпен през август 2026.
- [28] Vinnie Falco. Boost.beast: Http and websocket built on boost.asio. <https://www.boost.org/doc/libs/release/libs/beast/>, 2016. Accessed: 2024.
- [29] Pistache Project. Pistache: An elegant c++ rest framework. <http://pistache.io/>, 2015. Accessed: 2024.
- [30] TechEmpower. Web framework benchmarks. <https://www.techempower.com/benchmarks/>, 2024. Accessed: 2024.
- [31] Lewis Baker and Gor Nishanov. A unified executors proposal for c++. In *ISO/IEC JTC1/SC22/WG21*, 2019. P0443R14.
- [32] ISO/IEC. Programming languages – c++. International Standard ISO/IEC 14882:2020, International Organization for Standardization, 2020.
- [33] Rob von Behren, Jeremy Condit, and Eric Brewer. Why events are a bad idea (for high-concurrency servers). In *Proceedings of the 9th Workshop on Hot Topics in Operating Systems (HotOS IX)*, Lihue, HI, 2003. USENIX Association.
- [34] Matt Welsh, David Culler, and Eric Brewer. SEDA: An architecture for well-conditioned, scalable internet services. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP '01)*, pages 230–243. ACM, 2001.
- [35] Ken Thompson. Programming techniques: Regular expression search algorithm. *Communications of the ACM*, 11(6):419–422, June 1968.
- [36] Michael O. Rabin and Dana Scott. Finite automata and their decision problems. *IBM Journal of Research and Development*, 3(2):114–125, 1959.
- [37] Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4):481–494, October 1964.

- [38] Alfred V. Aho and Margaret J. Corasick. Efficient string matching: An aid to bibliographic search. *Communications of the ACM*, 18(6):333–340, June 1975.
- [39] Ivan Stankov and Alex Tsvetanov. Matching text from start to finish against multiple regular expressions. In *32nd National Conference with International Participation “Telecom 2024”*, pages 21–22, Sofia, Bulgaria, November 2024. IEEE. 979-8-3503-5500-0/24.
- [40] Eric Rescorla. The transport layer security (tls) protocol version 1.3. RFC 8446, IETF, 2018. <https://tools.ietf.org/html/rfc8446>.
- [41] Adam Langley, Alistair Riddoch, Alyssa Wilk, Antonio Vicente, Charles Krasnic, Dan Zhang, Fan Yang, Fedor Kouranov, Ian Swett, Janardhan Iyengar, Jeff Bailey, Jeremy Dorfman, Jim Roskind, Joanna Kulik, Patrik Westin, Raman Tenneti, Robbie Shade, Ryan Hamilton, Victor Vasiliev, Wan-Teh Chang, and Zhongyi Shi. The QUIC transport protocol: Design and Internet-scale deployment. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM ’17)*, pages 183–196. ACM, 2017.
- [42] Robin Marx, Joris Herbots, Wim Lamotte, and Peter Quax. Same standards, different decisions: A study of QUIC and HTTP/3 implementation diversity. In *Proceedings of the Workshop on the Evolution, Performance, and Interoperability of QUIC (EPIQ ’20)*. ACM, 2020.
- [43] Gaurav Banga, Jeffrey C. Mogul, and Peter Druschel. A scalable and explicit event delivery mechanism for UNIX. In *Proceedings of the USENIX Annual Technical Conference*, Monterey, CA, 1999. USENIX Association.
- [44] Jens Axboe. Efficient io with io_uring. *Linux Kernel Documentation*, 2019. https://kernel.dk/io_uring.pdf.
- [45] Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. Coverage-based greybox fuzzing as Markov chain. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS ’16)*, pages 1032–1043. ACM, 2016.
- [46] Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. AddressSanitizer: A fast address sanity checker. In *Proceedings of the USENIX Annual Technical Conference (ATC ’12)*, pages 309–318. USENIX Association, 2012.

- [47] Tatsuhiro Tsujikawa. nghttp2: Http/2 c library. <https://nghttp2.org/>, 2013. Accessed: 2024.
- [48] Bianca Schroeder, Adam Wierman, and Mor Harchol-Balter. Open versus closed: A cautionary tale. In *Proceedings of the 3rd Symposium on Networked Systems Design and Implementation (NSDI '06)*, San Jose, CA, 2006. USENIX Association.
- [49] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 265–276. ACM, 2009.
- [50] Charlie Curtsinger and Emery D. Berger. STABILIZER: Statistically sound performance evaluation. In *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '13)*, pages 219–228. ACM, 2013.
- [51] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous java performance evaluation. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '07)*, pages 57–76. ACM, 2007.